In [1]:

```
import pandas as pd
import matplotlib.pyplot as plt

# Create dataset
data = {
    'Year': list(range(2010, 2021)),
    'Job Postings': [150, 300, 450, 600, 800, 1200,
                     1600, 2100, 2700, 3400, 4200]
}

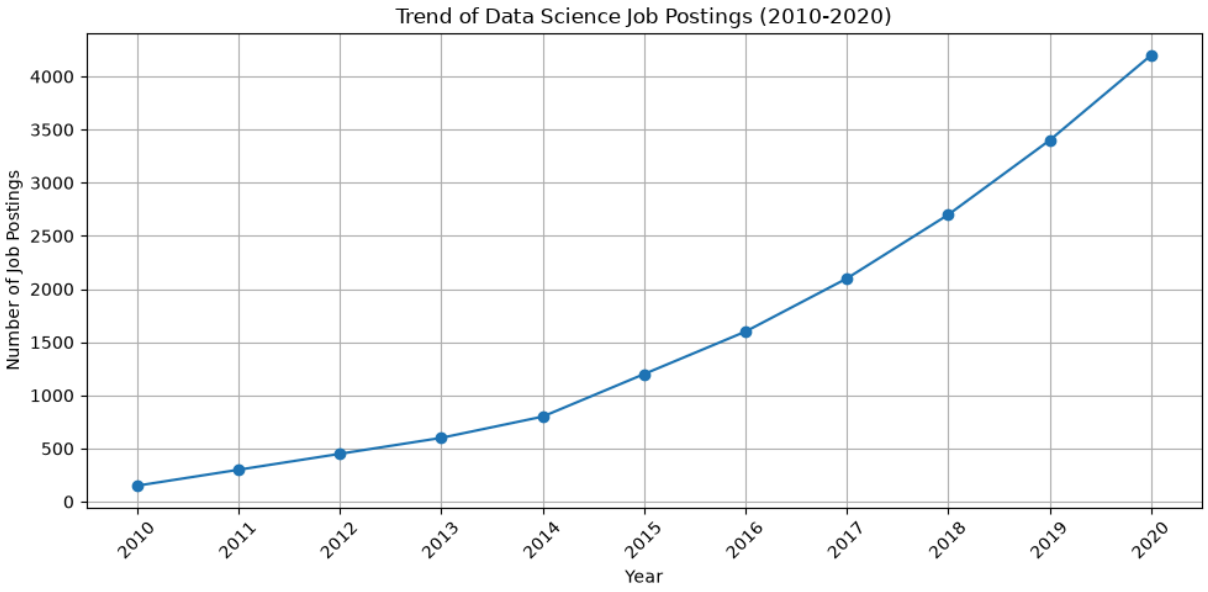
# Create DataFrame
df = pd.DataFrame(data)

# Display dataset
print("Data Science Job Postings:")
print(df)

# Plot the trend
plt.figure(figsize=(10, 5))
plt.plot(df['Year'], df['Job Postings'],
         marker='o', linestyle='-')

plt.title('Trend of Data Science Job Postings (2010-2020)')
plt.xlabel('Year')
plt.ylabel('Number of Job Postings')
plt.xticks(df['Year'], rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
Data Science Job Postings:
    Year  Job Postings
0   2010           150
1   2011           300
2   2012           450
3   2013           600
4   2014           800
5   2015          1200
6   2016          1600
7   2017          2100
8   2018          2700
9   2019          3400
10  2020          4200
```



In [2]:
```
import matplotlib.pyplot as plt

roles = ['Data Analyst', 'Data Engineer', 'Data Scientist',
         'ML Engineer', 'Business Analyst']

counts = [300, 500, 450, 200, 150]

plt.figure(figsize=(9, 5))
plt.bar(roles, counts)

plt.title('Distribution of Data Science Roles')
plt.xlabel('Role')
plt.ylabel('Count')
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```



In [3]:

```python
import pandas as pd
import json

# Structured data example
structured_data = pd.DataFrame({
    'ID': [1, 2, 3],
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35]
})

print("Structured Data:\n")
print(structured_data)

# Unstructured data example
unstructured_data = (
    "This is an example of unstructured data. "
    "It can be a piece of text, an image, or a video file."
)

print("\nUnstructured Data:\n")
print(unstructured_data)

# Semi-structured data example (JSON)
semi_structured_data = {
    'ID': 1,
    'Name': 'Alice',
    'Attributes': {
        'Height': 165,
        'Weight': 68
    }
}

print("\nSemi-structured Data:\n")
print(json.dumps(semi_structured_data, indent=4))
```

```
Structured Data:

   ID     Name  Age
0   1    Alice   25
1   2      Bob   30
2   3  Charlie   35

Unstructured Data:

This is an example of unstructured data. It can be a piece of text, an image, or a v
ideo file.

Semi-structured Data:

{
    "ID": 1,
    "Name": "Alice",
    "Attributes": {
        "Height": 165,
        "Weight": 68
    }
}
```

In [4]:
```python
from cryptography.fernet import Fernet

# Generate encryption key
key = Fernet.generate_key()
cipher_suite = Fernet(key)

# Original sensitive data
plain_text = b"Rajalakshmi Engineering College."

# Encrypt the data
cipher_text = cipher_suite.encrypt(plain_text)

# Decrypt the data
decrypted_text = cipher_suite.decrypt(cipher_text)

# Display the results
print("Original Data:", plain_text)
print("Encrypted Data:", cipher_text)
print("Decrypted Data:", decrypted_text)
```

```
Original Data: b'Rajalakshmi Engineering College.'
Encrypted Data: b'gAAAAABqyILuP1RrQQR_4B0uIRlz5979VK3-379PuO8yiW_TyCeVe4tIbGZw2PjdOE
ZKUDPqkER7bXfuj4aps2iXL7-tWU7nD4USgg6IbMXY28t8x97M8zBF0aXVPtrJqINUVjhl2ER6'
Decrypted Data: b'Rajalakshmi Engineering College.'
```

In [ ]:
```python

```

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create sample sales dataset
data = {
    'Date': ['2026-01-01', '2026-01-02', '2026-01-03',
             '2026-01-04', '2026-01-05', '2026-01-06',
             '2026-01-07', '2026-01-08', '2026-01-09'],
    'Product': ['Laptop', 'Mobile', 'Tablet',
                'Laptop', 'Mobile', 'Tablet',
                'Laptop', 'Mobile', 'Tablet'],
    'Sales': [50000, 20000, 15000, 55000, 25000,
              18000, 60000, 30000, 20000],
    'Quantity': [2, 4, 3, 2, 5, 4, 3, 6, 4],
    'Region': ['Chennai', 'Madurai', 'Trichy',
               'Chennai', 'Madurai', 'Trichy',
               'Chennai', 'Madurai', 'Trichy']
}

df = pd.DataFrame(data)

# Display the first five rows
print("First Five Rows:")
print(df.head())

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Handle missing values
df['Sales'] = df['Sales'].fillna(df['Sales'].mean())
df.dropna(subset=['Product', 'Quantity', 'Region', 'Date'],
          inplace=True)

# Convert Date column to datetime
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
df.dropna(subset=['Date'], inplace=True)

# Summary statistics
print("\nSummary Statistics:")
print(df.describe())

# Group by product
product_summary = df.groupby('Product').agg({
    'Sales': 'sum',
    'Quantity': 'sum'
}).reset_index()

print("\nProduct Summary:")
print(product_summary)

# Bar chart of total sales by product
```

```
plt.figure(figsize=(8, 5))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.tight_layout()
plt.show()

# Line chart of sales over time
sales_over_time = df.groupby('Date')['Sales'].sum().reset_index()

plt.figure(figsize=(9, 5))
plt.plot(sales_over_time['Date'], sales_over_time['Sales'],
         marker='o')
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()

# Pivot table: sales by region and product
pivot_table = df.pivot_table(
    values='Sales',
    index='Region',
    columns='Product',
    aggfunc='sum',
    fill_value=0
)

print("\nSales by Region and Product:")
print(pivot_table)

# Correlation matrix
correlation_matrix = df.select_dtypes(
    include=np.number
).corr()

print("\nCorrelation Matrix:")
print(correlation_matrix)

# Heatmap
plt.figure(figsize=(7, 5))
sns.heatmap(correlation_matrix, annot=True,
            cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix')
plt.tight_layout()
plt.show()

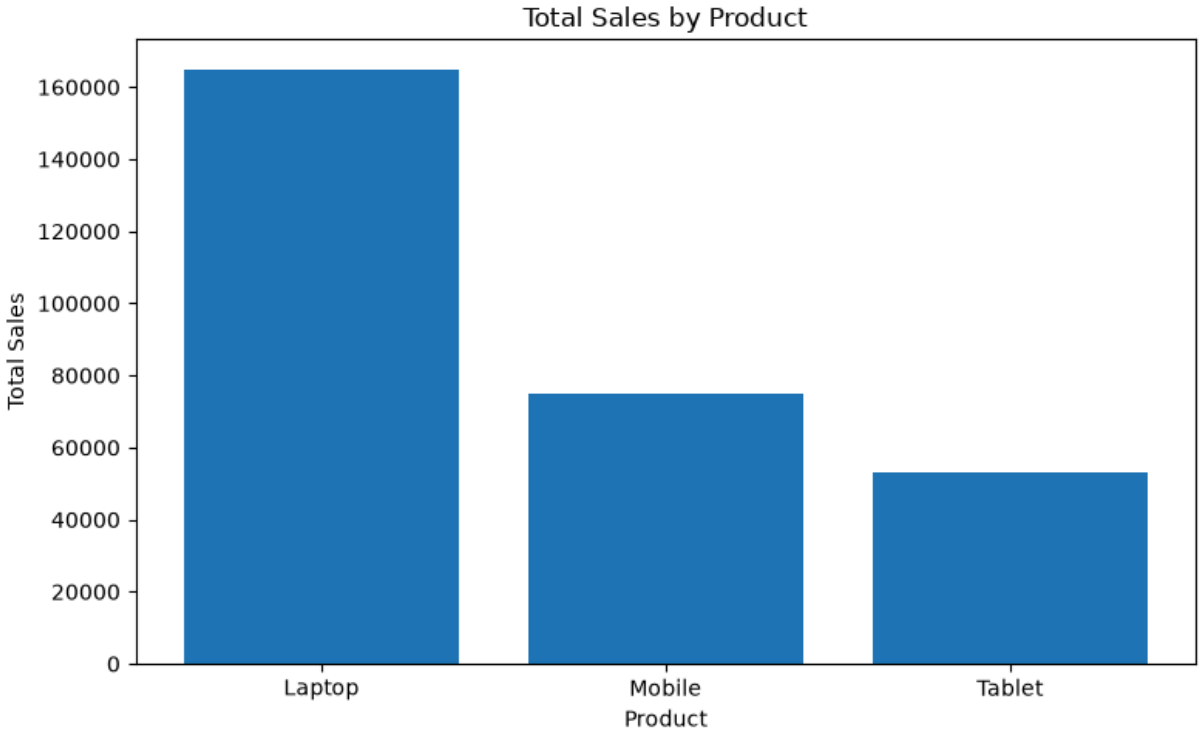
print("\nSales Data Analysis Completed Successfully!")
```

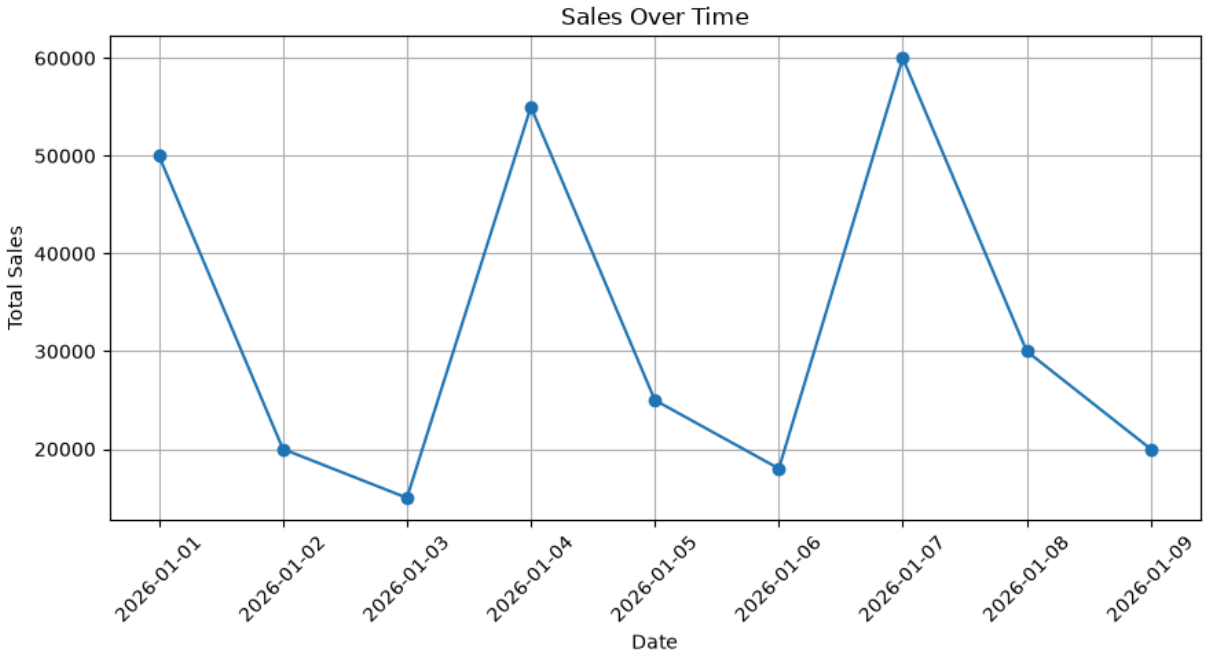
```
First Five Rows:
         Date Product  Sales  Quantity   Region
0  2026-01-01  Laptop  50000         2  Chennai
1  2026-01-02  Mobile  20000         4  Madurai
2  2026-01-03  Tablet  15000         3   Trichy
3  2026-01-04  Laptop  55000         2  Chennai
4  2026-01-05  Mobile  25000         5  Madurai

Missing Values:
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64

Summary Statistics:
                      Date         Sales  Quantity
count                    9      9.000000  9.000000
mean   2026-01-05 00:00:00  32555.555556  3.666667
min    2026-01-01 00:00:00  15000.000000  2.000000
25%    2026-01-03 00:00:00  20000.000000  3.000000
50%    2026-01-05 00:00:00  25000.000000  4.000000
75%    2026-01-07 00:00:00  50000.000000  4.000000
max    2026-01-09 00:00:00  60000.000000  6.000000
std                    NaN  17536.469935  1.322876

Product Summary:
  Product   Sales  Quantity
0  Laptop  165000         7
1  Mobile   75000        15
2  Tablet   53000        11
```



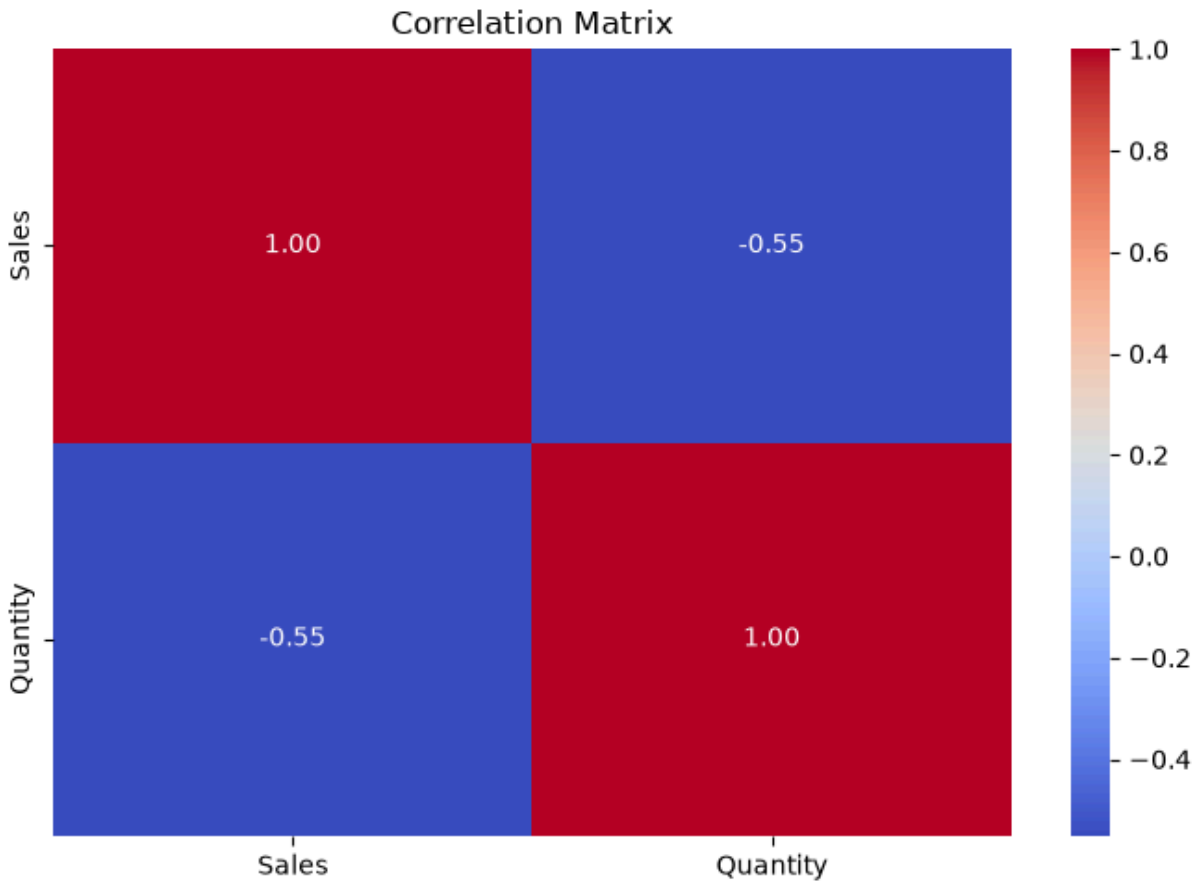


Sales by Region and Product:

Region	Laptop	Mobile	Tablet
Chennai	165000	0	0
Madurai	0	75000	0
Trichy	0	0	53000

Correlation Matrix:

	Sales	Quantity
Sales	1.000000	-0.551399
Quantity	-0.551399	1.000000




```
Sales Data Analysis Completed Successfully!
```

In [ ]:

In [1]:

```
import numpy as np
import pandas as pd

# Create the dataset
data = {
    'Country': ['France', 'Spain', 'Germany', 'Spain', 'Germany',
                'France', 'Spain', 'France', np.nan, 'France'],
    'Age': [44, 27, 30, 38, 40, 35, np.nan, 48, 50, 37],
    'Salary': [72000, 48000, 54000, 61000, np.nan,
               58000, 52000, 79000, 83000, 67000],
    'Purchased': ['No', 'Yes', 'No', 'No', 'Yes',
                  'Yes', 'No', 'Yes', 'No', 'Yes']
}

df = pd.DataFrame(data)

# Display original dataset
print("Original Dataset:")
print(df)

# Display dataset information
print("\nDataset Information:")
df.info()

# Handle missing values
df['Country'] = df['Country'].fillna(df['Country'].mode()[0])
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Salary'] = df['Salary'].fillna(round(df['Salary'].mean()))

# Display dataset after handling missing values
print("\nDataset After Handling Missing Values:")
print(df)

# Convert categorical Country column into dummy variables
country_dummies = pd.get_dummies(df['Country'])

# Combine dummy variables with other columns
updated_dataset = pd.concat(
    [country_dummies, df[['Age', 'Salary', 'Purchased']]],
    axis=1
)

# Encode Purchased column: No = 0, Yes = 1
updated_dataset['Purchased'] = updated_dataset['Purchased'].map({
    'No': 0,
    'Yes': 1
})

# Display final preprocessed dataset
print("\nFinal Preprocessed Dataset:")
print(updated_dataset)

# Check for missing values
print("\nMissing Values in Final Dataset:")
```

```
print(updated_dataset.isnull().sum())

# Display final dataset information
print("\nFinal Dataset Information:")
updated_dataset.info()
```

```
Original Dataset:
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
8      NaN  50.0  83000.0        No
9   France  37.0  67000.0       Yes

Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    9 non-null      str
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     str
dtypes: float64(2), str(2)
memory usage: 532.0 bytes

Dataset After Handling Missing Values:
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0  63778.0       Yes
5   France  35.0  58000.0       Yes
6    Spain  38.0  52000.0        No
7   France  48.0  79000.0       Yes
8   France  50.0  83000.0        No
9   France  37.0  67000.0       Yes

Final Preprocessed Dataset:
   France  Germany  Spain   Age   Salary  Purchased
0    True    False  False  44.0  72000.0          0
1   False    False   True  27.0  48000.0          1
2   False     True  False  30.0  54000.0          0
3   False    False   True  38.0  61000.0          0
4   False     True  False  40.0  63778.0          1
5    True    False  False  35.0  58000.0          1
6   False    False   True  38.0  52000.0          0
7    True    False  False  48.0  79000.0          1
8    True    False  False  50.0  83000.0          0
9    True    False  False  37.0  67000.0          1

Missing Values in Final Dataset:
France       0
Germany      0
Spain        0
```

```
Age          0
Salary       0
Purchased    0
dtype: int64

Final Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 6 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   France     10 non-null     bool   
 1   Germany    10 non-null     bool   
 2   Spain      10 non-null     bool   
 3   Age        10 non-null     float64
 4   Salary     10 non-null     float64
 5   Purchased  10 non-null     int64  
dtypes: bool(3), float64(2), int64(1)
memory usage: 402.0 bytes
```

In [ ]:

In [1]:

```
import numpy as np
import pandas as pd

# Create the Hotel dataset
data = {
    'CustomerID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 9, 10],
    'Age_Group': ['20-25', '30-35', '25-30', '20-25', '35+',
                  '35+', '35+', '20-25', '25-30', '25-30', '30-35'],
    'Rating(1-5)': [4, 5, 6, -1, 3, 3, 4, 7, 2, 2, 5],
    'Hotel': ['Ibis', 'LemonTree', 'RedFox', 'LemonTree', 'Ibis',
              'Ibys', 'RedFox', 'LemonTree', 'Ibis', 'Ibis', 'RedFox'],
    'FoodPreference': ['veg', 'Non-Veg', 'Veg', 'Veg', 'Vegetarian',
                       'Non-Veg', 'Vegetarian', 'Veg', 'Non-Veg',
                       'Non-Veg', 'non-Veg'],
    'Bill': [1300, 2000, 1322, 1234, 989, 1909, 1000, 2999,
             3456, 3456, 6755],
    'NoOfPax': [2, 3, 2, 2, 2, 2, -1, -10, 3, 3, 4],
    'EstimatedSalary': [40000, 59000, 30000, 120000, 45000,
                        122220, 21122, 345673, -99999, -99999, 87777],
    'Age_Group.1': ['20-25', '30-35', '25-30', '20-25', '35+',
                    '35+', '35+', '20-25', '25-30', '25-30', '30-35']
}

df = pd.DataFrame(data)

# Display original dataset
print("Original Dataset:")
print(df)

# Check dataset information
print("\nDataset Information:")
df.info()

# Identify duplicate rows
print("\nDuplicate Rows:")
print(df.duplicated())

# Remove duplicate rows
df.drop_duplicates(inplace=True)

# Reset index
df.reset_index(drop=True, inplace=True)

print("\nDataset After Removing Duplicates:")
print(df)

# Remove unnecessary duplicate column
df.drop(columns=['Age_Group.1'], inplace=True)

# Replace inappropriate numerical values with NaN
df.loc[df['CustomerID'] < 0, 'CustomerID'] = np.nan
df.loc[df['Bill'] < 0, 'Bill'] = np.nan
df.loc[df['EstimatedSalary'] < 0, 'EstimatedSalary'] = np.nan
```

```python
# Validate rating range: 1 to 5
df.loc[~df['Rating(1-5)'].between(1, 5), 'Rating(1-5)'] = np.nan

# Validate number of people
df.loc[~df['NoOfPax'].between(1, 20), 'NoOfPax'] = np.nan

# Correct hotel name
df['Hotel'] = df['Hotel'].replace({'Ibys': 'Ibis'})

# Standardize food preferences
df['FoodPreference'] = df['FoodPreference'].replace({
    'veg': 'Veg',
    'Vegetarian': 'Veg',
    'non-Veg': 'Non-Veg'
})

print("\nDataset After Correcting Inappropriate Data:")
print(df)

# Fill missing numerical values
df['CustomerID'] = df['CustomerID'].fillna(
    df['CustomerID'].median()
)
df['Bill'] = df['Bill'].fillna(round(df['Bill'].mean()))
df['EstimatedSalary'] = df['EstimatedSalary'].fillna(
    round(df['EstimatedSalary'].mean())
)
df['NoOfPax'] = df['NoOfPax'].fillna(
    round(df['NoOfPax'].median())
)
df['Rating(1-5)'] = df['Rating(1-5)'].fillna(
    round(df['Rating(1-5)'].median())
)

# Fill missing categorical values with mode
df['Age_Group'] = df['Age_Group'].fillna(
    df['Age_Group'].mode()[0]
)
df['Hotel'] = df['Hotel'].fillna(df['Hotel'].mode()[0])
df['FoodPreference'] = df['FoodPreference'].fillna(
    df['FoodPreference'].mode()[0]
)

# Display final dataset
print("\nFinal Cleaned Dataset:")
print(df)

# Check remaining missing values
print("\nMissing Values After Preprocessing:")
print(df.isnull().sum())

print("\nData Preprocessing Completed Successfully!")
```

Original Dataset:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	\
0	1	20-25	4	Ibis	veg	1300	
1	2	30-35	5	LemonTree	Non-Veg	2000	
2	3	25-30	6	RedFox	Veg	1322	
3	4	20-25	-1	LemonTree	Veg	1234	
4	5	35+	3	Ibis	Vegetarian	989	
5	6	35+	3	Ibys	Non-Veg	1909	
6	7	35+	4	RedFox	Vegetarian	1000	
7	8	20-25	7	LemonTree	Veg	2999	
8	9	25-30	2	Ibis	Non-Veg	3456	
9	9	25-30	2	Ibis	Non-Veg	3456	
10	10	30-35	5	RedFox	non-Veg	6755	

	NoOfPax	EstimatedSalary	Age_Group.1
0	2	40000	20-25
1	3	59000	30-35
2	2	30000	25-30
3	2	120000	20-25
4	2	45000	35+
5	2	122220	35+
6	-1	21122	35+
7	-10	345673	20-25
8	3	-99999	25-30
9	3	-99999	25-30
10	4	87777	30-35

Dataset Information:

```
<class 'pandas.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       11 non-null     int64
 1   Age_Group        11 non-null     str
 2   Rating(1-5)      11 non-null     int64
 3   Hotel            11 non-null     str
 4   FoodPreference   11 non-null     str
 5   Bill             11 non-null     int64
 6   NoOfPax          11 non-null     int64
 7   EstimatedSalary  11 non-null     int64
 8   Age_Group.1      11 non-null     str
dtypes: int64(5), str(4)
memory usage: 1.1 KB
```

Duplicate Rows:

```
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9      True
```

```
10    False
dtype: bool

Dataset After Removing Duplicates:
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  NoOfPax  \
0           1     20-25            4       Ibis            veg  1300        2
1           2     30-35            5  LemonTree        Non-Veg  2000        3
2           3     25-30            6    RedFox            Veg  1322        2
3           4     20-25           -1  LemonTree            Veg  1234        2
4           5       35+            3       Ibis     Vegetarian   989        2
5           6       35+            3       Ibys        Non-Veg  1909        2
6           7       35+            4    RedFox     Vegetarian  1000       -1
7           8     20-25            7  LemonTree            Veg  2999      -10
8           9     25-30            2       Ibis        Non-Veg  3456        3
9          10     30-35            5    RedFox        non-Veg  6755        4

   EstimatedSalary Age_Group.1
0            40000       20-25
1            59000       30-35
2            30000       25-30
3           120000       20-25
4            45000         35+
5           122220         35+
6            21122         35+
7           345673       20-25
8           -99999       25-30
9            87777       30-35

Dataset After Correcting Inappropriate Data:
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25          4.0       Ibis            Veg  1300.0
1         2.0     30-35          5.0  LemonTree        Non-Veg  2000.0
2         3.0     25-30          NaN    RedFox            Veg  1322.0
3         4.0     20-25          NaN  LemonTree            Veg  1234.0
4         5.0       35+          3.0       Ibis            Veg   989.0
5         6.0       35+          3.0       Ibis        Non-Veg  1909.0
6         7.0       35+          4.0    RedFox            Veg  1000.0
7         8.0     20-25          NaN  LemonTree            Veg  2999.0
8         9.0     25-30          2.0       Ibis        Non-Veg  3456.0
9        10.0     30-35          5.0    RedFox        Non-Veg  6755.0

   NoOfPax  EstimatedSalary
0      2.0          40000.0
1      3.0          59000.0
2      2.0          30000.0
3      2.0         120000.0
4      2.0          45000.0
5      2.0         122220.0
6      NaN          21122.0
7      NaN         345673.0
8      3.0              NaN
9      4.0          87777.0

Final Cleaned Dataset:
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25          4.0       Ibis            Veg  1300.0
```

```
1         2.0     30-35          5.0  LemonTree        Non-Veg  2000.0
2         3.0     25-30          4.0     RedFox            Veg  1322.0
3         4.0     20-25          4.0  LemonTree            Veg  1234.0
4         5.0       35+          3.0       Ibis            Veg   989.0
5         6.0       35+          3.0       Ibis        Non-Veg  1909.0
6         7.0       35+          4.0     RedFox            Veg  1000.0
7         8.0     20-25          4.0  LemonTree            Veg  2999.0
8         9.0     25-30          2.0       Ibis        Non-Veg  3456.0
9        10.0     30-35          5.0     RedFox        Non-Veg  6755.0

   NoOfPax  EstimatedSalary
0      2.0          40000.0
1      3.0          59000.0
2      2.0          30000.0
3      2.0         120000.0
4      2.0          45000.0
5      2.0         122220.0
6      2.0          21122.0
7      2.0         345673.0
8      3.0          96755.0
9      4.0          87777.0

Missing Values After Preprocessing:
CustomerID         0
Age_Group          0
Rating(1-5)        0
Hotel              0
FoodPreference     0
Bill               0
NoOfPax            0
EstimatedSalary    0
dtype: int64

Data Preprocessing Completed Successfully!
```

In [ ]:

In [5]:
```
import numpy as np

array = np.array([27, 50, 44, 6, 58, 61, 23, 86, 67, 20, 75, 7, 79, 61, 90, 54])

print(array)
```

```
[27 50 44  6 58 61 23 86 67 20 75  7 79 61 90 54]
```

In [6]:
```
array.mean()
```

Out[6]:
```
np.float64(50.5)
```

In [8]:
```
np.percentile(array, 25)
```

Out[8]:
```
np.float64(26.0)
```

In [9]:
```
np.percentile(array, 50)
```

Out[9]:
```
np.float64(56.0)
```

In [10]:
```
np.percentile(array, 75)
```

Out[10]:
```
np.float64(69.0)
```

In [11]:
```
np.percentile(array, 100)
```

Out[11]:
```
np.float64(90.0)
```

In [12]:
```
def outDetection(array):
    sorted(array)
    Q1, Q3 = np.percentile(array, [25, 75])
    IQR = Q3 - Q1
    lr = Q1 - (1.5 * IQR)
    ur = Q3 + (1.5 * IQR)
    return lr, ur
```

In [13]:
```
lr, ur = outDetection(array)

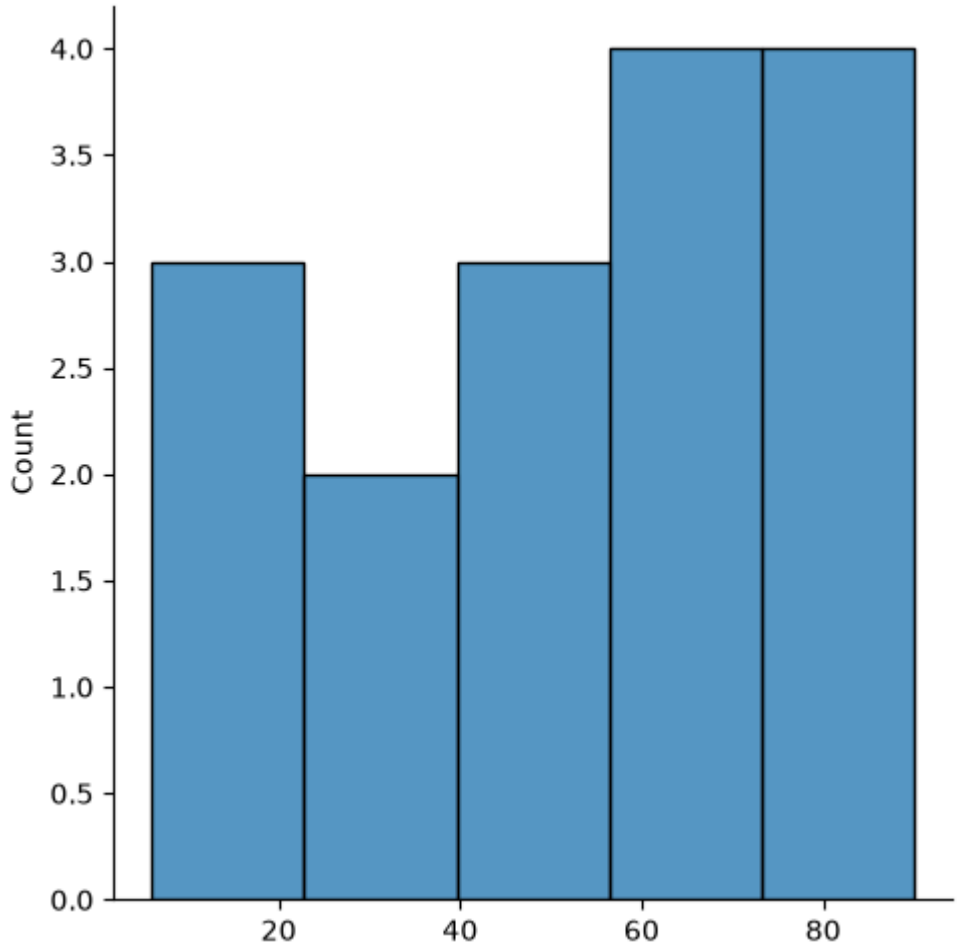
lr, ur
```

Out[13]:
```
(np.float64(-38.5), np.float64(133.5))
```

In [14]:
```
import seaborn as sns
%matplotlib inline
```

In [15]:
```
sns.displot(array)
```

Out[15]:
```
<seaborn.axisgrid.FacetGrid at 0x1b798abd2b0>
```



In [16]:
```
sns.distplot(array)
```

```
C:\Users\chaar\AppData\Local\Temp\ipykernel_32948\1855776059.py:1: UserWarning:

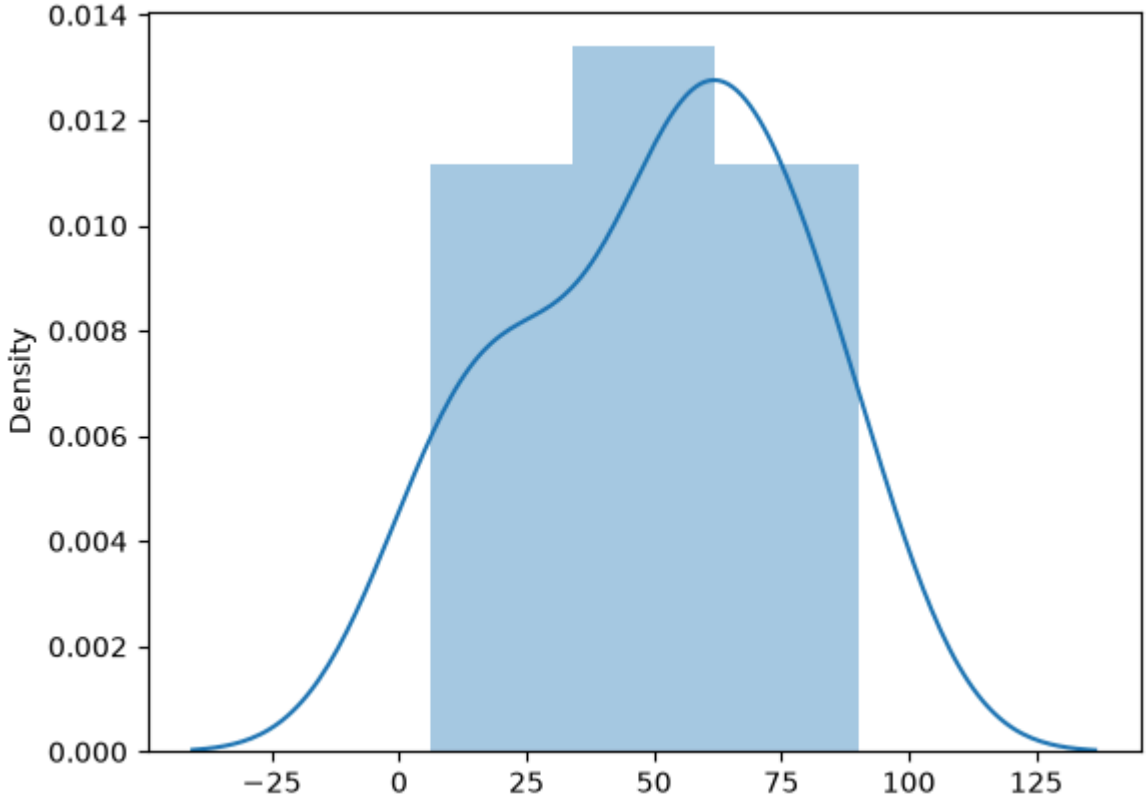
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(array)
```

Out[16]:
```
<Axes: ylabel='Density'>
```



In [ ]:

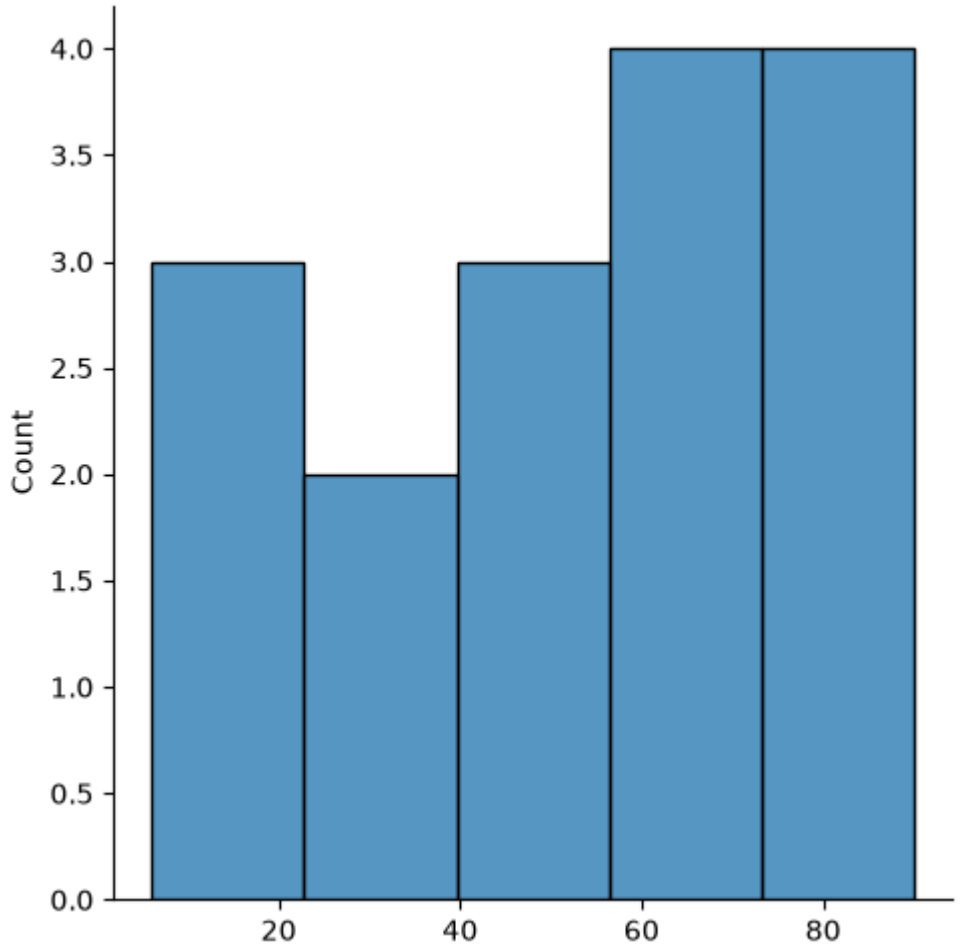
In [18]:
```
new_array = array[(array > lr) & (array < ur)]

new_array
```

Out[18]:
```
array([27, 50, 44,  6, 58, 61, 23, 86, 67, 20, 75,  7, 79, 61, 90, 54])
```

In [19]:
```
sns.displot(new_array)
```

Out[19]:
```
<seaborn.axisgrid.FacetGrid at 0x1b79929e710>
```



In [20]:
```
lr1, ur1 = outDetection(new_array)

lr1, ur1
```

Out[20]:
```
(np.float64(-38.5), np.float64(133.5))
```

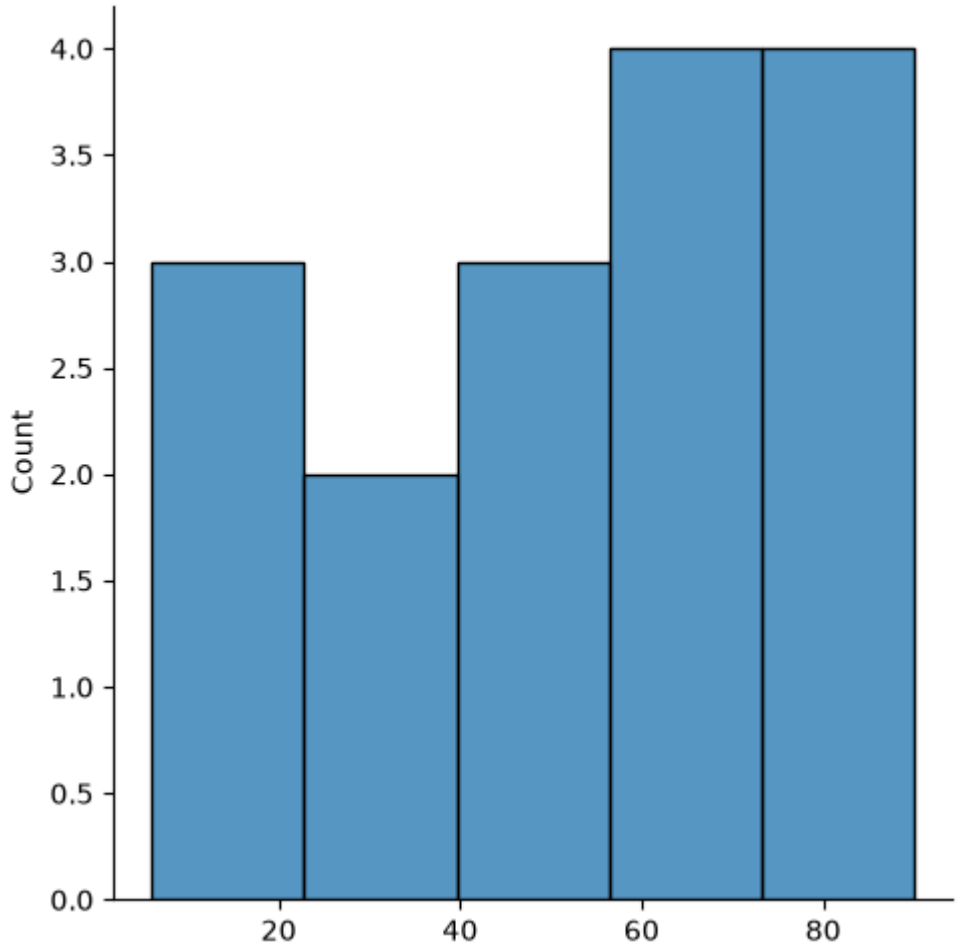
In [21]:
```
final_array = new_array[(new_array > lr1) & (new_array < ur1)]

final_array
```

Out[21]:
```
array([27, 50, 44,  6, 58, 61, 23, 86, 67, 20, 75,  7, 79, 61, 90, 54])
```

In [23]:
```
sns.displot(final_array)
```

Out[23]:
```
<seaborn.axisgrid.FacetGrid at 0x1b7993c6350>
```



In [ ]:

In [2]:
```
import numpy as np
import pandas as pd
df=pd.read_csv('pre-process_datasample.csv')
df
```

Out[2]:

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes
5	France	35.0	58000.0	Yes
6	Spain	NaN	52000.0	No
7	France	48.0	79000.0	Yes
8	NaN	50.0	83000.0	No
9	France	37.0	67000.0	Yes

In [4]:
```
df.head()
```

Out[4]:

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes

In [8]:
```
df['Country']=df['Country'].fillna(df['Country'].mode()[0])
features=df.iloc[:,:-1].values
```

In [9]:
```
label=df.iloc[:,-1].values
```

In [10]:
```
from sklearn.impute import SimpleImputer
age=SimpleImputer(strategy="mean",missing_values=np.nan)
Salary=SimpleImputer(strategy="mean",missing_values=np.nan)
age.fit(features[:,[1]])
```

Out[10]:

SimpleImputer

▶ Parameters

▶ Fitted attributes

▸ 1 feature

In [11]:
```
Salary.fit(features[:,[2]])
```

Out[11]:

SimpleImputer

▶ Parameters

▶ Fitted attributes

▸ 1 feature

In [12]:
```
features[:,[1]]=age.transform(features[:,[1]])
features[:,[2]]=Salary.transform(features[:,[2]])
features
```

Out[12]:
```
array([['France', 44.0, 72000.0],
       ['Spain', 27.0, 48000.0],
       ['Germany', 30.0, 54000.0],
       ['Spain', 38.0, 61000.0],
       ['Germany', 40.0, 63777.77777777778],
       ['France', 35.0, 58000.0],
       ['Spain', 38.77777777777778, 52000.0],
       ['France', 48.0, 79000.0],
       ['France', 50.0, 83000.0],
       ['France', 37.0, 67000.0]], dtype=object)
```

In [13]:
```
from sklearn.preprocessing import OneHotEncoder
oh = OneHotEncoder(sparse_output=False)
Country=oh.fit_transform(features[:,[0]])
Country
```

Out[13]:
```
array([[1., 0., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [1., 0., 0.],
       [0., 0., 1.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.]])
```

In [14]:
```
final_set=np.concatenate((Country,features[:,[1,2]]),axis=1)
final_set
```

Out[14]:
```
array([[1.0, 0.0, 0.0, 44.0, 72000.0],
       [0.0, 0.0, 1.0, 27.0, 48000.0],
       [0.0, 1.0, 0.0, 30.0, 54000.0],
       [0.0, 0.0, 1.0, 38.0, 61000.0],
       [0.0, 1.0, 0.0, 40.0, 63777.77777777778],
       [1.0, 0.0, 0.0, 35.0, 58000.0],
       [0.0, 0.0, 1.0, 38.77777777777778, 52000.0],
       [1.0, 0.0, 0.0, 48.0, 79000.0],
       [1.0, 0.0, 0.0, 50.0, 83000.0],
       [1.0, 0.0, 0.0, 37.0, 67000.0]], dtype=object)
```

In [15]:
```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
sc.fit(final_set)
feat_standard_scaler=sc.transform(final_set)
feat_standard_scaler
```

Out[15]:
```
array([[ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,
         7.58874362e-01,  7.49473254e-01],
       [-1.00000000e+00, -5.00000000e-01,  1.52752523e+00,
        -1.71150388e+00, -1.43817841e+00],
       [-1.00000000e+00,  2.00000000e+00, -6.54653671e-01,
        -1.27555478e+00, -8.91265492e-01],
       [-1.00000000e+00, -5.00000000e-01,  1.52752523e+00,
        -1.13023841e-01, -2.53200424e-01],
       [-1.00000000e+00,  2.00000000e+00, -6.54653671e-01,
         1.77608893e-01,  6.63219199e-16],
       [ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,
        -5.48972942e-01, -5.26656882e-01],
       [-1.00000000e+00, -5.00000000e-01,  1.52752523e+00,
         0.00000000e+00, -1.07356980e+00],
       [ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,
         1.34013983e+00,  1.38753832e+00],
       [ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,
         1.63077256e+00,  1.75214693e+00],
       [ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,
        -2.58340208e-01,  2.93712492e-01]])
```

In [16]:
```
from sklearn.preprocessing import MinMaxScaler
mms=MinMaxScaler(feature_range=(0,1))
mms.fit(final_set)
feat_minmax_scaler=mms.transform(final_set)
feat_minmax_scaler
```

Out[16]:
```
array([[1.        , 0.        , 0.        , 0.73913043, 0.68571429],
       [0.        , 0.        , 1.        , 0.        , 0.        ],
       [0.        , 1.        , 0.        , 0.13043478, 0.17142857],
       [0.        , 0.        , 1.        , 0.47826087, 0.37142857],
       [0.        , 1.        , 0.        , 0.56521739, 0.45079365],
       [1.        , 0.        , 0.        , 0.34782609, 0.28571429],
       [0.        , 0.        , 1.        , 0.51207729, 0.11428571],
       [1.        , 0.        , 0.        , 0.91304348, 0.88571429],
       [1.        , 0.        , 0.        , 1.        , 1.        ],
       [1.        , 0.        , 0.        , 0.43478261, 0.54285714]])
```

In [ ]:

In [3]:
```
# Histogram without KDE
sns.displot(data=tips, x='total_bill', kde=False)
plt.show()

# Joint plot
sns.jointplot(data=tips, x='tip', y='total_bill')
plt.show()

# Regression plot
sns.jointplot(data=tips, x='tip', y='total_bill', kind='reg')
plt.show()

# Hexagonal plot
sns.jointplot(data=tips, x='tip', y='total_bill', kind='hex')
plt.show()

# Pair plot
sns.pairplot(data=tips, vars=['total_bill', 'tip', 'size'])
plt.show()

# Pair plot by time
sns.pairplot(data=tips, vars=['total_bill', 'tip', 'size'], hue='time')
plt.show()

# Pair plot by day
sns.pairplot(data=tips, vars=['total_bill', 'tip', 'size'], hue='day')
plt.show()

# Correlation heatmap
sns.heatmap(tips.corr(numeric_only=True), annot=True)
plt.show()

# Box plots
sns.boxplot(data=tips, x='total_bill')
plt.show()

sns.boxplot(data=tips, x='tip')
plt.show()

# Count plots
sns.countplot(data=tips, x='day')
plt.show()

sns.countplot(data=tips, y='sex')
plt.show()

sns.countplot(data=tips, x='time')
plt.show()

sns.countplot(data=tips, x='smoker')
plt.show()

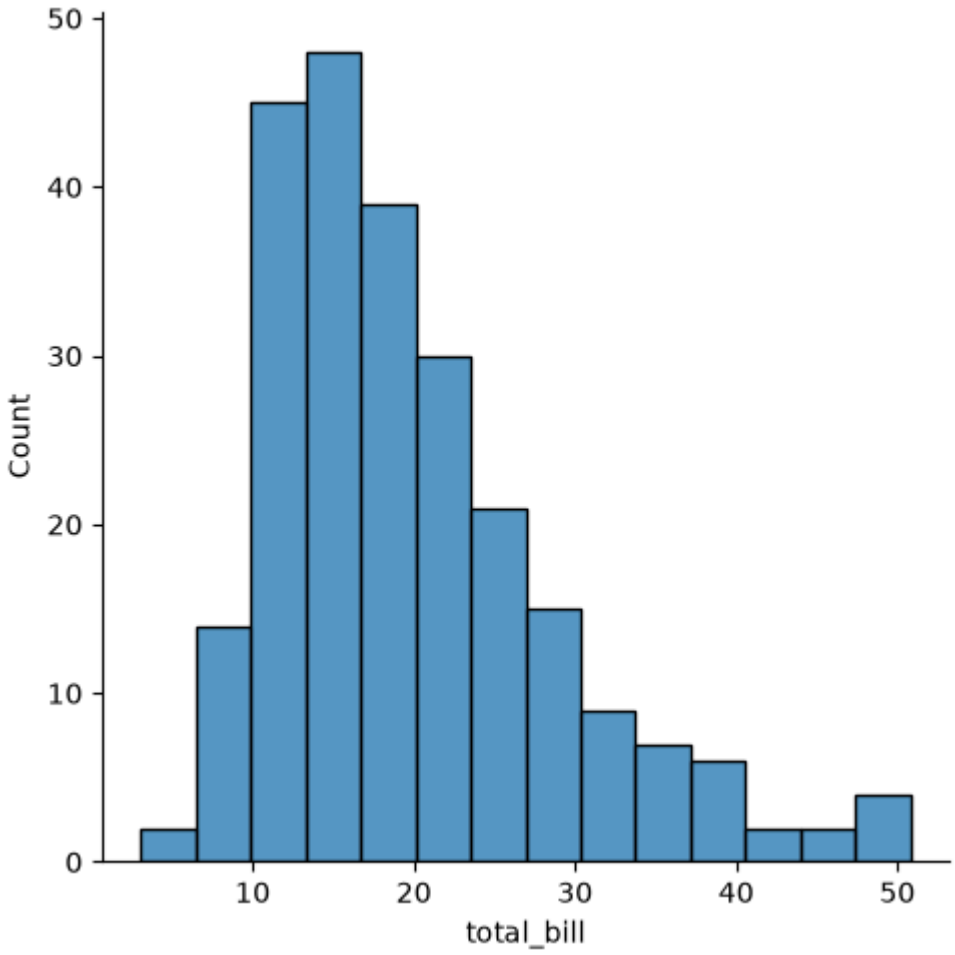
sns.countplot(data=tips, x='size')
plt.show()

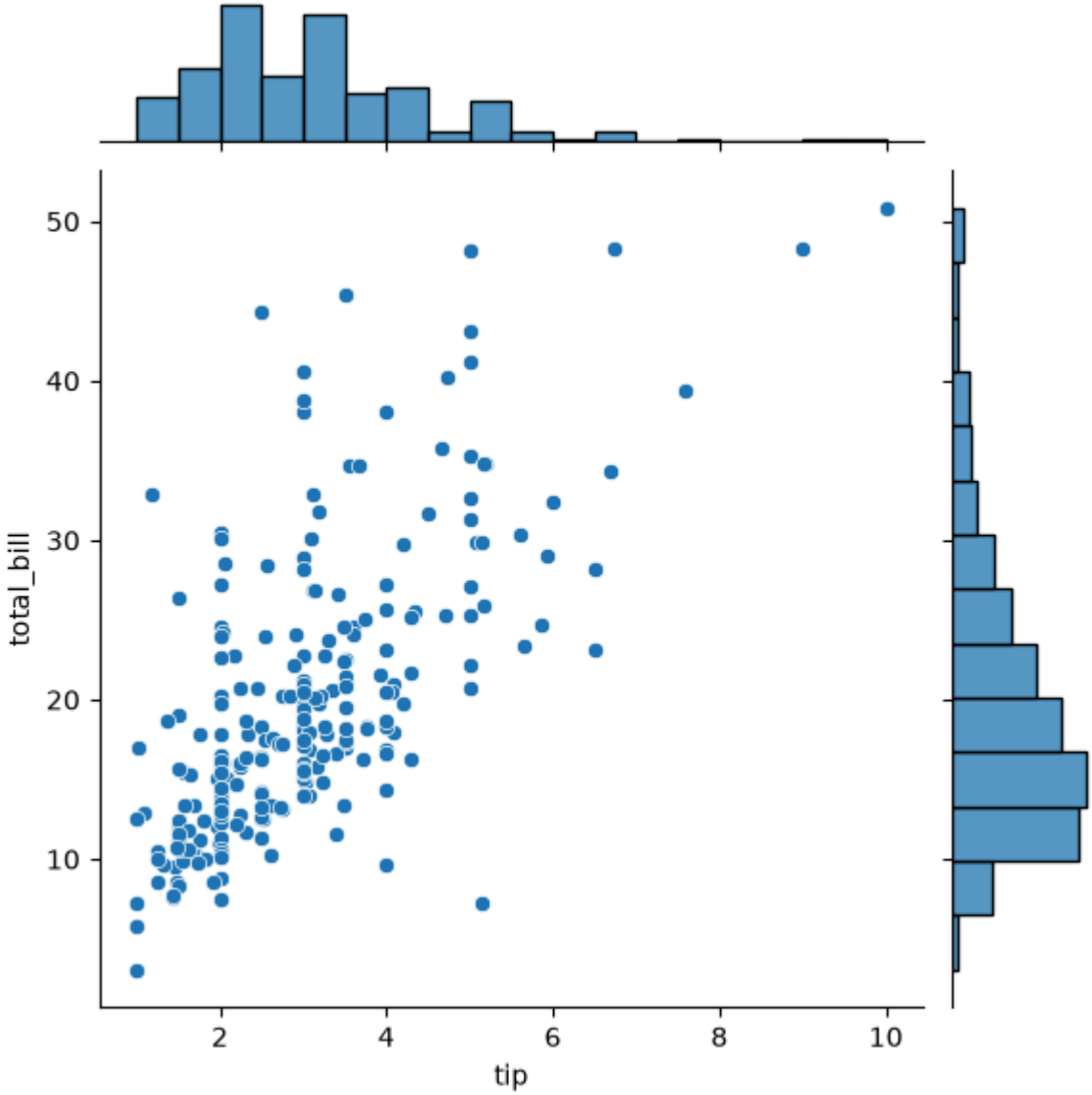
# Pie chart
tips['sex'].value_counts().plot(kind='pie', autopct='%1.1f%%')
plt.ylabel('')
```

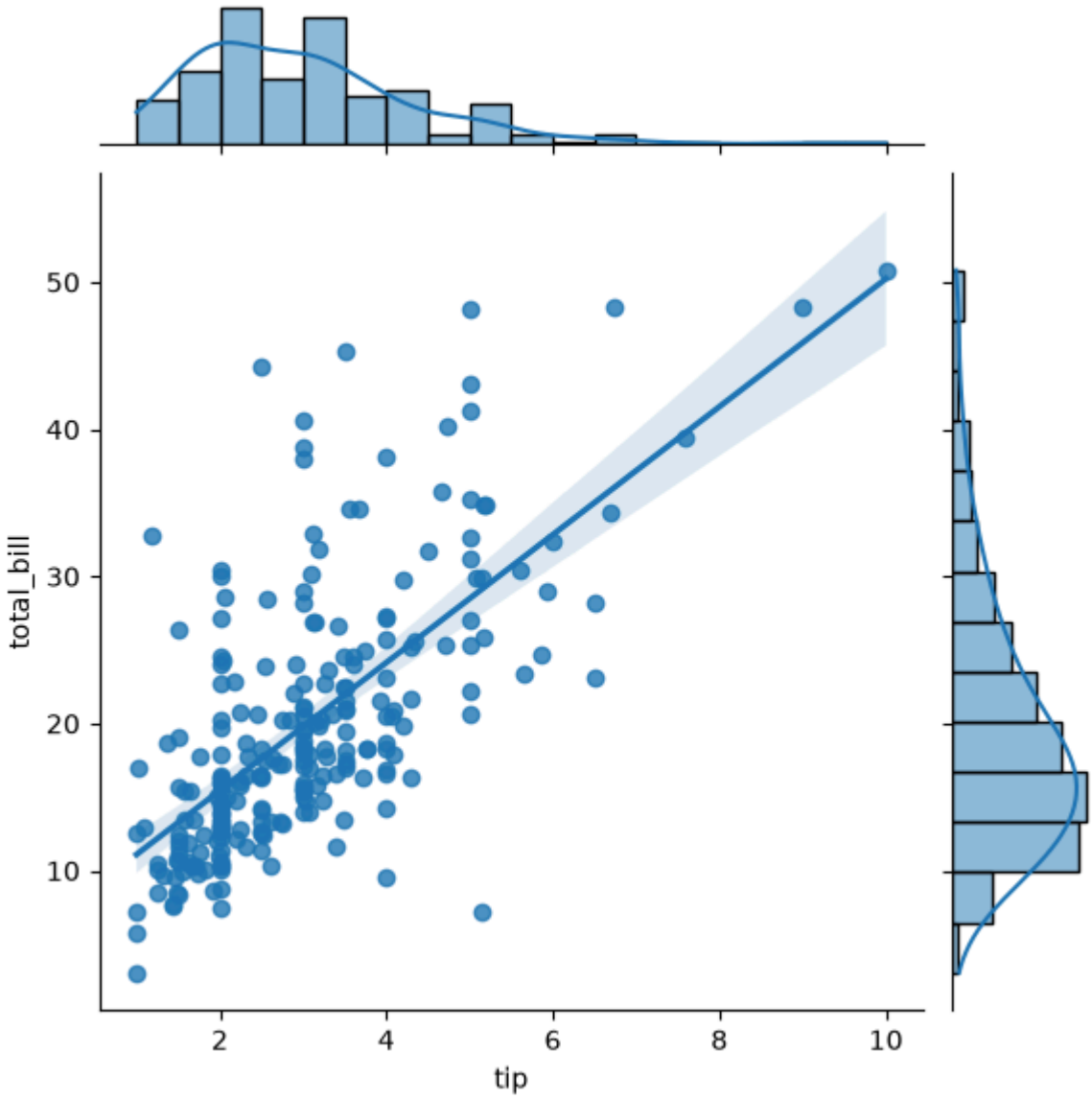
```
plt.show()

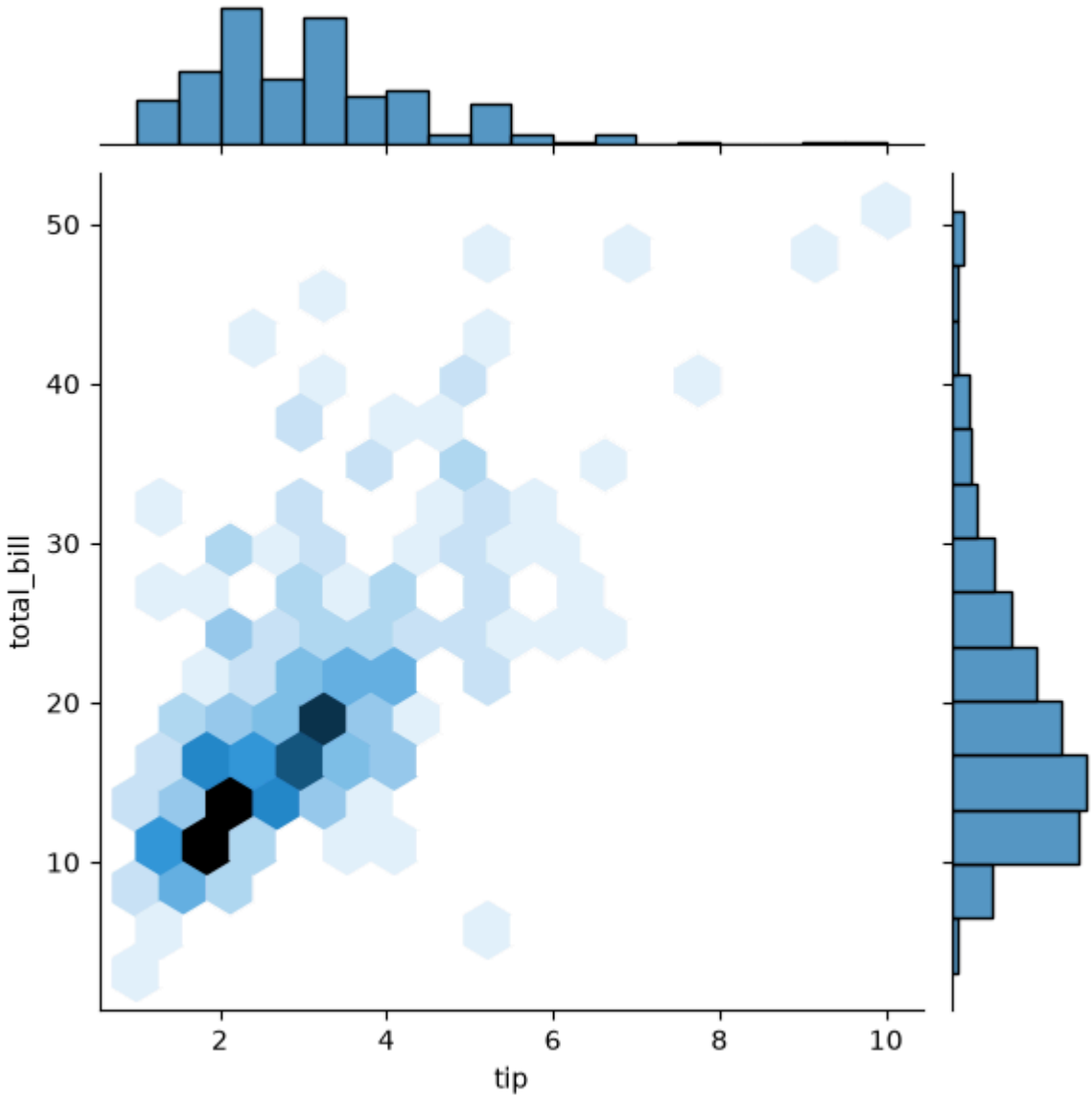
# Bar chart
tips['sex'].value_counts().plot(kind='bar')
plt.show()

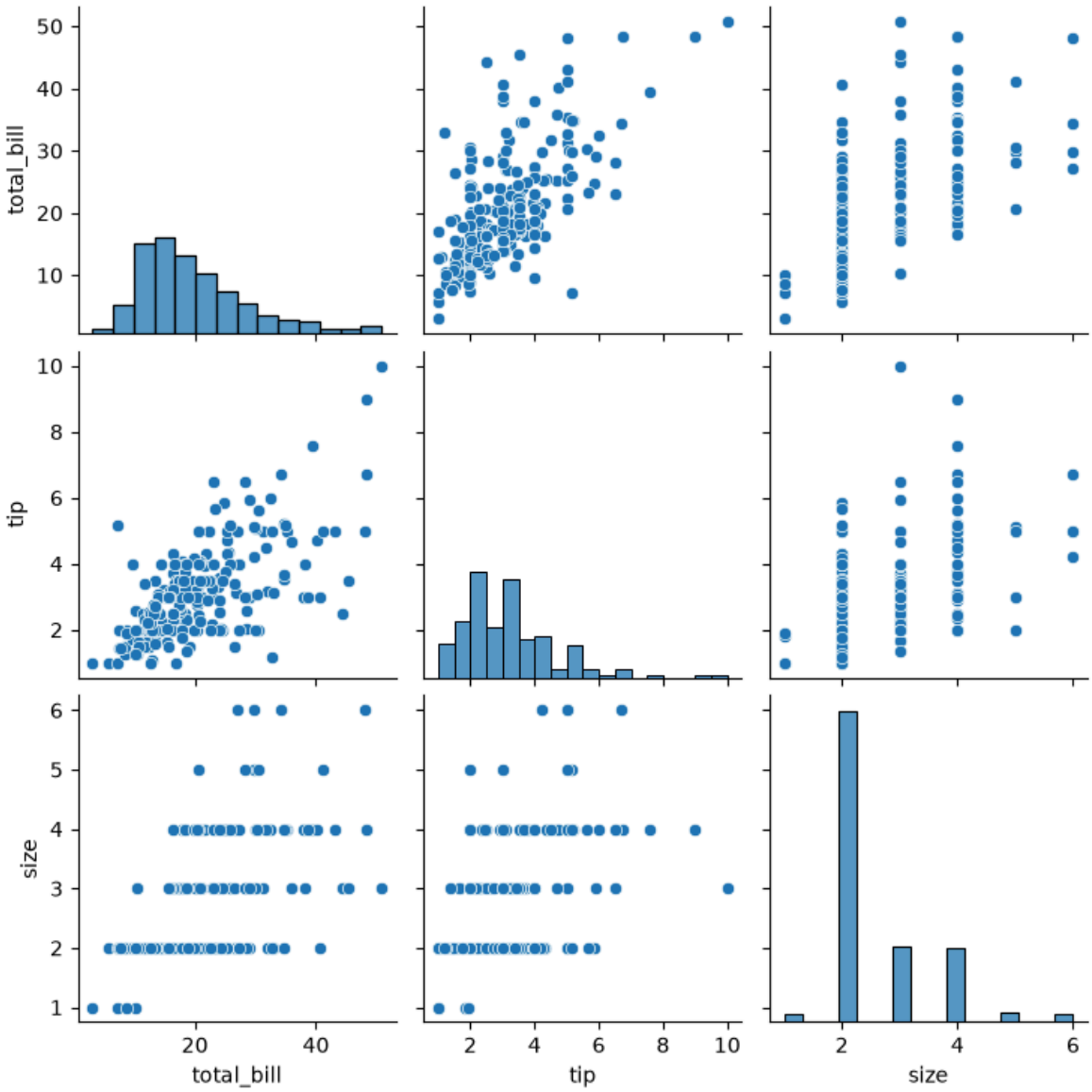
# Value counts
print(tips['time'].value_counts())
```

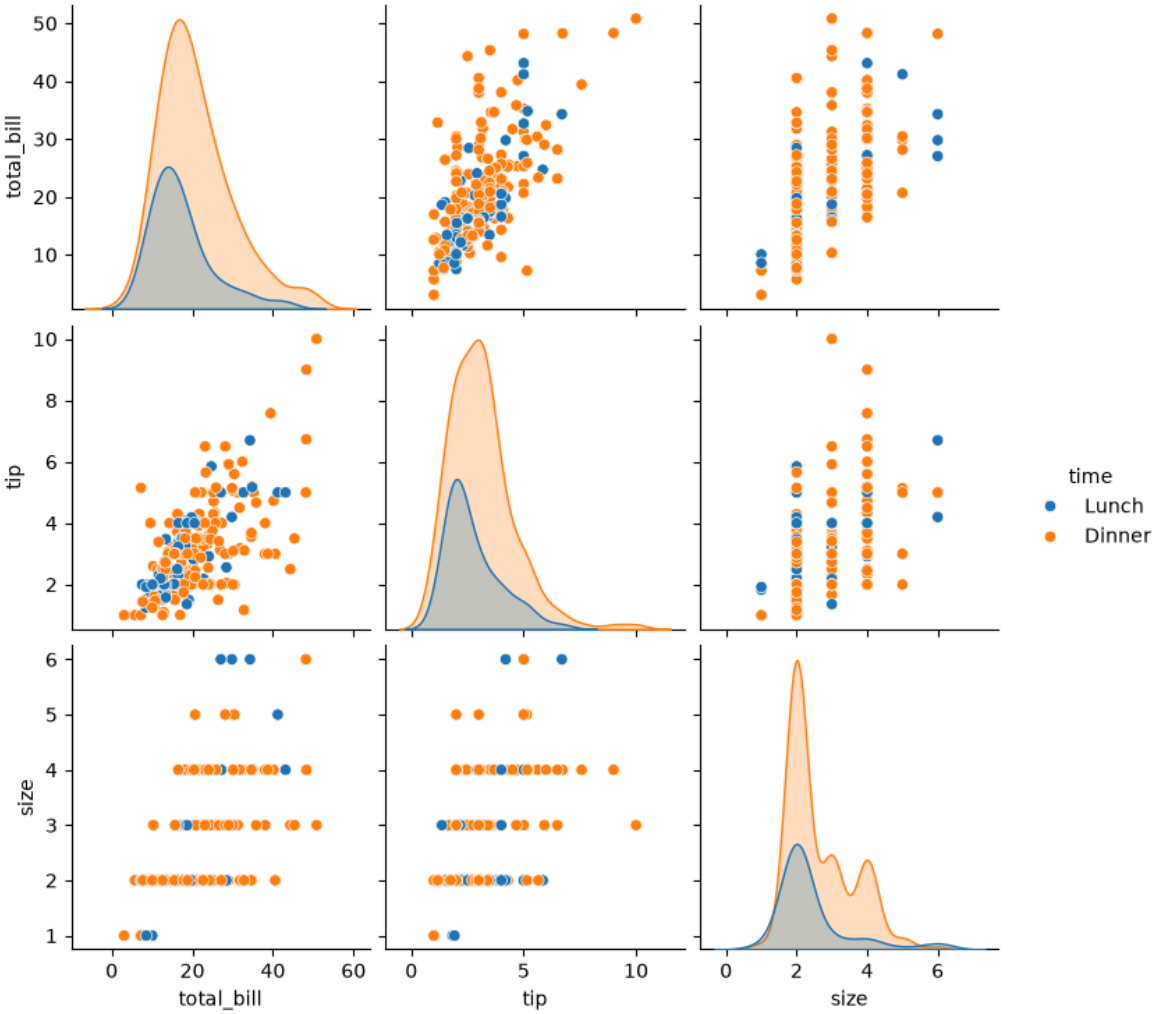


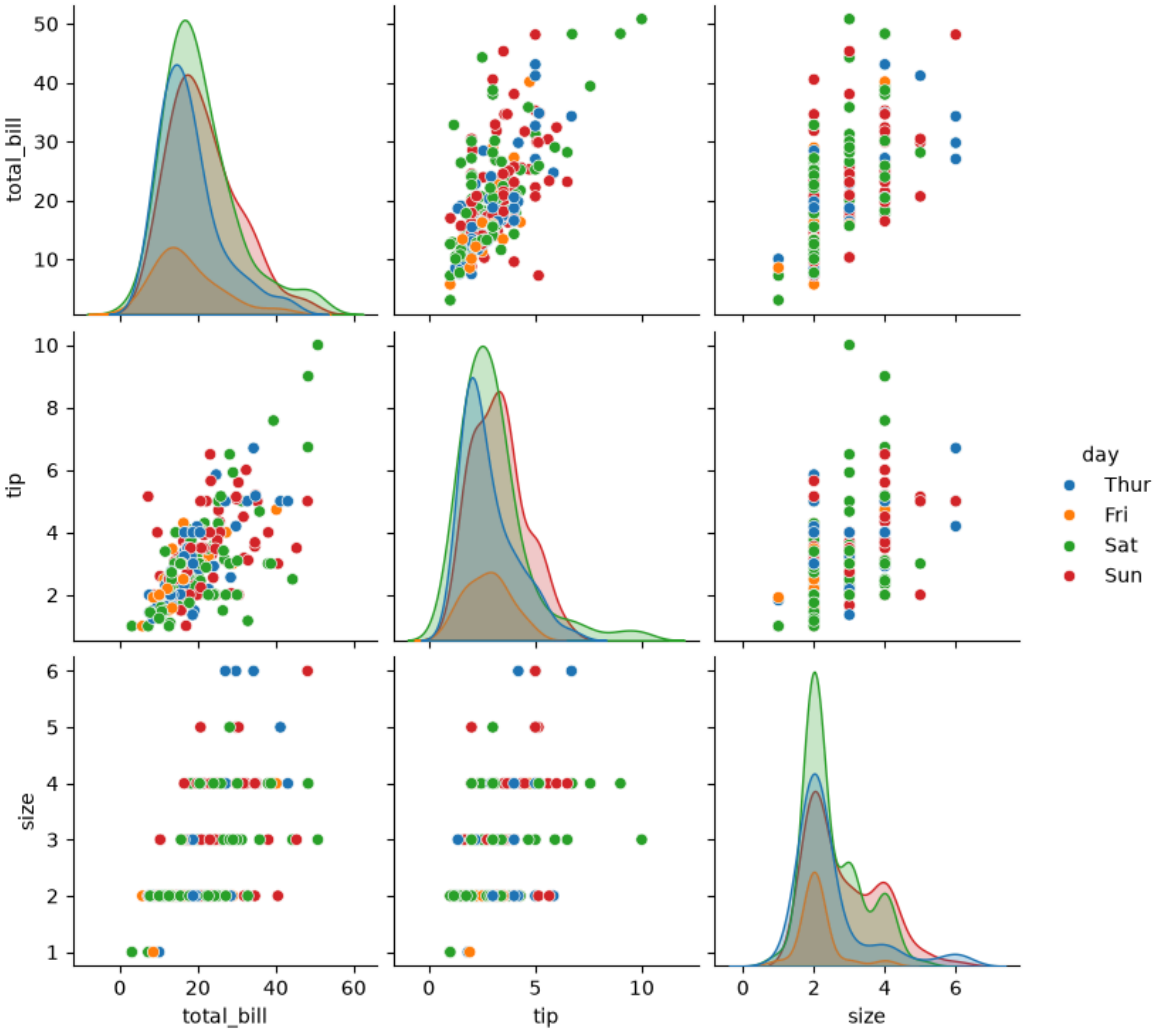

total_bill
tip
50
40
30
20
10
2
4
6
8
10

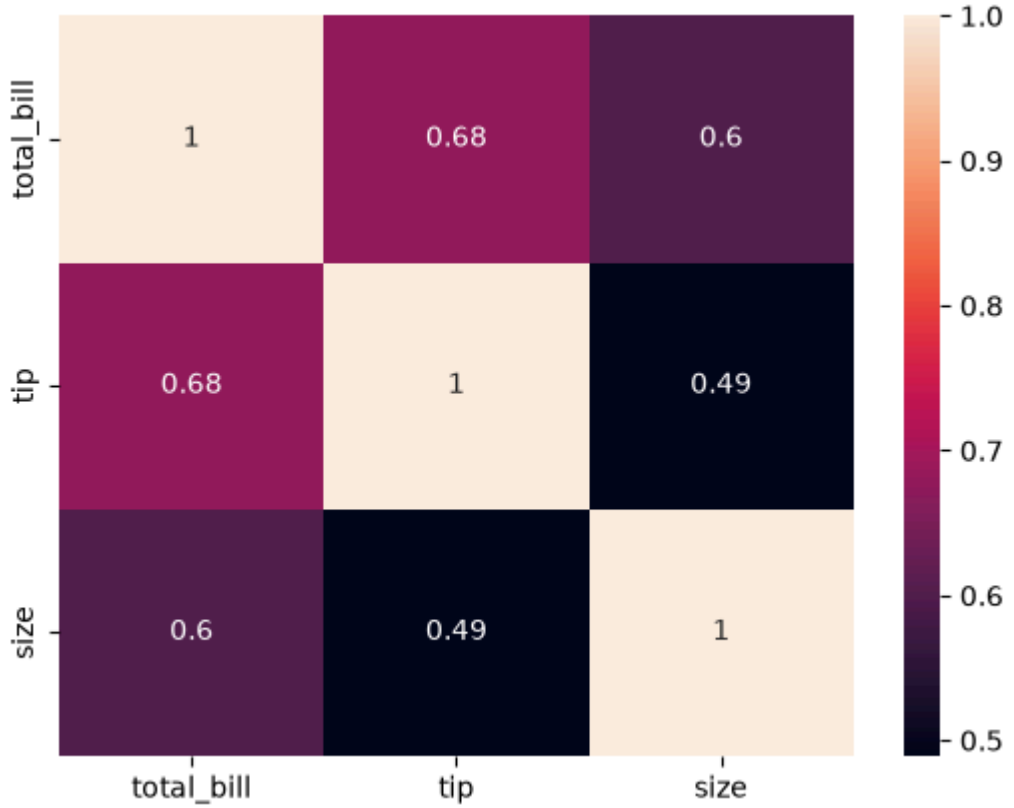

total_bill
tip
50
40
30
20
10
2
4
6
8
10

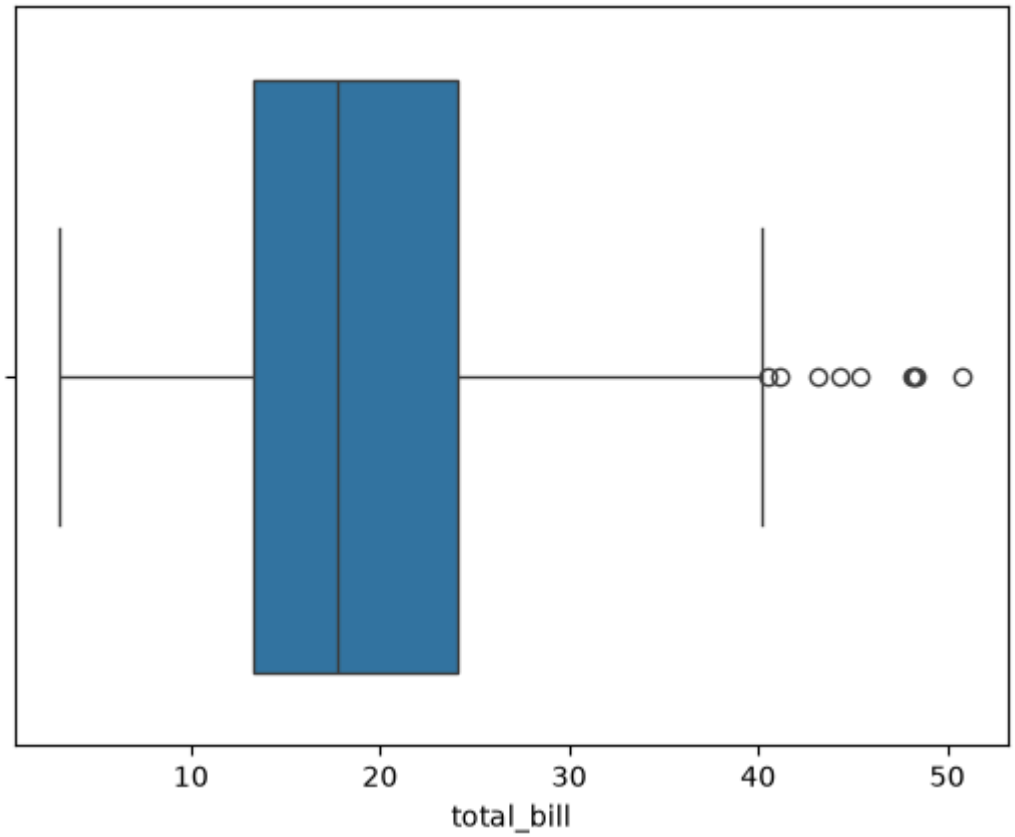

50
40
30
20
10
total_bill
2
4
6
8
10
tip

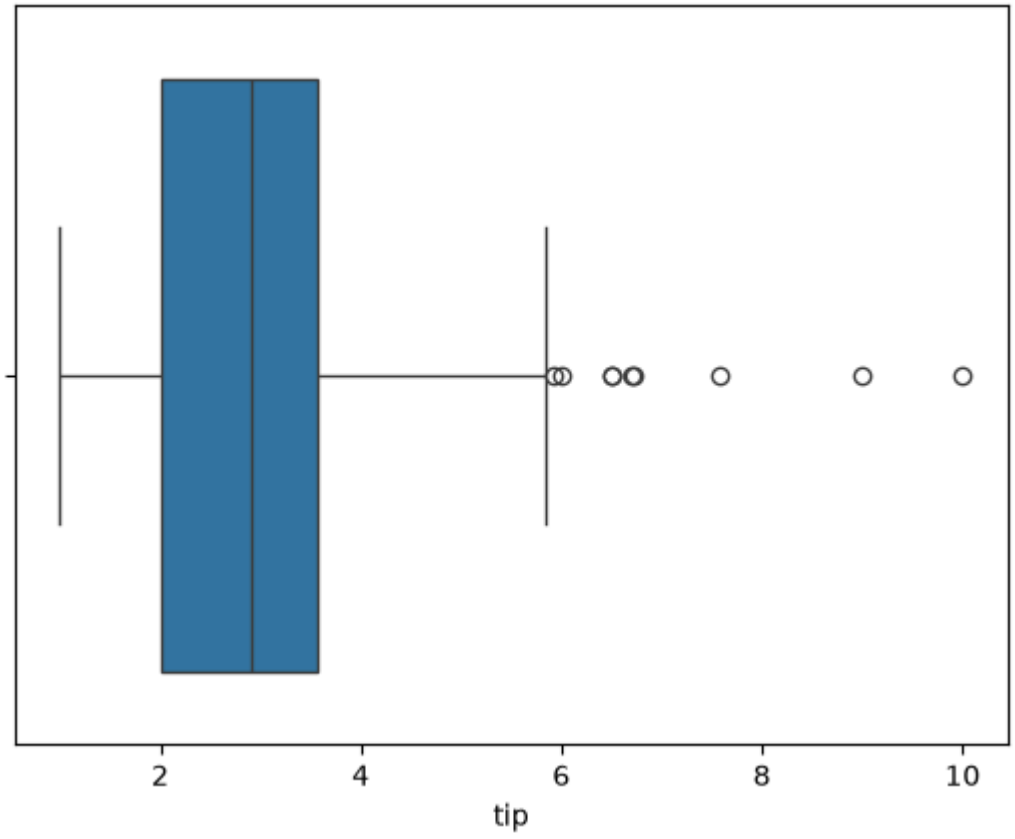

total_bill
tip
size
50
40
30
20
10
10
8
6
4
2
6
5
4
3
2
1
20
40
2
4
6
8
10
2
4
6

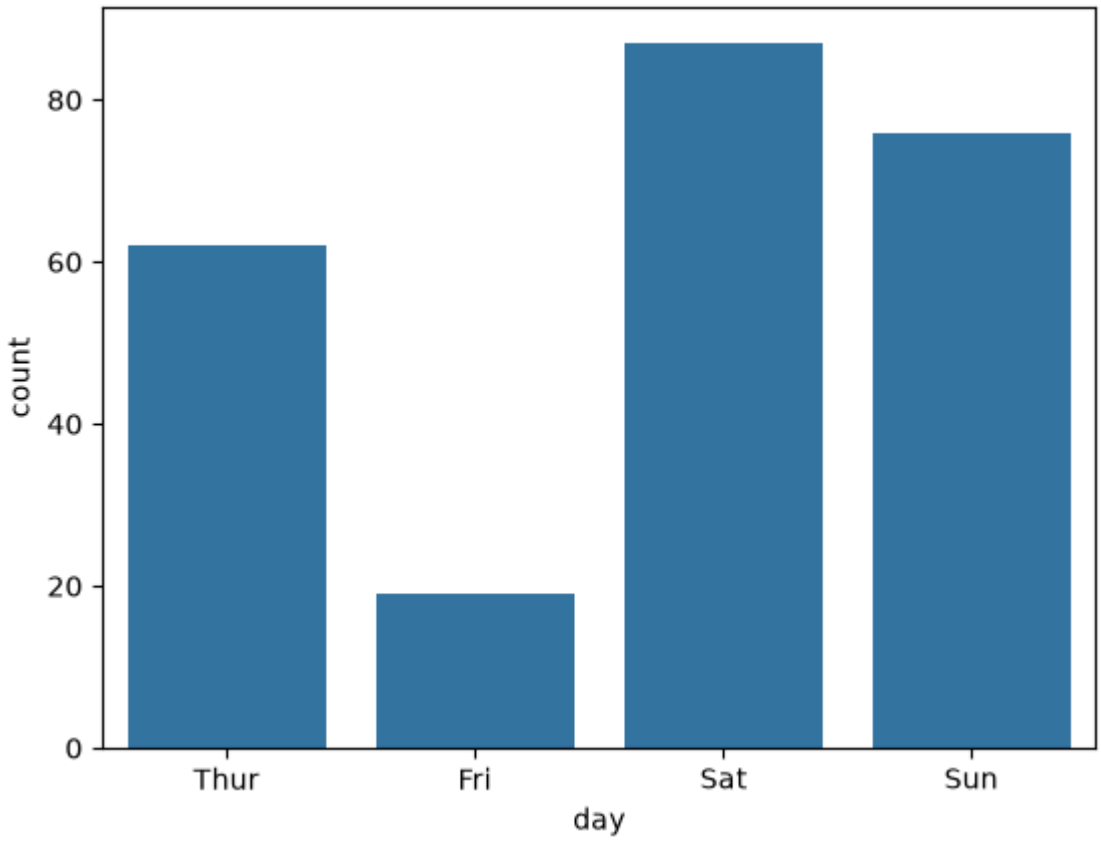

total_bill
tip
size
time
Lunch
Dinner

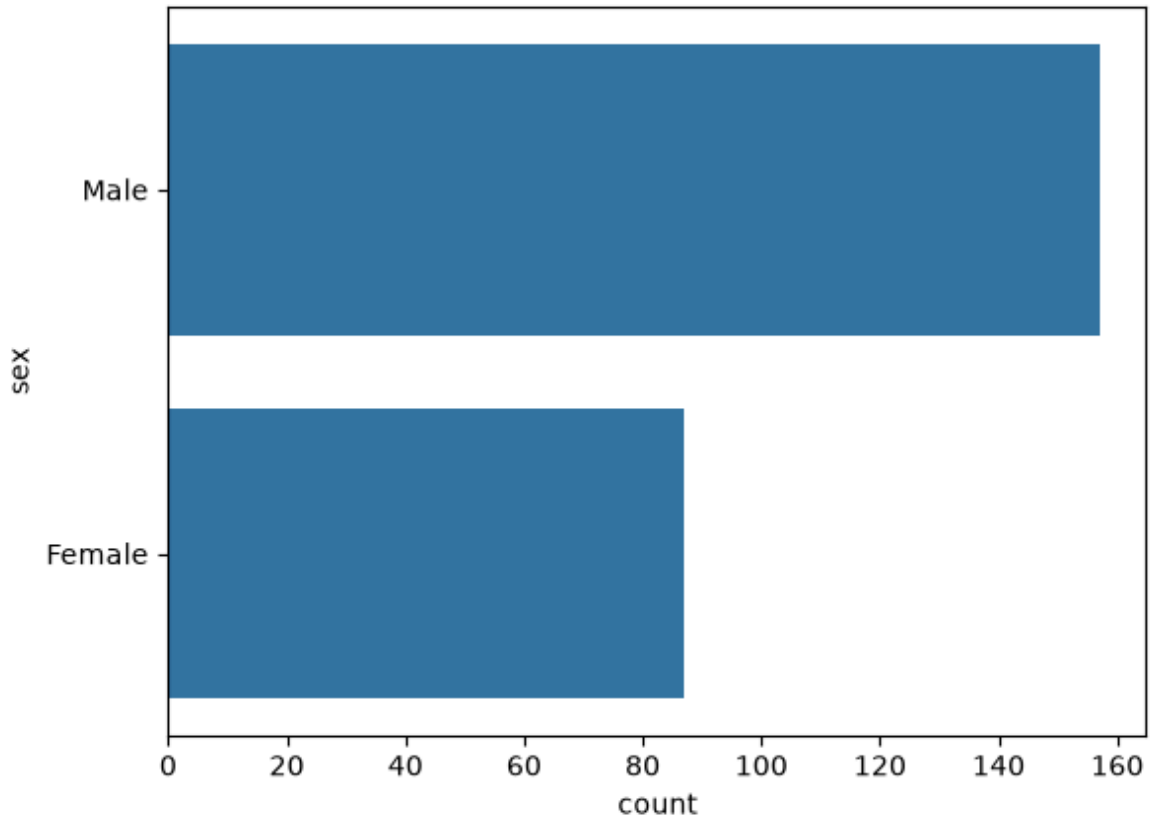

total_bill
tip
size
day
Thur
Fri
Sat
Sun

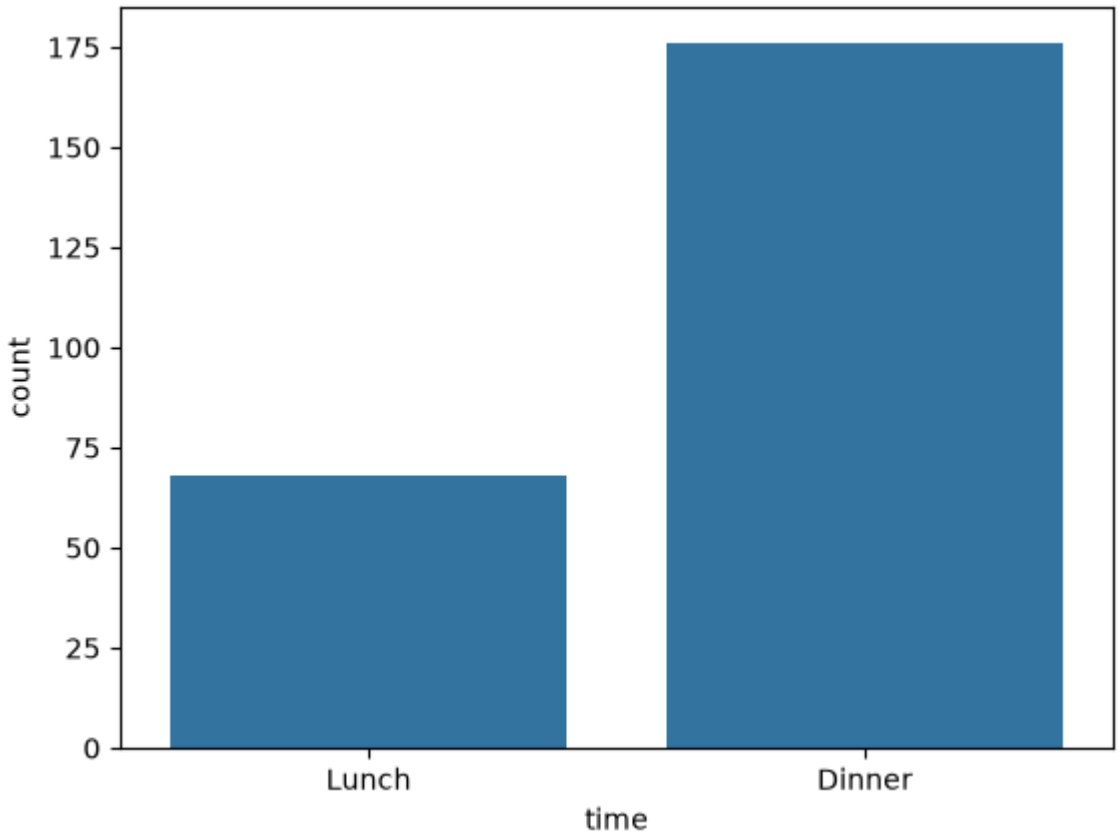

total_bill
tip
size
1
0.68
0.6
0.68
1
0.49
0.6
0.49
1
1.0
0.9
0.8
0.7
0.6
0.5

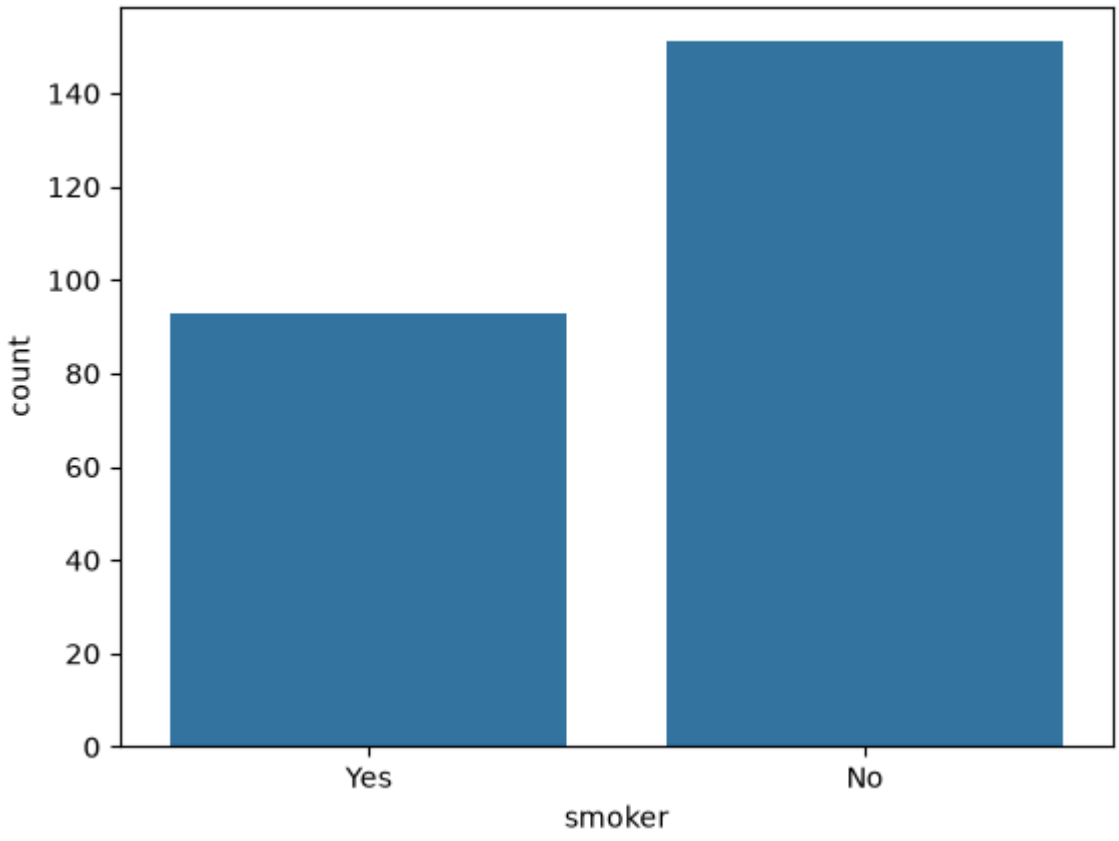

10
20
30
40
50
total_bill

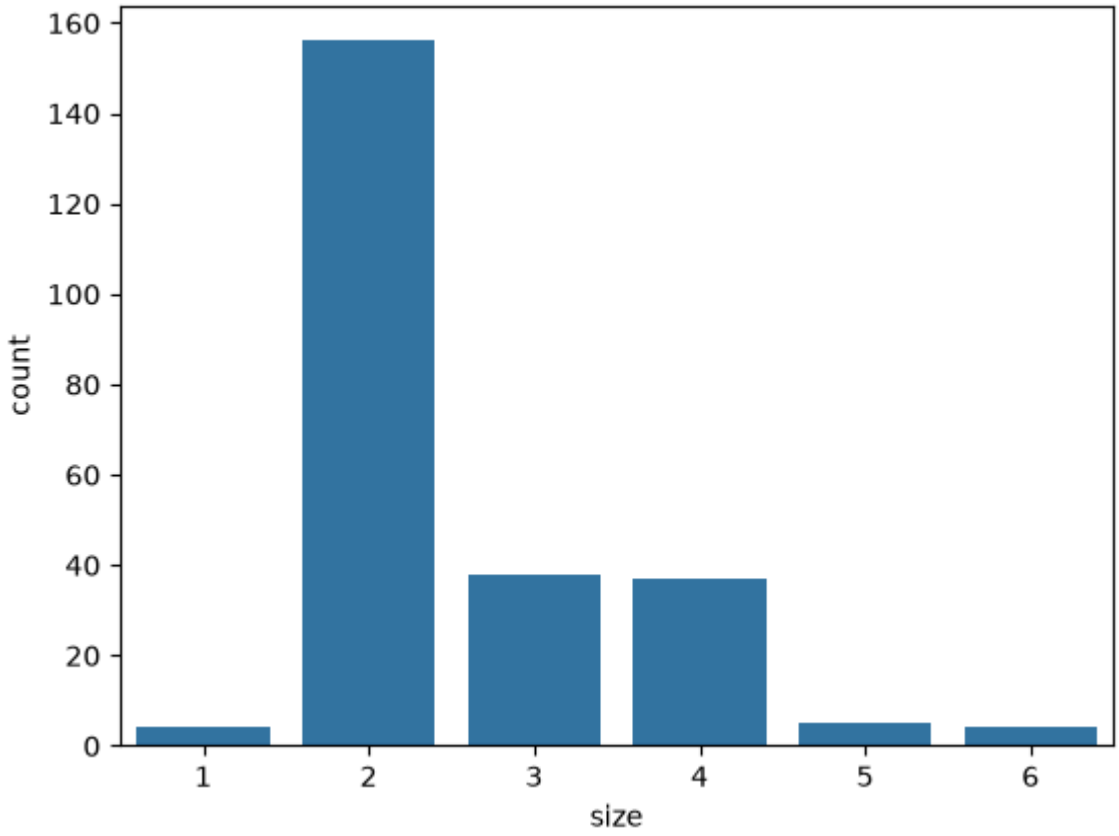

2
4
6
8
10
tip

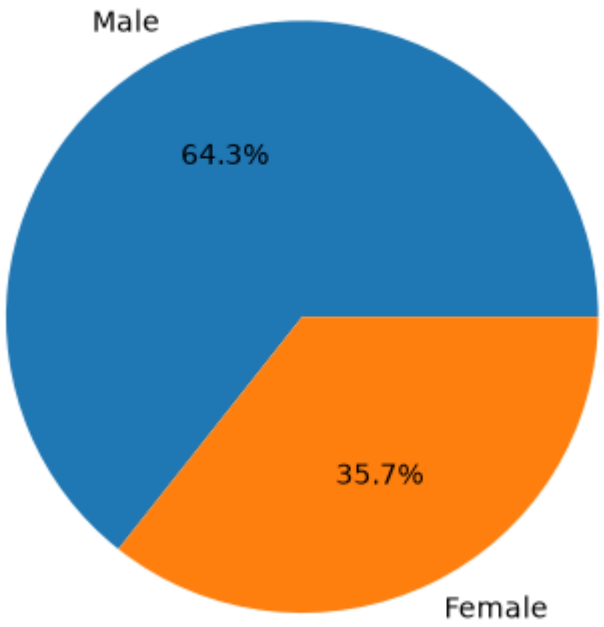

80
60
40
20
0
count
Thur
Fri
Sat
Sun
day

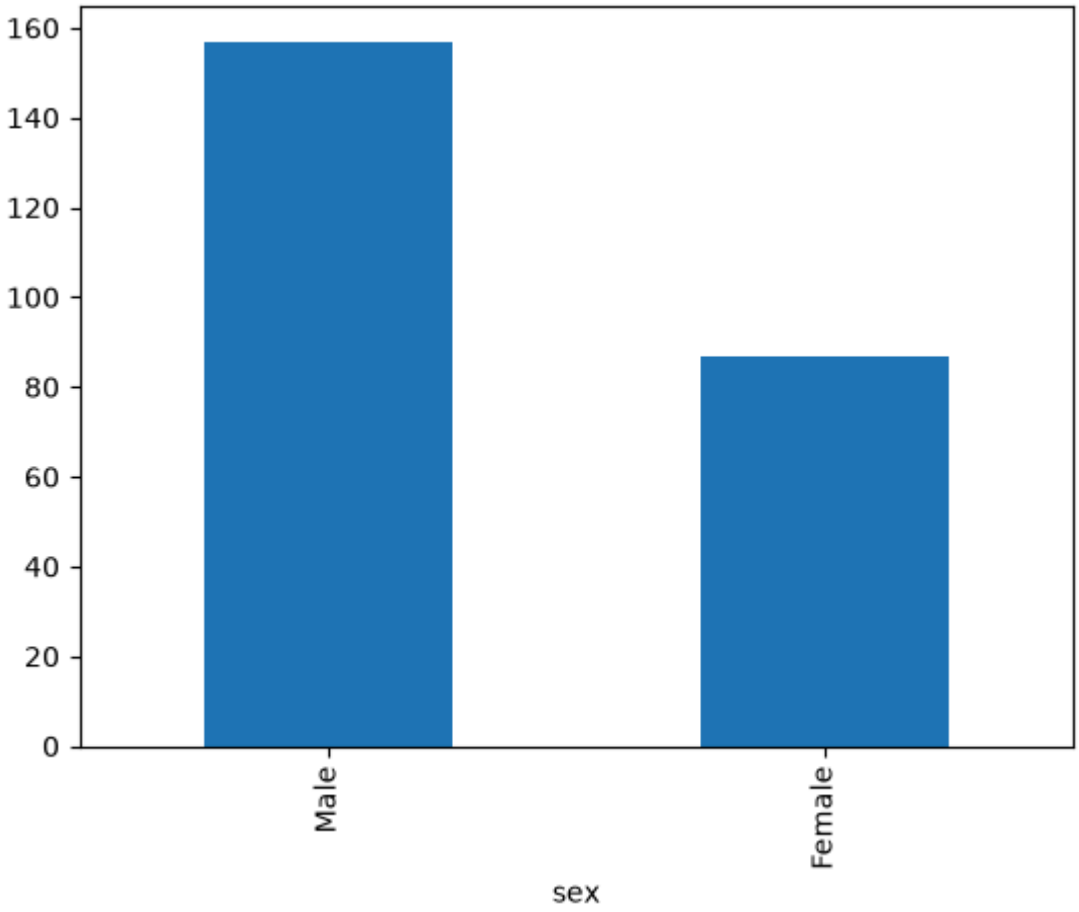

Male
Female
sex
0
20
40
60
80
100
120
140
160
count


175
150
125
100
75
50
25
0
count
Lunch
Dinner
time


140
120
100
80
60
40
20
0
count
Yes
No
smoker


160
140
120
100
80
60
40
20
0
count
1
2
3
4
5
6
size


Male
64.3%
35.7%
Female




```
time
Dinner    176
Lunch      68
Name: count, dtype: int64
```

In [ ]:

In [2]:
```
import numpy as np
import pandas as pd
df=pd.read_csv('Salary_data csv')
df
```

Out[2]:

	YearsExperience	Salary
0	1.1	39343
1	1.3	46205
2	1.5	37731
3	2.0	43525
4	2.2	39891
5	2.9	56642
6	3.0	60150
7	3.2	54445
8	3.2	64445
9	3.7	57189
10	3.9	63218
11	4.0	55794
12	4.0	56957
13	4.1	57081
14	4.5	61111
15	4.9	67938
16	5.1	66029
17	5.3	83088
18	5.9	81363
19	6.0	93940
20	6.8	91738
21	7.1	98273
22	7.9	101302
23	8.2	113812
24	8.7	109431
25	9.0	105582
26	9.5	116969
27	9.6	112635
28	10.3	122391
29	10.5	121872

In [3]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

In [4]:
```
df.dropna(inplace=True)
```

In [5]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

In [6]:
```
df.describe()
```

Out[6]:

	YearsExperience	Salary
count	30.000000	30.000000
mean	5.313333	76003.000000
std	2.837888	27414.429785
min	1.100000	37731.000000
25%	3.200000	56720.750000
50%	4.700000	65237.000000
75%	7.700000	100544.750000
max	10.500000	122391.000000

In [7]:
```
features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values
```

In [9]:
```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,rand
```

In [10]:
```
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
```

Out[10]:

▾ LinearRegression

▸ Parameters

▸ Fitted attributes

In [11]:
```
model.score(x_train,y_train)
```

Out[11]: 0.9411949620562126

In [12]:
```
model.score(x_test,y_test)
```

Out[12]: 0.988169515729126

In [13]:
```
model.coef_
```

Out[13]: array([[9312.57512673]])

In [14]:
```
model.intercept_
```

Out[14]: array([26780.09915063])

In [15]:
```
import pickle
pickle.dump(model,open('SalaryPred.model','wb'))
```

In [16]:
```
model=pickle.load(open('SalaryPred.model','rb'))
```

In [17]:
```
yr_of_exp=float(input("Enter Years of Experience: "))
yr_of_exp_NP=np.array([[yr_of_exp]])
Salary=model.predict(yr_of_exp_NP)
```

In [19]:
```
print("Estimated Salary for {} years of experience is {}: " .format(yr_of_exp,Sa
```

```
Estimated Salary for 44.0 years of experience is [[436533.40472671]]:
```

In [ ]:

In [1]:
```
import numpy as np
import pandas as pd
df=pd.read_csv('Social_Network_Ads.csv')
df
```

Out[1]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

In [2]:
```
df.head()
```

Out[2]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

In [3]:
```
features=df.iloc[:,[2,3]].values
label=df.iloc[:,4].values
features
```

Out[3]:
```
array([[    19,  19000],
       [    35,  20000],
       [    26,  43000],
       [    27,  57000],
       [    19,  76000],
       [    27,  58000],
       [    27,  84000],
       [    32, 150000],
       [    25,  33000],
       [    35,  65000],
       [    26,  80000],
       [    26,  52000],
       [    20,  86000],
       [    32,  18000],
       [    18,  82000],
       [    29,  80000],
       [    47,  25000],
       [    45,  26000],
       [    46,  28000],
       [    48,  29000],
       [    45,  22000],
       [    47,  49000],
       [    48,  41000],
       [    45,  22000],
       [    46,  23000],
       [    47,  20000],
       [    49,  28000],
       [    47,  30000],
       [    29,  43000],
       [    31,  18000],
       [    31,  74000],
       [    27, 137000],
       [    21,  16000],
       [    28,  44000],
       [    27,  90000],
       [    35,  27000],
       [    33,  28000],
       [    30,  49000],
       [    26,  72000],
       [    27,  31000],
       [    27,  17000],
       [    33,  51000],
       [    35, 108000],
       [    30,  15000],
       [    28,  84000],
       [    23,  20000],
       [    25,  79000],
       [    27,  54000],
       [    30, 135000],
       [    31,  89000],
       [    24,  32000],
       [    18,  44000],
       [    29,  83000],
       [    35,  23000],
       [    27,  58000],
       [    24,  55000],
       [    23,  48000],
       [    28,  79000],
       [    22,  18000],
       [    32, 117000],
```

```
[    27,  20000],
[    25,  87000],
[    23,  66000],
[    32, 120000],
[    59,  83000],
[    24,  58000],
[    24,  19000],
[    23,  82000],
[    22,  63000],
[    31,  68000],
[    25,  80000],
[    24,  27000],
[    20,  23000],
[    33, 113000],
[    32,  18000],
[    34, 112000],
[    18,  52000],
[    22,  27000],
[    28,  87000],
[    26,  17000],
[    30,  80000],
[    39,  42000],
[    20,  49000],
[    35,  88000],
[    30,  62000],
[    31, 118000],
[    24,  55000],
[    28,  85000],
[    26,  81000],
[    35,  50000],
[    22,  81000],
[    30, 116000],
[    26,  15000],
[    29,  28000],
[    29,  83000],
[    35,  44000],
[    35,  25000],
[    28, 123000],
[    35,  73000],
[    28,  37000],
[    27,  88000],
[    28,  59000],
[    32,  86000],
[    33, 149000],
[    19,  21000],
[    21,  72000],
[    26,  35000],
[    27,  89000],
[    26,  86000],
[    38,  80000],
[    39,  71000],
[    37,  71000],
[    38,  61000],
[    37,  55000],
[    42,  80000],
[    40,  57000],
[    35,  75000],
[    36,  52000],
[    40,  59000],
[    41,  59000],
```

```
[    36,  75000],
[    37,  72000],
[    40,  75000],
[    35,  53000],
[    41,  51000],
[    39,  61000],
[    42,  65000],
[    26,  32000],
[    30,  17000],
[    26,  84000],
[    31,  58000],
[    33,  31000],
[    30,  87000],
[    21,  68000],
[    28,  55000],
[    23,  63000],
[    20,  82000],
[    30, 107000],
[    28,  59000],
[    19,  25000],
[    19,  85000],
[    18,  68000],
[    35,  59000],
[    30,  89000],
[    34,  25000],
[    24,  89000],
[    27,  96000],
[    41,  30000],
[    29,  61000],
[    20,  74000],
[    26,  15000],
[    41,  45000],
[    31,  76000],
[    36,  50000],
[    40,  47000],
[    31,  15000],
[    46,  59000],
[    29,  75000],
[    26,  30000],
[    32, 135000],
[    32, 100000],
[    25,  90000],
[    37,  33000],
[    35,  38000],
[    33,  69000],
[    18,  86000],
[    22,  55000],
[    35,  71000],
[    29, 148000],
[    29,  47000],
[    21,  88000],
[    34, 115000],
[    26, 118000],
[    34,  43000],
[    34,  72000],
[    23,  28000],
[    35,  47000],
[    25,  22000],
[    24,  23000],
[    31,  34000],
```

```
[    26,  16000],
[    31,  71000],
[    32, 117000],
[    33,  43000],
[    33,  60000],
[    31,  66000],
[    20,  82000],
[    33,  41000],
[    35,  72000],
[    28,  32000],
[    24,  84000],
[    19,  26000],
[    29,  43000],
[    19,  70000],
[    28,  89000],
[    34,  43000],
[    30,  79000],
[    20,  36000],
[    26,  80000],
[    35,  22000],
[    35,  39000],
[    49,  74000],
[    39, 134000],
[    41,  71000],
[    58, 101000],
[    47,  47000],
[    55, 130000],
[    52, 114000],
[    40, 142000],
[    46,  22000],
[    48,  96000],
[    52, 150000],
[    59,  42000],
[    35,  58000],
[    47,  43000],
[    60, 108000],
[    49,  65000],
[    40,  78000],
[    46,  96000],
[    59, 143000],
[    41,  80000],
[    35,  91000],
[    37, 144000],
[    60, 102000],
[    35,  60000],
[    37,  53000],
[    36, 126000],
[    56, 133000],
[    40,  72000],
[    42,  80000],
[    35, 147000],
[    39,  42000],
[    40, 107000],
[    49,  86000],
[    38, 112000],
[    46,  79000],
[    40,  57000],
[    37,  80000],
[    46,  82000],
[    53, 143000],
```

```
[    42, 149000],
[    38,  59000],
[    50,  88000],
[    56, 104000],
[    41,  72000],
[    51, 146000],
[    35,  50000],
[    57, 122000],
[    41,  52000],
[    35,  97000],
[    44,  39000],
[    37,  52000],
[    48, 134000],
[    37, 146000],
[    50,  44000],
[    52,  90000],
[    41,  72000],
[    40,  57000],
[    58,  95000],
[    45, 131000],
[    35,  77000],
[    36, 144000],
[    55, 125000],
[    35,  72000],
[    48,  90000],
[    42, 108000],
[    40,  75000],
[    37,  74000],
[    47, 144000],
[    40,  61000],
[    43, 133000],
[    59,  76000],
[    60,  42000],
[    39, 106000],
[    57,  26000],
[    57,  74000],
[    38,  71000],
[    49,  88000],
[    52,  38000],
[    50,  36000],
[    59,  88000],
[    35,  61000],
[    37,  70000],
[    52,  21000],
[    48, 141000],
[    37,  93000],
[    37,  62000],
[    48, 138000],
[    41,  79000],
[    37,  78000],
[    39, 134000],
[    49,  89000],
[    55,  39000],
[    37,  77000],
[    35,  57000],
[    36,  63000],
[    42,  73000],
[    43, 112000],
[    45,  79000],
[    46, 117000],
```

```
[    58,  38000],
[    48,  74000],
[    37, 137000],
[    37,  79000],
[    40,  60000],
[    42,  54000],
[    51, 134000],
[    47, 113000],
[    36, 125000],
[    38,  50000],
[    42,  70000],
[    39,  96000],
[    38,  50000],
[    49, 141000],
[    39,  79000],
[    39,  75000],
[    54, 104000],
[    35,  55000],
[    45,  32000],
[    36,  60000],
[    52, 138000],
[    53,  82000],
[    41,  52000],
[    48,  30000],
[    48, 131000],
[    41,  60000],
[    41,  72000],
[    42,  75000],
[    36, 118000],
[    47, 107000],
[    38,  51000],
[    48, 119000],
[    42,  65000],
[    40,  65000],
[    57,  60000],
[    36,  54000],
[    58, 144000],
[    35,  79000],
[    38,  55000],
[    39, 122000],
[    53, 104000],
[    35,  75000],
[    38,  65000],
[    47,  51000],
[    47, 105000],
[    41,  63000],
[    53,  72000],
[    54, 108000],
[    39,  77000],
[    38,  61000],
[    38, 113000],
[    37,  75000],
[    42,  90000],
[    37,  57000],
[    36,  99000],
[    60,  34000],
[    54,  70000],
[    41,  72000],
[    40,  71000],
[    42,  54000],
```

```
[    43, 129000],
[    53,  34000],
[    47,  50000],
[    42,  79000],
[    42, 104000],
[    59,  29000],
[    58,  47000],
[    46,  88000],
[    38,  71000],
[    54,  26000],
[    60,  46000],
[    60,  83000],
[    39,  73000],
[    59, 130000],
[    37,  80000],
[    46,  32000],
[    46,  74000],
[    42,  53000],
[    41,  87000],
[    58,  23000],
[    42,  64000],
[    48,  33000],
[    44, 139000],
[    49,  28000],
[    57,  33000],
[    56,  60000],
[    49,  39000],
[    39,  71000],
[    47,  34000],
[    48,  35000],
[    48,  33000],
[    47,  23000],
[    45,  45000],
[    60,  42000],
[    39,  59000],
[    46,  41000],
[    51,  23000],
[    50,  20000],
[    36,  33000],
[    49,  36000]])
```

In [4]:
```
label
```

Out[4]:
```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1,
       0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0,
       1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0,
       1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
       0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1,
       1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1,
       0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0,
       1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1,
       0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1,
       1, 1, 0, 1])
```

In [5]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

In [10]:
```
for i in range(1,401):
    x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,
    model=LogisticRegression()
    model.fit(x_train,y_train)
    train_score=model.score(x_train,y_train)
    test_score=model.score(x_test,y_test)
    if test_score>train_score:
        print("Test {} Train{} Random State {}".format(test_score,train_score,i)
```

```
Test 0.9 Train0.840625 Random State 4
Test 0.8625 Train0.85 Random State 5
Test 0.8625 Train0.859375 Random State 6
Test 0.8875 Train0.8375 Random State 7
Test 0.8625 Train0.8375 Random State 9
Test 0.9 Train0.840625 Random State 10
Test 0.8625 Train0.85625 Random State 14
Test 0.85 Train0.84375 Random State 15
Test 0.8625 Train0.85625 Random State 16
Test 0.875 Train0.834375 Random State 18
Test 0.85 Train0.84375 Random State 19
Test 0.875 Train0.84375 Random State 20
Test 0.8625 Train0.834375 Random State 21
Test 0.875 Train0.840625 Random State 22
Test 0.875 Train0.840625 Random State 24
Test 0.85 Train0.834375 Random State 26
Test 0.85 Train0.840625 Random State 27
Test 0.8625 Train0.834375 Random State 30
Test 0.8625 Train0.85625 Random State 31
Test 0.875 Train0.853125 Random State 32
Test 0.8625 Train0.84375 Random State 33
Test 0.875 Train0.83125 Random State 35
Test 0.8625 Train0.853125 Random State 36
Test 0.8875 Train0.840625 Random State 38
Test 0.875 Train0.8375 Random State 39
Test 0.8875 Train0.8375 Random State 42
Test 0.875 Train0.846875 Random State 46
Test 0.9125 Train0.83125 Random State 47
Test 0.875 Train0.83125 Random State 51
Test 0.9 Train0.84375 Random State 54
Test 0.85 Train0.84375 Random State 57
Test 0.875 Train0.84375 Random State 58
Test 0.925 Train0.8375 Random State 61
Test 0.8875 Train0.834375 Random State 65
Test 0.8875 Train0.840625 Random State 68
Test 0.9 Train0.83125 Random State 72
Test 0.8875 Train0.8375 Random State 75
Test 0.925 Train0.825 Random State 76
Test 0.8625 Train0.840625 Random State 77
Test 0.8625 Train0.859375 Random State 81
Test 0.875 Train0.8375 Random State 82
Test 0.8875 Train0.8375 Random State 83
Test 0.8625 Train0.853125 Random State 84
Test 0.8625 Train0.840625 Random State 85
Test 0.8625 Train0.840625 Random State 87
Test 0.875 Train0.846875 Random State 88
Test 0.9125 Train0.8375 Random State 90
Test 0.8625 Train0.85 Random State 95
Test 0.875 Train0.85 Random State 99
Test 0.85 Train0.840625 Random State 101
Test 0.85 Train0.840625 Random State 102
Test 0.9 Train0.825 Random State 106
Test 0.8625 Train0.840625 Random State 107
Test 0.85 Train0.834375 Random State 109
Test 0.85 Train0.840625 Random State 111
Test 0.9125 Train0.840625 Random State 112
Test 0.8625 Train0.85 Random State 115
Test 0.8625 Train0.840625 Random State 116
Test 0.875 Train0.834375 Random State 119
Test 0.9125 Train0.828125 Random State 120
```

```
Test 0.8625 Train0.859375 Random State 125
Test 0.85 Train0.846875 Random State 128
Test 0.875 Train0.85 Random State 130
Test 0.9 Train0.84375 Random State 133
Test 0.925 Train0.834375 Random State 134
Test 0.8625 Train0.85 Random State 135
Test 0.875 Train0.83125 Random State 138
Test 0.8625 Train0.85 Random State 141
Test 0.85 Train0.846875 Random State 143
Test 0.85 Train0.846875 Random State 146
Test 0.85 Train0.84375 Random State 147
Test 0.8625 Train0.85 Random State 148
Test 0.875 Train0.8375 Random State 150
Test 0.8875 Train0.83125 Random State 151
Test 0.925 Train0.84375 Random State 152
Test 0.85 Train0.840625 Random State 153
Test 0.9 Train0.84375 Random State 154
Test 0.9 Train0.840625 Random State 155
Test 0.8875 Train0.846875 Random State 156
Test 0.8875 Train0.834375 Random State 158
Test 0.875 Train0.828125 Random State 159
Test 0.9 Train0.83125 Random State 161
Test 0.85 Train0.8375 Random State 163
Test 0.875 Train0.83125 Random State 164
Test 0.8625 Train0.85 Random State 169
Test 0.875 Train0.840625 Random State 171
Test 0.85 Train0.840625 Random State 172
Test 0.9 Train0.825 Random State 180
Test 0.85 Train0.834375 Random State 184
Test 0.925 Train0.821875 Random State 186
Test 0.9 Train0.83125 Random State 193
Test 0.8625 Train0.85 Random State 195
Test 0.8625 Train0.840625 Random State 196
Test 0.8625 Train0.8375 Random State 197
Test 0.875 Train0.840625 Random State 198
Test 0.8875 Train0.8375 Random State 199
Test 0.8875 Train0.84375 Random State 200
Test 0.8625 Train0.8375 Random State 202
Test 0.8625 Train0.840625 Random State 203
Test 0.8875 Train0.83125 Random State 206
Test 0.8625 Train0.834375 Random State 211
Test 0.85 Train0.84375 Random State 212
Test 0.8625 Train0.834375 Random State 214
Test 0.875 Train0.83125 Random State 217
Test 0.9625 Train0.81875 Random State 220
Test 0.875 Train0.84375 Random State 221
Test 0.85 Train0.840625 Random State 222
Test 0.9 Train0.84375 Random State 223
Test 0.8625 Train0.853125 Random State 227
Test 0.8625 Train0.834375 Random State 228
Test 0.9 Train0.840625 Random State 229
Test 0.85 Train0.84375 Random State 232
Test 0.875 Train0.846875 Random State 233
Test 0.9125 Train0.840625 Random State 234
Test 0.8625 Train0.840625 Random State 235
Test 0.85 Train0.846875 Random State 236
Test 0.875 Train0.846875 Random State 239
Test 0.85 Train0.84375 Random State 241
Test 0.8875 Train0.85 Random State 242
Test 0.8875 Train0.825 Random State 243
```

```
Test 0.875 Train0.846875 Random State 244
Test 0.875 Train0.840625 Random State 245
Test 0.875 Train0.846875 Random State 246
Test 0.8625 Train0.859375 Random State 247
Test 0.8875 Train0.84375 Random State 248
Test 0.8625 Train0.85 Random State 250
Test 0.875 Train0.83125 Random State 251
Test 0.8875 Train0.84375 Random State 252
Test 0.8625 Train0.846875 Random State 255
Test 0.9 Train0.840625 Random State 257
Test 0.8625 Train0.85625 Random State 260
Test 0.8625 Train0.840625 Random State 266
Test 0.8625 Train0.8375 Random State 268
Test 0.875 Train0.840625 Random State 275
Test 0.8625 Train0.85 Random State 276
Test 0.925 Train0.8375 Random State 277
Test 0.875 Train0.846875 Random State 282
Test 0.85 Train0.846875 Random State 283
Test 0.85 Train0.84375 Random State 285
Test 0.9125 Train0.834375 Random State 286
Test 0.85 Train0.840625 Random State 290
Test 0.85 Train0.840625 Random State 291
Test 0.85 Train0.846875 Random State 292
Test 0.8625 Train0.8375 Random State 294
Test 0.8875 Train0.828125 Random State 297
Test 0.8625 Train0.834375 Random State 300
Test 0.8625 Train0.85 Random State 301
Test 0.8875 Train0.85 Random State 302
Test 0.875 Train0.846875 Random State 303
Test 0.8625 Train0.834375 Random State 305
Test 0.9125 Train0.8375 Random State 306
Test 0.875 Train0.846875 Random State 308
Test 0.9 Train0.84375 Random State 311
Test 0.8625 Train0.834375 Random State 313
Test 0.9125 Train0.834375 Random State 314
Test 0.875 Train0.8375 Random State 315
Test 0.9 Train0.846875 Random State 317
Test 0.9125 Train0.821875 Random State 319
Test 0.8625 Train0.85 Random State 321
Test 0.9125 Train0.828125 Random State 322
Test 0.85 Train0.846875 Random State 328
Test 0.85 Train0.8375 Random State 332
Test 0.8875 Train0.853125 Random State 336
Test 0.85 Train0.8375 Random State 337
Test 0.875 Train0.840625 Random State 343
Test 0.8625 Train0.84375 Random State 346
Test 0.8875 Train0.83125 Random State 351
Test 0.8625 Train0.85 Random State 352
Test 0.95 Train0.81875 Random State 354
Test 0.8625 Train0.85 Random State 356
Test 0.9125 Train0.840625 Random State 357
Test 0.8625 Train0.8375 Random State 358
Test 0.85 Train0.840625 Random State 362
Test 0.9 Train0.84375 Random State 363
Test 0.8625 Train0.853125 Random State 364
Test 0.9375 Train0.821875 Random State 366
Test 0.9125 Train0.840625 Random State 369
Test 0.8625 Train0.853125 Random State 371
Test 0.925 Train0.834375 Random State 376
Test 0.9125 Train0.828125 Random State 377
```

```
Test 0.8875 Train0.85 Random State 378
Test 0.8875 Train0.85 Random State 379
Test 0.8625 Train0.840625 Random State 382
Test 0.8625 Train0.859375 Random State 386
Test 0.85 Train0.8375 Random State 387
Test 0.875 Train0.828125 Random State 388
Test 0.85 Train0.84375 Random State 394
Test 0.8625 Train0.8375 Random State 395
Test 0.9 Train0.84375 Random State 397
Test 0.8625 Train0.84375 Random State 400
```

In [12]:
```
x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,rand
finalModel=LogisticRegression()
finalModel.fit(x_train,y_train)
```

Out[12]:

▾ LogisticRegression

▸ Parameters

▸ Fitted attributes

In [13]:
```
print(finalModel.score(x_train,y_train))
print(finalModel.score(x_test,y_test))
```

```
0.840625
0.9
```

In [14]:
```
from sklearn.metrics import classification_report
print(classification_report(label,finalModel.predict(features)))
```

```
              precision    recall  f1-score   support

           0       0.87      0.91      0.89       257
           1       0.82      0.75      0.78       143

    accuracy                           0.85       400
   macro avg       0.84      0.83      0.84       400
weighted avg       0.85      0.85      0.85       400
```

In [ ]:

In [5]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline

df = pd.read_csv('mall_customers.csv')

df.info()
df.head()
sns.pairplot(df)
plt.show()
features = df.iloc[:, [3, 4]].values
from sklearn.cluster import KMeans

model = KMeans(n_clusters=5, random_state=42, n_init=10)
model.fit(features)

KMeans(n_clusters=5, random_state=42, n_init=10)
Final = df.iloc[:, [3, 4]].copy()

Final['label'] = model.predict(features)

Final.head()
sns.set_style("whitegrid")

sns.FacetGrid(Final, hue="label", height=8) \
    .map(plt.scatter, "Annual Income (k$)", "Spending Score (1-100)") \
    .add_legend()

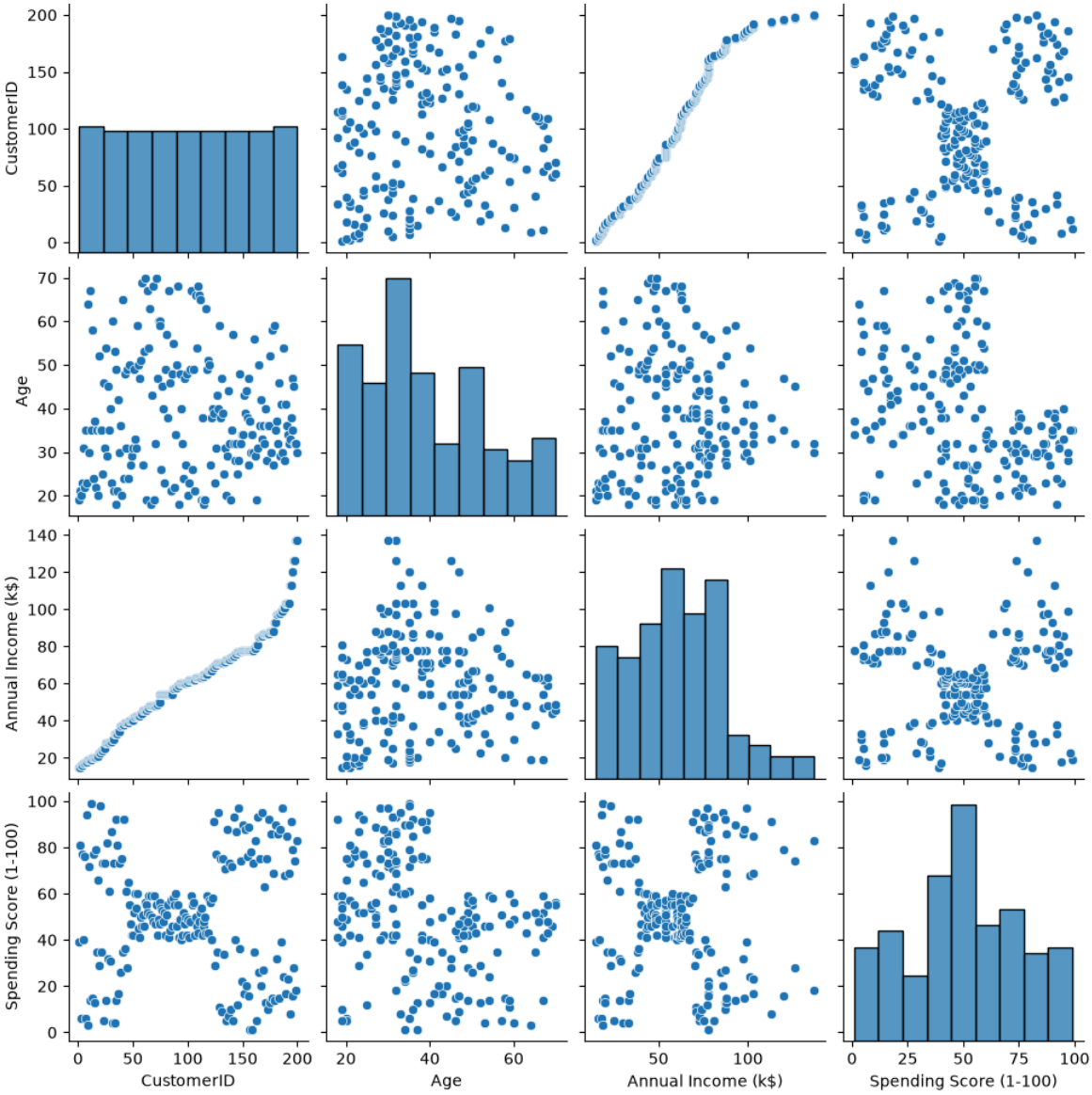
plt.show()
features_el = df.iloc[:, [2, 3, 4]].values

wcss = []

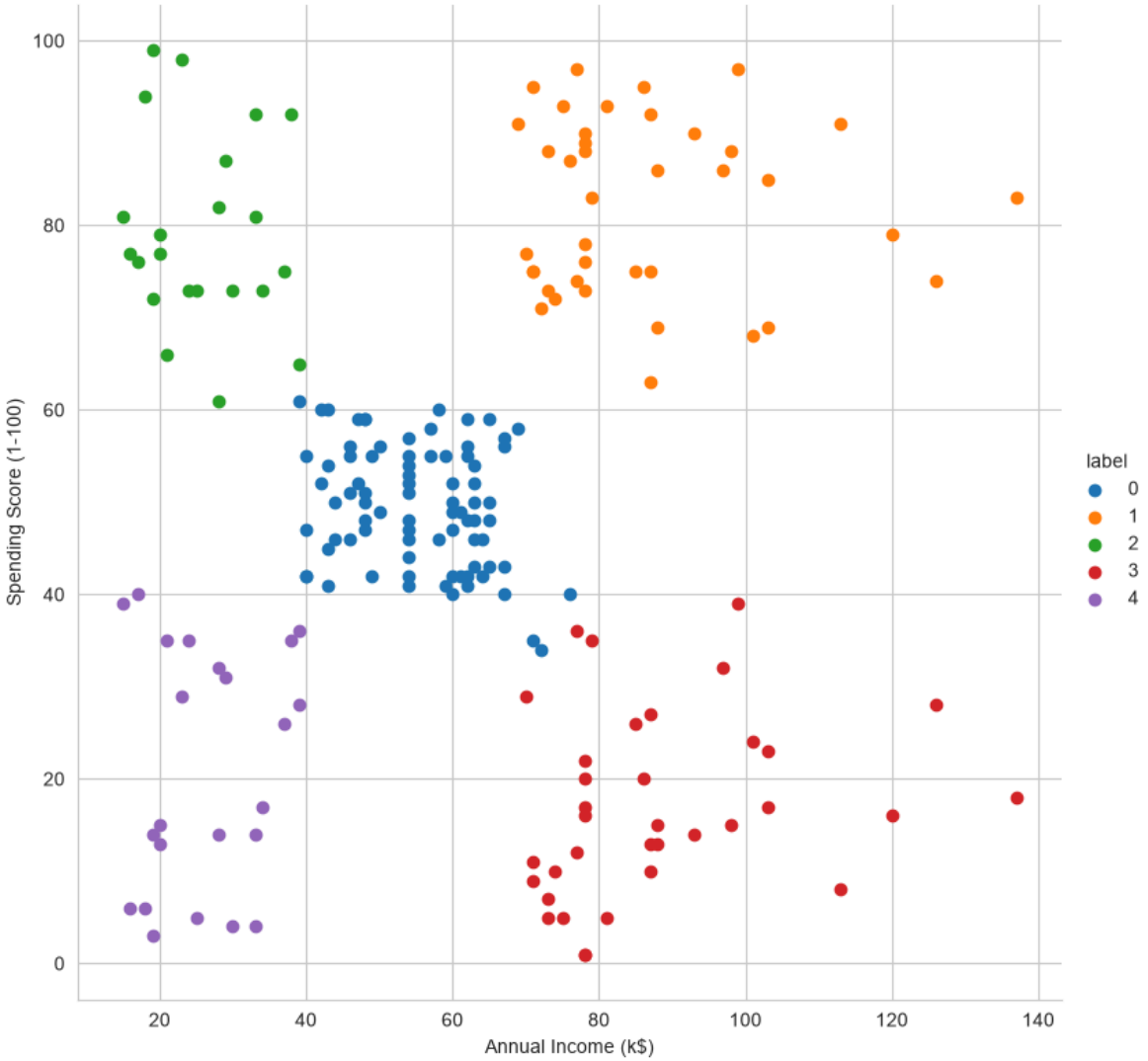
for i in range(1, 10):
    model = KMeans(
        n_clusters=i,
        random_state=42,
        n_init=10
    )
    model.fit(features_el)
    wcss.append(model.inertia_)

plt.plot(range(1, 10), wcss, marker='o')
plt.title('The Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()
```

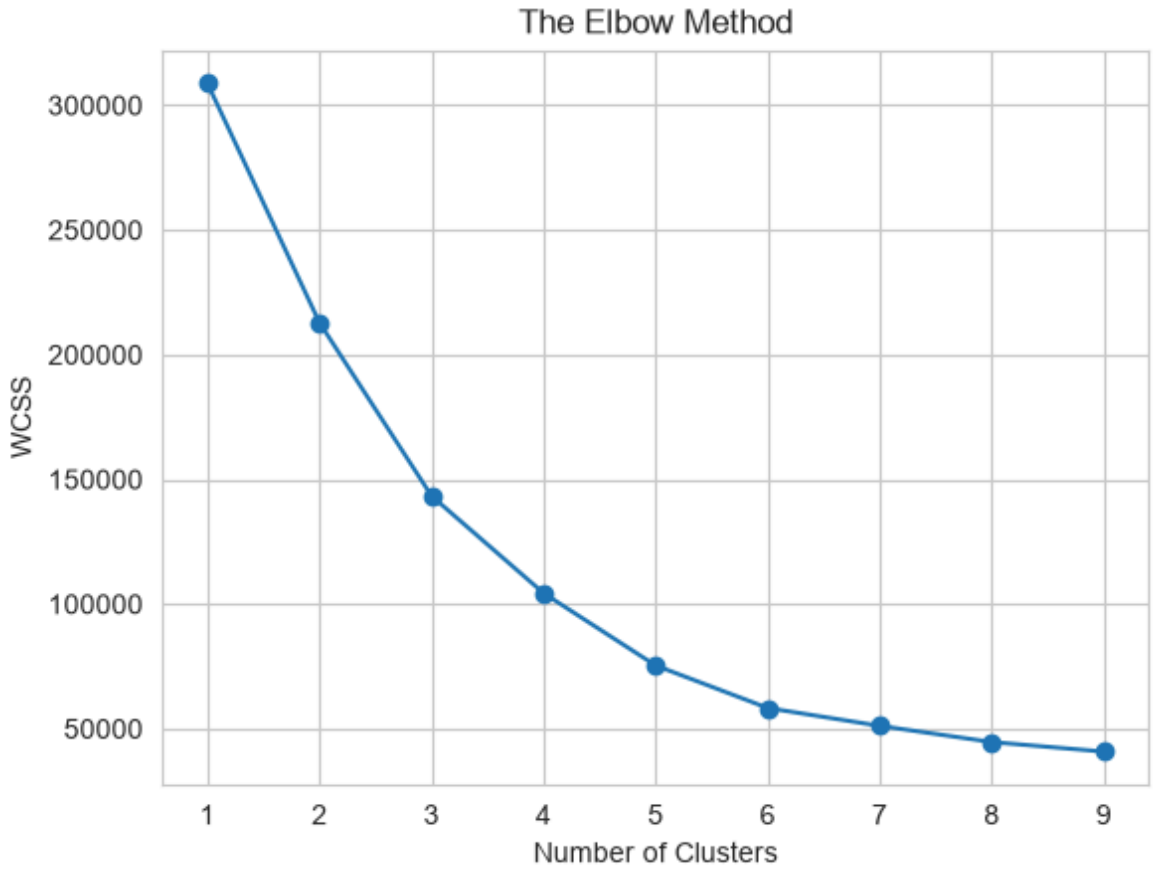
```
<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    str
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), str(1)
memory usage: 8.9 KB
```




```
C:\Users\srenaath\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
```


Spending Score (1-100)
Annual Income (k$)
label
0
1
2
3
4
100
80
60
40
20
0
20
40
60
80
100
120
140

```
C:\Users\srenaath\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\srenaath\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\srenaath\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\srenaath\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\srenaath\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\srenaath\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\srenaath\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\srenaath\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\srenaath\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
```


The Elbow Method
WCSS
300000
250000
200000
150000
100000
50000
1
2
3
4
5
6
7
8
9
Number of Clusters

In [ ]:

In [1]:
```
import numpy as np
import pandas as pd
```

In [2]:
```
df=pd.read_csv('Iris.csv')
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    str    
dtypes: float64(4), str(1)
memory usage: 7.2 KB
```

In [3]:
```
df.variety.value_counts()
```

Out[3]:
```
variety
Setosa        50
Versicolor    50
Virginica     50
Name: count, dtype: int64
```

In [4]:
```
df.head()
```

Out[4]:

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa

In [10]:
```
import numpy as np
features=df.iloc[:,:-1].to_numpy(dtype=float)
label=df.iloc[:,4].astype(str).to_numpy(dtype=object)
```

In [6]:
```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
```

In [11]:
```
xtrain,xtest,ytrain,ytest=train_test_split(features,label,test_size=.2,random_st
model_KNN=KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain,ytrain)
```

Out[11]:

▾ KNeighborsClassifier

▸ Parameters

▸ Fitted attributes

In [12]:
```
print(model_KNN.score(xtrain,ytrain))
print(model_KNN.score(xtest,ytest))
```

```
0.95
0.9666666666666667
```

In [13]:
```
from sklearn.metrics import confusion_matrix
confusion_matrix(label,model_KNN.predict(features))
```

Out[13]:
```
array([[50,  0,  0],
       [ 0, 45,  5],
       [ 0,  2, 48]])
```

In [14]:
```
from sklearn.metrics import classification_report
print(classification_report(label,model_KNN.predict(features)))
```

```
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        50
  Versicolor       0.96      0.90      0.93        50
   Virginica       0.91      0.96      0.93        50

    accuracy                           0.95       150
   macro avg       0.95      0.95      0.95       150
weighted avg       0.95      0.95      0.95       150
```

In [ ]:

In [2]:
```python
import numpy as np
from scipy import stats
# Sample data
marks = np.array([72, 68, 75, 70, 74, 69, 71, 73, 70, 72])
# Hypothesized mean
mu_0 = 70
# One-sample t-test
t_stat, p_value = stats.ttest_1samp(marks, mu_0)
print(f"T-statistic: {t_stat:.3f}")
print(f"P-value: {p_value:.4f}")
alpha = 0.05
if p_value < alpha:
    print("Reject Null Hypothesis → Mean is significantly different from 70.")
else:
    print("Fail to Reject Null Hypothesis → No significant difference.")
```

```
T-statistic: 1.993
P-value: 0.0774
Fail to Reject Null Hypothesis → No significant difference.
```

In [3]:
```python
import numpy as np
from math import sqrt
from scipy.stats import norm
# Given data
x_bar = 51.2 # sample mean
mu_0 = 50 # population mean
sigma = 3 # population standard deviation
n = 36 # sample size
# Calculate Z-statistic
z_stat = (x_bar - mu_0) / (sigma / sqrt(n))
# Two-tailed p-value
p_value = 2 * (1 - norm.cdf(abs(z_stat)))
print(f"Z-statistic: {z_stat:.3f}")
print(f"P-value: {p_value:.4f}")
alpha = 0.05
if p_value < alpha:
    print("Reject Null Hypothesis → Mean is significantly different from 50 g.")
else:
    print("Fail to Reject Null Hypothesis → No significant difference.")
```

```
Z-statistic: 2.400
P-value: 0.0164
Reject Null Hypothesis → Mean is significantly different from 50 g.
```

In [4]:
```python
import numpy as np
from scipy import stats

# Data
A = [20, 22, 23]
B = [19, 20, 18]
C = [25, 27, 26]

# Perform one-way ANOVA
f_stat, p_value = stats.f_oneway(A, B, C)
```

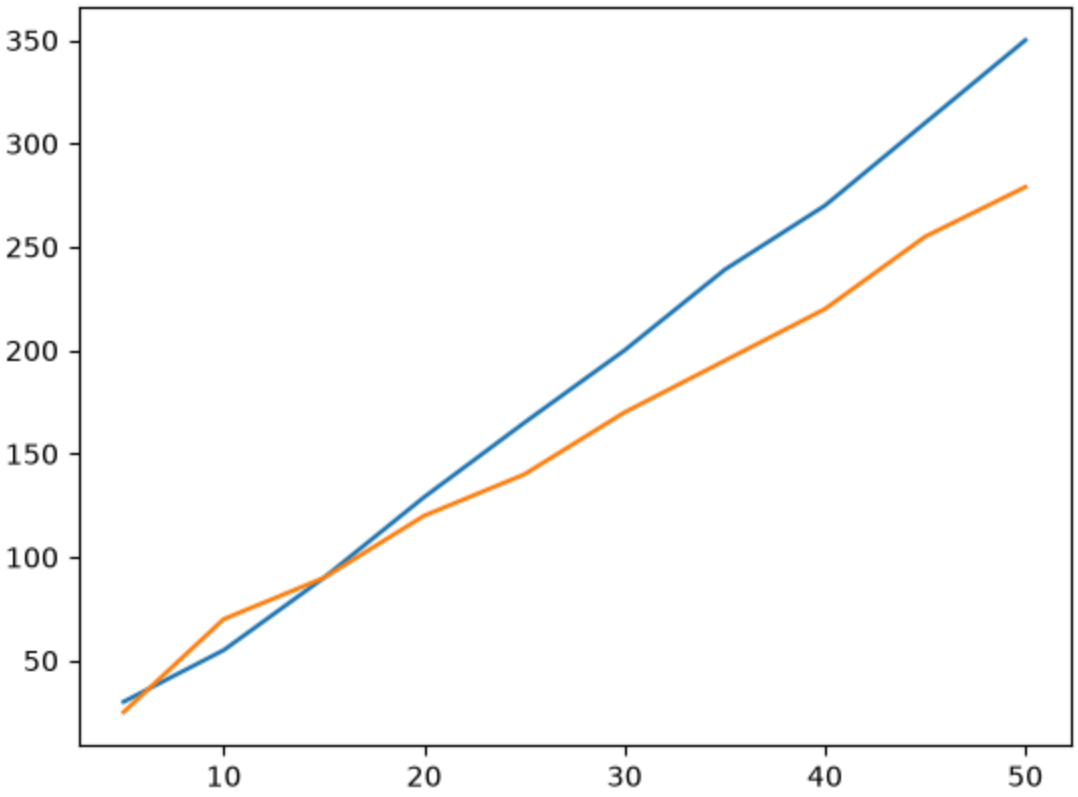
```
print(f"F-statistic: {f_stat:.3f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
    print("Reject Null Hypothesis → Means are significantly different.")
else:
    print("Fail to Reject Null Hypothesis → No significant difference.")
```

```
F-statistic: 25.923
P-value: 0.0011
Reject Null Hypothesis → Means are significantly different.
```

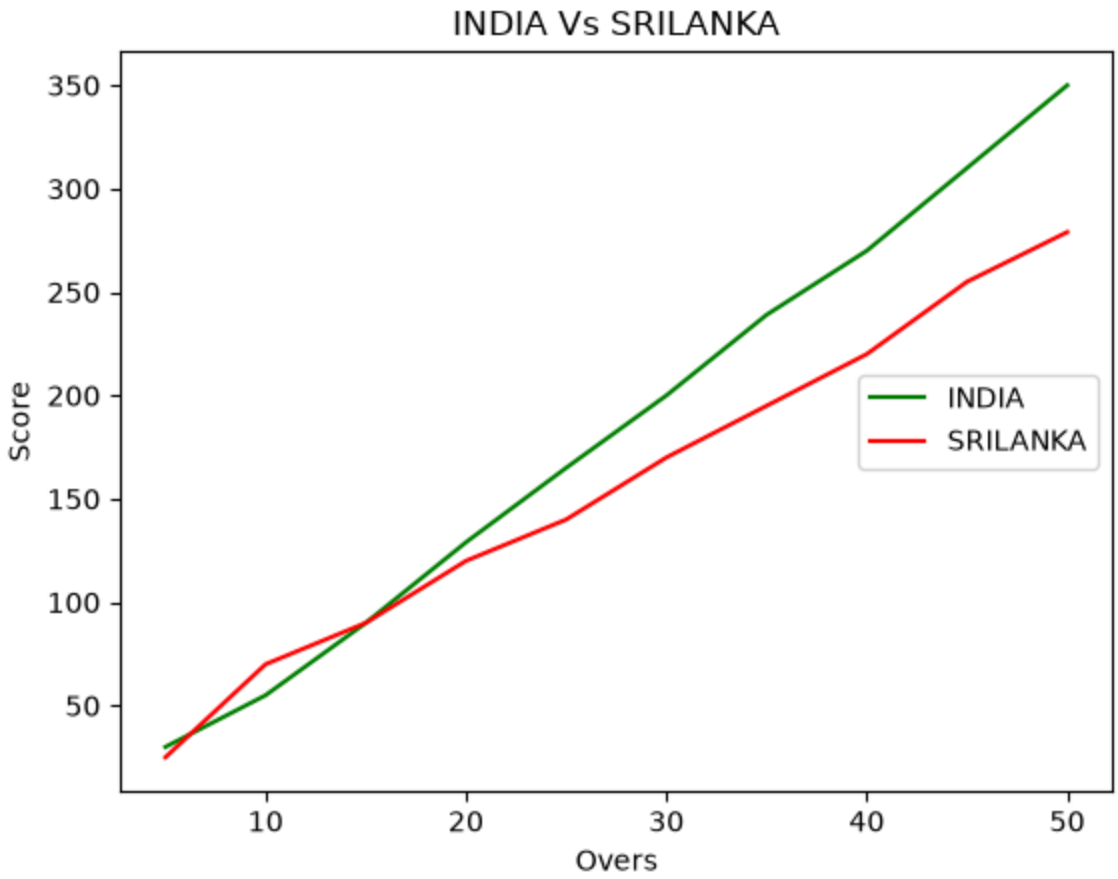
In [ ]:

In [1]:
```
import matplotlib.pyplot as cricket
Overs=list(range(5,51,5))
Indian_Score=[30,55,90,129,165,200,239,270,310,350]
Srilankan_Score=[25,70,90,120,140,170,195,220,255,279]
cricket.plot(Overs,Indian_Score)
cricket.plot(Overs,Srilankan_Score)
cricket.show()
cricket.title("INDIA Vs SRILANKA")
cricket.xlabel("Overs")
cricket.ylabel("Score")
cricket.legend()
cricket.plot(Overs,Indian_Score,color="green",label="INDIA")
cricket.plot(Overs,Srilankan_Score,color="red",label="SRILANKA")
cricket.legend(loc="center right")
```

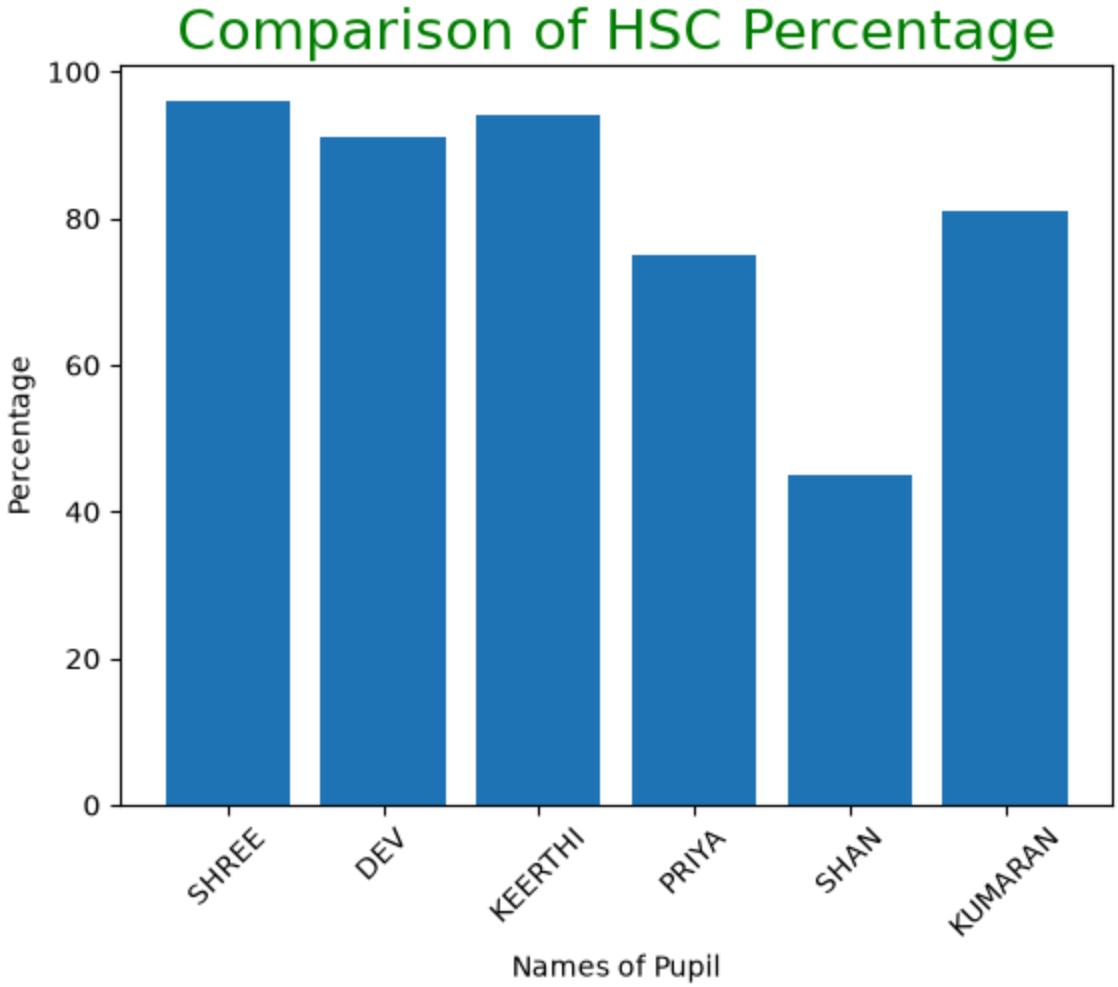



```
C:\Users\user\AppData\Local\Temp\ipykernel_15452\2239802548.py:11: UserWarning: No a
rtists with labels found to put in legend.  Note that artists whose label start with
an underscore are ignored when legend() is called with no argument.
  cricket.legend()
```

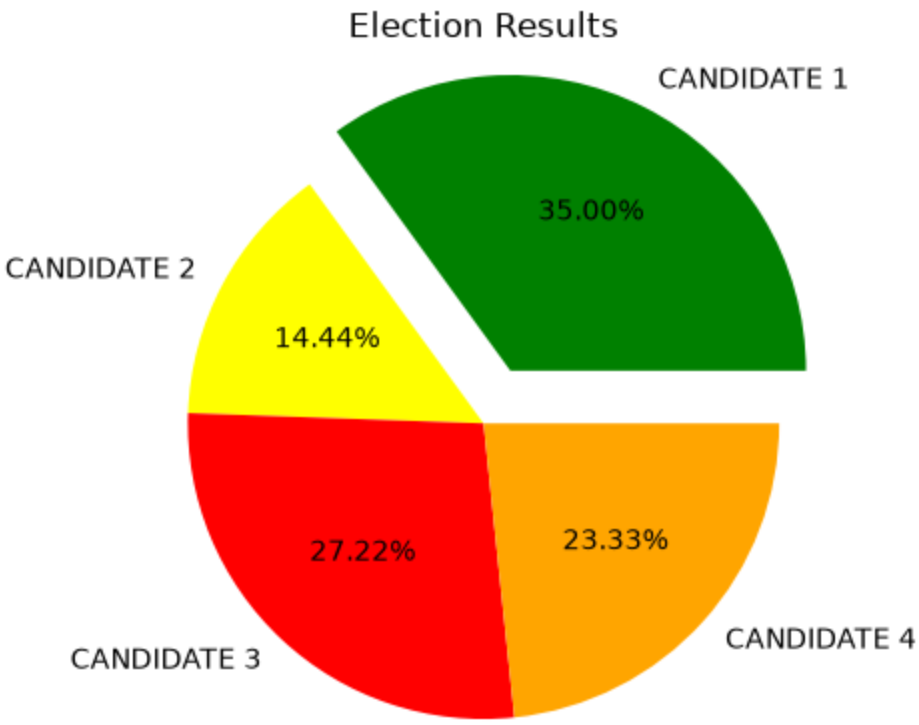
Out[1]: <matplotlib.legend.Legend at 0x24a2e307770>



In [2]:
```
import matplotlib.pyplot as hscmark
import numpy as np
Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']
xaxis = np.arange(len(Names))
Percentage_hsc = [96, 91, 94, 75, 45, 81]
hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
hscmark.title("Comparison of HSC Percentage", fontsize=20, color="green")
hscmark.show()
```



In [3]:
```
import matplotlib.pyplot as election
# Election data
labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']
Votes = [315, 130, 245, 210]
colors = ['green', 'yellow', 'red', 'orange']
explode = (0.2, 0, 0, 0)
# Plotting the pie chart
election.pie(Votes, labels=labels, colors=colors, explode=explode, autopct='%0.2f%%
election.title('Election Results')
election.show()
```


Election Results
CANDIDATE 1
35.00%
CANDIDATE 2
14.44%
27.22%
23.33%
CANDIDATE 3
CANDIDATE 4

In [ ]:

In [1]:
```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg
from collections import Counter
import string

nltk.download('gutenberg')
nltk.download('punkt')
nltk.download('punkt_tab')

# Load the text
sample = gutenberg.raw("austen-emma.txt")

# Tokenize the text
tokens = word_tokenize(sample)

# Select the first 50 word tokens, excluding punctuation
wlist = [
    word for word in tokens
    if word not in string.punctuation
][:50]

# Calculate word frequencies
wordfreq = Counter(wlist)

# Display the result
print("Word-Frequency Pairs:")
print(list(wordfreq.items()))
```

```
[nltk_data] Downloading package gutenberg to
[nltk_data]     C:\Users\user\AppData\Roaming\nltk_data...
[nltk_data]   Package gutenberg is already up-to-date!
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\user\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\user\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
Word-Frequency Pairs:
[('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816', 1), ('VOLUME', 1),
('I', 2), ('CHAPTER', 1), ('Woodhouse', 1), ('handsome', 1), ('clever', 1), ('and',
3), ('rich', 1), ('with', 2), ('a', 1), ('comfortable', 1), ('home', 1), ('happy',
1), ('disposition', 1), ('seemed', 1), ('to', 2), ('unite', 1), ('some', 1), ('of',
2), ('the', 2), ('best', 1), ('blessings', 1), ('existence', 1), ('had', 1), ('live
d', 1), ('nearly', 1), ('twenty-one', 1), ('years', 1), ('in', 1), ('world', 1), ('v
ery', 1), ('little', 1), ('distress', 1), ('or', 1), ('vex', 1), ('her', 1), ('She',
1)]
```

In [ ]:

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create a sample diabetes dataset
data = {
    'Pregnancies': [6, 1, 8, 1, 0, 5, 3, 10, 2, 4],
    'Glucose': [148, 85, 183, 89, 137, 116, 78, 115, 197, 110],
    'BloodPressure': [72, 66, 64, 66, 40, 74, 50, 0, 70, 92],
    'SkinThickness': [35, 29, 0, 23, 35, 0, 32, 0, 45, 0],
    'Insulin': [0, 0, 0, 94, 168, 0, 88, 0, 543, 0],
    'BMI': [33.6, 26.6, 23.3, 28.1, 43.1, 25.6, 31.0, 35.3, 30.5, 37.6],
    'DiabetesPedigreeFunction': [0.627, 0.351, 0.672, 0.167, 2.288, 0.201, 0.248, 0
    'Age': [50, 31, 32, 21, 33, 30, 26, 29, 53, 30],
    'Outcome': [1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
}

diabetes_df = pd.DataFrame(data)

# 1. Display the first five rows
print("First Five Rows:")
print(diabetes_df.head())

# 2. Display the last five rows
print("\nLast Five Rows:")
print(diabetes_df.tail())

# 3. Display dataset information
print("\nDataset Information:")
diabetes_df.info()

# 4. Display the number of rows and columns
print("\nDataset Shape:")
print(diabetes_df.shape)

# 5. Display column names
print("\nColumn Names:")
print(diabetes_df.columns)

# 6. Check for missing values
print("\nMissing Values:")
print(diabetes_df.isnull().sum())

# 7. Display statistical summary
print("\nStatistical Summary:")
print(diabetes_df.describe())

# 8. Check duplicate rows
print("\nNumber of Duplicate Rows:")
print(diabetes_df.duplicated().sum())

# 9. Visualize the correlation between columns
plt.figure(figsize=(10, 6))
```

```
sns.heatmap(diabetes_df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title("Correlation Heatmap of Diabetes Dataset")
plt.show()

# 10. Visualize the outcome distribution
sns.countplot(data=diabetes_df, x='Outcome')
plt.title("Diabetes Outcome Distribution")
plt.xlabel("Outcome (0 = No Diabetes, 1 = Diabetes)")
plt.ylabel("Count")
plt.show()
```

```
First Five Rows:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1

Last Five Rows:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
5            5      116             74              0        0  25.6
6            3       78             50             32       88  31.0
7           10      115              0              0        0  35.3
8            2      197             70             45      543  30.5
9            4      110             92              0        0  37.6

   DiabetesPedigreeFunction  Age  Outcome
5                     0.201   30        0
6                     0.248   26        1
7                     0.134   29        0
8                     0.158   53        1
9                     0.191   30        0

Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               10 non-null     int64
 1   Glucose                   10 non-null     int64
 2   BloodPressure             10 non-null     int64
 3   SkinThickness             10 non-null     int64
 4   Insulin                   10 non-null     int64
 5   BMI                       10 non-null     float64
 6   DiabetesPedigreeFunction  10 non-null     float64
 7   Age                       10 non-null     int64
 8   Outcome                   10 non-null     int64
dtypes: float64(2), int64(7)
memory usage: 852.0 bytes

Dataset Shape:
(10, 9)

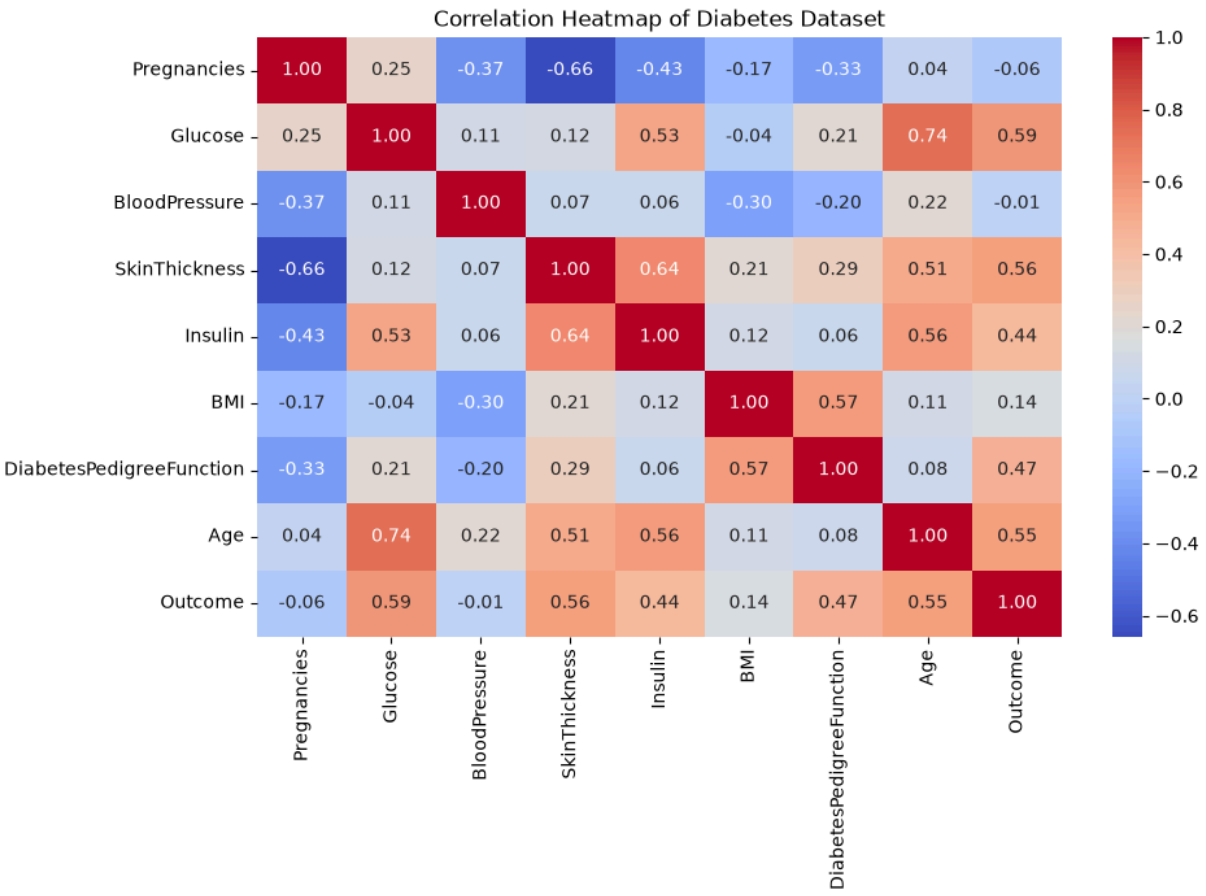
Column Names:
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],
      dtype='str')
```

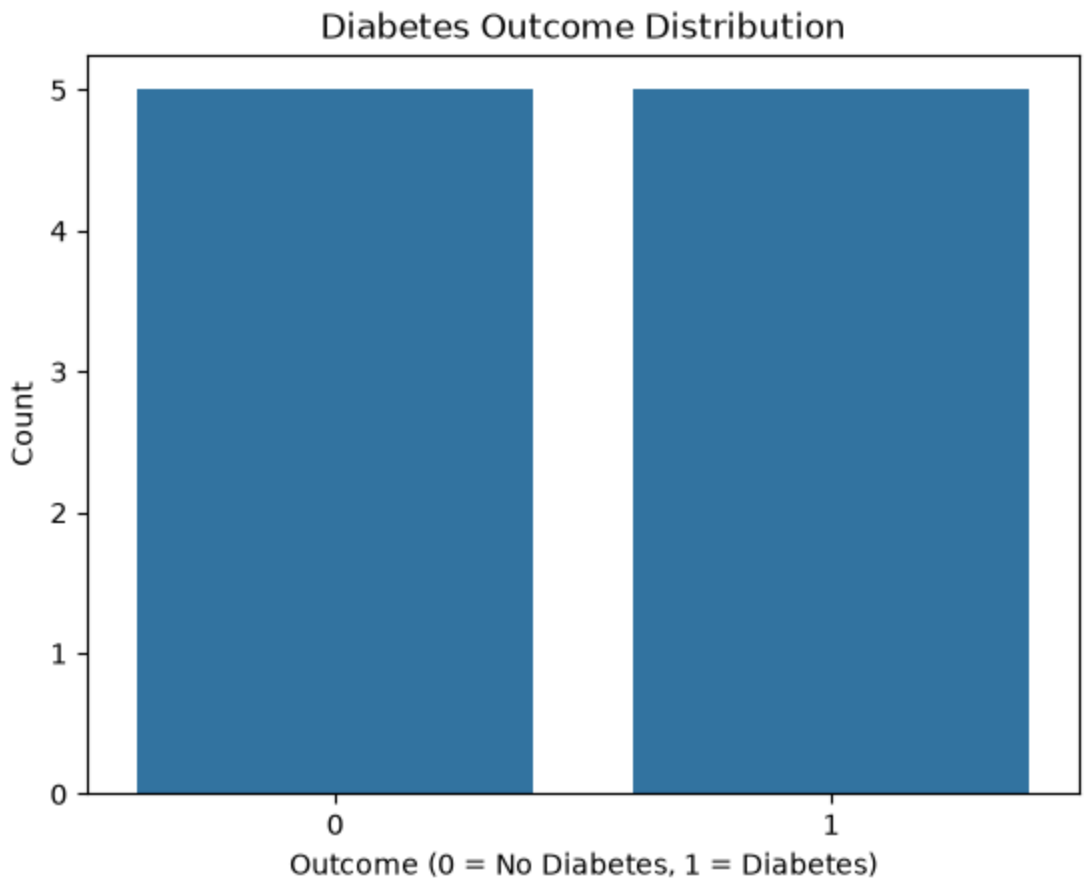
```
Missing Values:
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64

Statistical Summary:
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count    10.000000   10.000000      10.000000      10.000000   10.000000
mean      4.000000  125.800000      59.400000      19.900000   89.300000
std       3.265986   40.449694      25.051946      17.978073  169.937276
min       0.000000   78.000000       0.000000       0.000000    0.000000
25%       1.250000   94.250000      53.500000       0.000000    0.000000
50%       3.500000  115.500000      66.000000      26.000000    0.000000
75%       5.750000  145.250000      71.500000      34.250000   92.500000
max      10.000000  197.000000      92.000000      45.000000  543.000000

             BMI  DiabetesPedigreeFunction        Age    Outcome
count  10.000000                  10.00000  10.000000  10.000000
mean   31.470000                   0.50370  33.500000   0.500000
std     6.050537                   0.65616  10.102255   0.527046
min    23.300000                   0.13400  21.000000   0.000000
25%    26.975000                   0.17300  29.250000   0.000000
50%    30.750000                   0.22450  30.500000   0.500000
75%    34.875000                   0.55800  32.750000   1.000000
max    43.100000                   2.28800  53.000000   1.000000

Number of Duplicate Rows:
0
```


Correlation Heatmap of Diabetes Dataset
Pregnancies 1.00 0.25 -0.37 -0.66 -0.43 -0.17 -0.33 0.04 -0.06
Glucose 0.25 1.00 0.11 0.12 0.53 -0.04 0.21 0.74 0.59
BloodPressure -0.37 0.11 1.00 0.07 0.06 -0.30 -0.20 0.22 -0.01
SkinThickness -0.66 0.12 0.07 1.00 0.64 0.21 0.29 0.51 0.56
Insulin -0.43 0.53 0.06 0.64 1.00 0.12 0.06 0.56 0.44
BMI -0.17 -0.04 -0.30 0.21 0.12 1.00 0.57 0.11 0.14
DiabetesPedigreeFunction -0.33 0.21 -0.20 0.29 0.06 0.57 1.00 0.08 0.47
Age 0.04 0.74 0.22 0.51 0.56 0.11 0.08 1.00 0.55
Outcome -0.06 0.59 -0.01 0.56 0.44 0.14 0.47 0.55 1.00
Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6


Diabetes Outcome Distribution
Count
0
1
2
3
4
5
0
1
Outcome (0 = No Diabetes, 1 = Diabetes)

In [ ]:

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

# Create sample website traffic data
np.random.seed(42)

dates = pd.date_range(start='2025-01-01', periods=120, freq='D')
page_views = (
    500
    + np.arange(120) * 2
    + 50 * np.sin(2 * np.pi * np.arange(120) / 7)
    + np.random.normal(0, 20, 120)
)

# Add a few traffic spikes
page_views[40] += 200
page_views[85] += 250

data = pd.DataFrame({
    'Date': dates,
    'Page_Views': page_views
})

data.set_index('Date', inplace=True)

# Display the dataset
print("Website Traffic Data:")
print(data.head())

# 1. Time Series Decomposition
decomposition = seasonal_decompose(
    data['Page_Views'],
    model='additive',
    period=7
)

decomposition.plot()
plt.suptitle("Time Series Decomposition", y=1.02)
plt.tight_layout()
plt.show()

# 2. Moving Average
data['Page_Views_MA'] = data['Page_Views'].rolling(
    window=7
).mean()

data[['Page_Views', 'Page_Views_MA']].plot(
    figsize=(10, 5)
)
plt.title("Page Views with Moving Average")
plt.xlabel("Date")
plt.ylabel("Page Views")
```

```
plt.show()

# 3. Anomaly Detection using Z-score
mean = data['Page_Views'].mean()
std = data['Page_Views'].std()

data['Page_Views_Z'] = (
    data['Page_Views'] - mean
) / std

anomalies = data[data['Page_Views_Z'].abs() > 3]

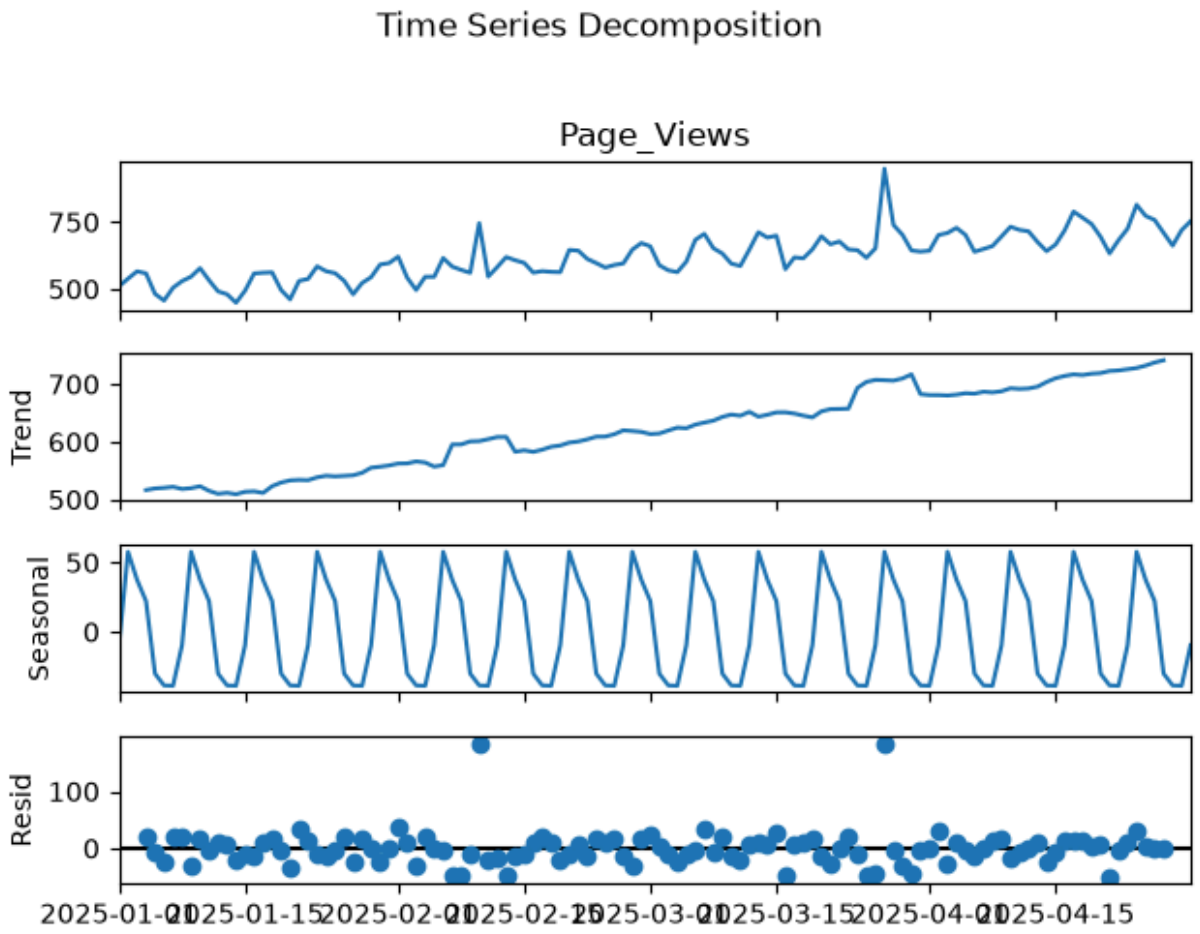
print("\nDetected Anomalies:")
print(anomalies[['Page_Views', 'Page_Views_Z']])

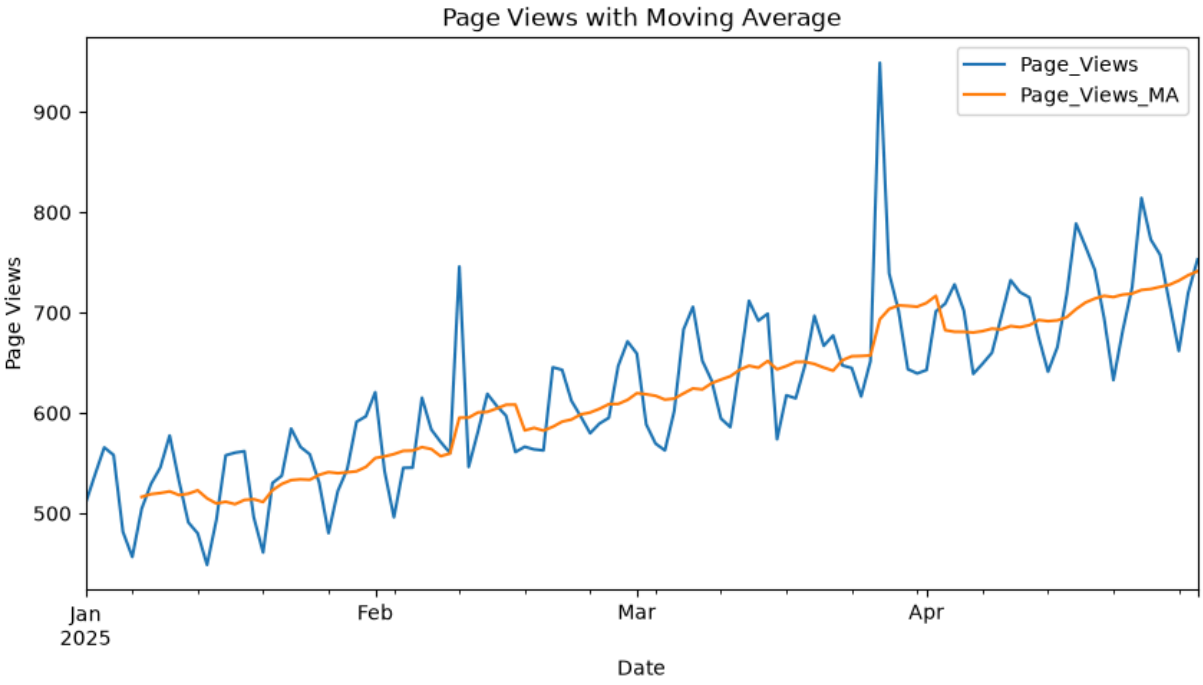
# Plot anomalies
plt.figure(figsize=(10, 5))
plt.plot(
    data.index,
    data['Page_Views'],
    label='Page Views'
)

plt.scatter(
    anomalies.index,
    anomalies['Page_Views'],
    color='red',
    label='Anomalies'
)

plt.title("Website Traffic Anomaly Detection")
plt.xlabel("Date")
plt.ylabel("Page Views")
plt.legend()
plt.show()
```

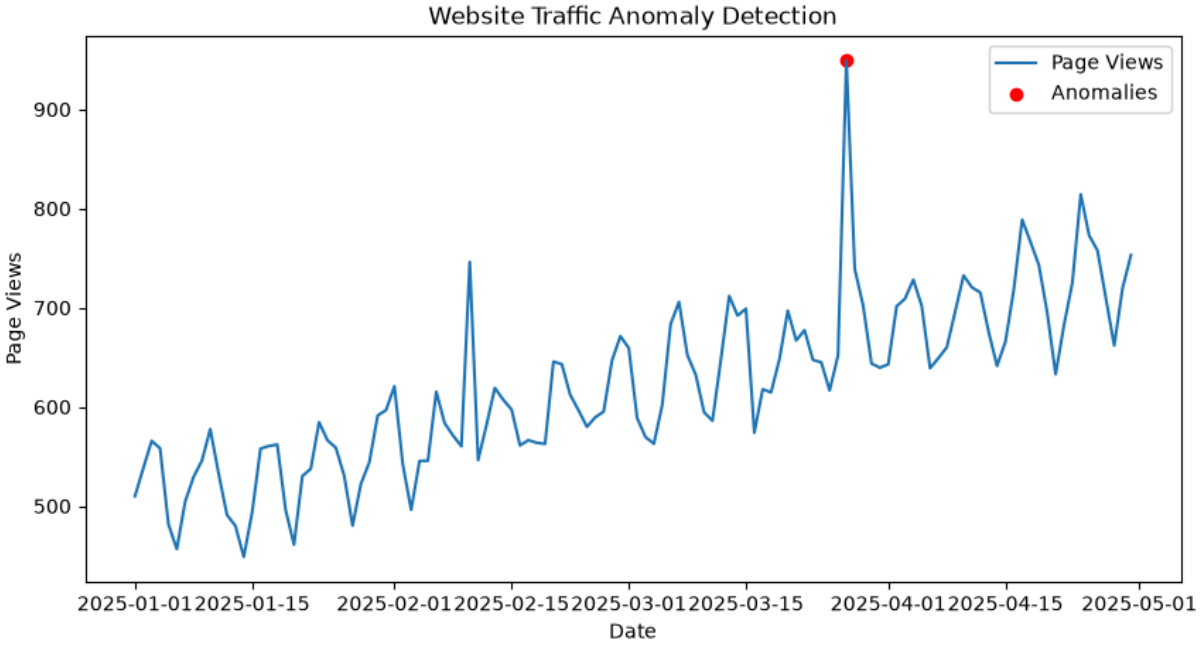
```
Website Traffic Data:
            Page_Views
Date
2025-01-01  509.934283
2025-01-02  538.326288
2025-01-03  565.700166
2025-01-04  558.154784
2025-01-05  481.622746
```





Detected Anomalies:

Date	Page_Views	Page_Views_Z
2025-03-27	949.056433	3.823627


Website Traffic Anomaly Detection
Page Views
Anomalies
900
800
700
600
500
Page Views
2025-01-01
2025-01-15
2025-02-01
2025-02-15
2025-03-01
2025-03-15
2025-04-01
2025-04-15
2025-05-01
Date

In [ ]:

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
```

In [2]:
```
population_mean = 50
population_std = 10
population_size = 100000
population = np.random.normal(population_mean, population_std, population_size)
```

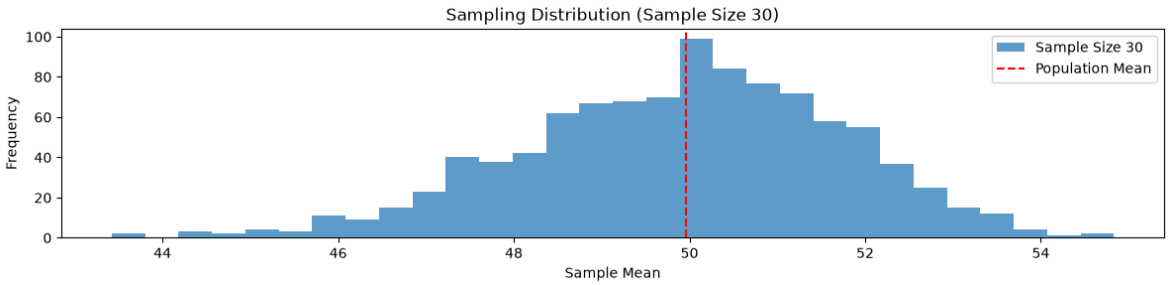
In [4]:
```
sample_sizes = [30, 50, 100] # different sample sizes to consider
num_samples = 1000 # number of samples for each sample size
sample_means = {}

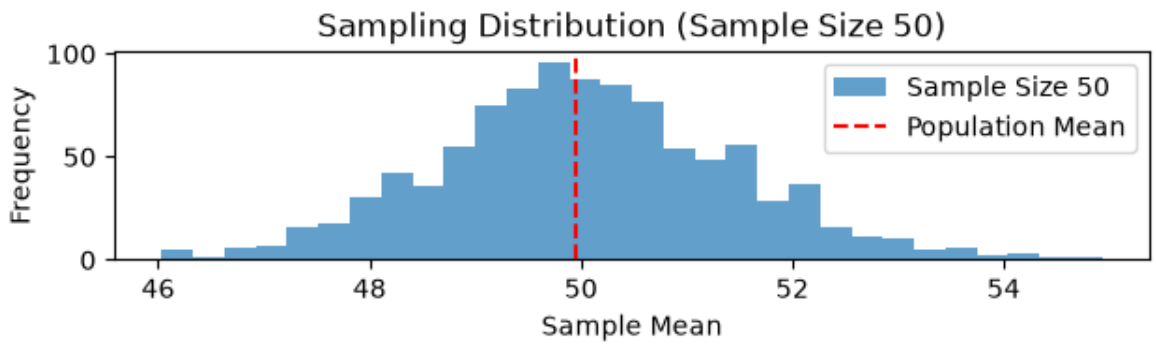
for size in sample_sizes:
    sample_means[size] = []
    for _ in range(num_samples):
        sample = np.random.choice(population, size=size, replace=False)
        sample_means[size].append(np.mean(sample))
```

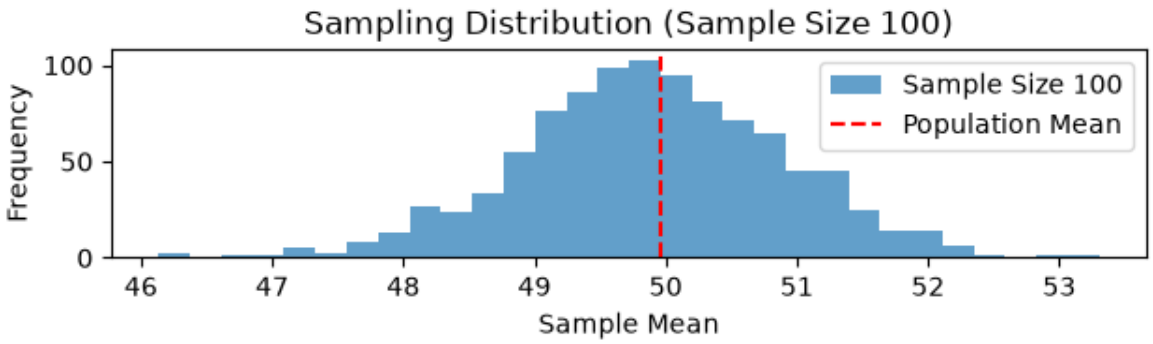
In [7]:
```
plt.figure(figsize=(12, 8))

for i, size in enumerate(sample_sizes):
    plt.subplot(len(sample_sizes), 1, i+1)
    plt.hist(sample_means[size], bins=30, alpha=0.7, label=f'Sample Size {size}'
    plt.axvline(np.mean(population), color='red', linestyle='dashed', linewidth=
    plt.title(f'Sampling Distribution (Sample Size {size})')
    plt.xlabel('Sample Mean')
    plt.ylabel('Frequency')
    plt.legend()

    plt.tight_layout()
    plt.show()
```






Sampling Distribution (Sample Size 100)
Frequency
100
50
0
46
47
48
49
50
51
52
53
Sample Mean
Sample Size 100
Population Mean

In [ ]:

In [1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
```

In [3]:
```
data=pd.read_csv('Iris.csv')
data
```

Out[3]:

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Virginica
146	6.3	2.5	5.0	1.9	Virginica
147	6.5	3.0	5.2	2.0	Virginica
148	6.2	3.4	5.4	2.3	Virginica
149	5.9	3.0	5.1	1.8	Virginica

150 rows × 5 columns

In [4]:
```
data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    str    
dtypes: float64(4), str(1)
memory usage: 7.2 KB
```

In [5]:
```
data.describe()
```

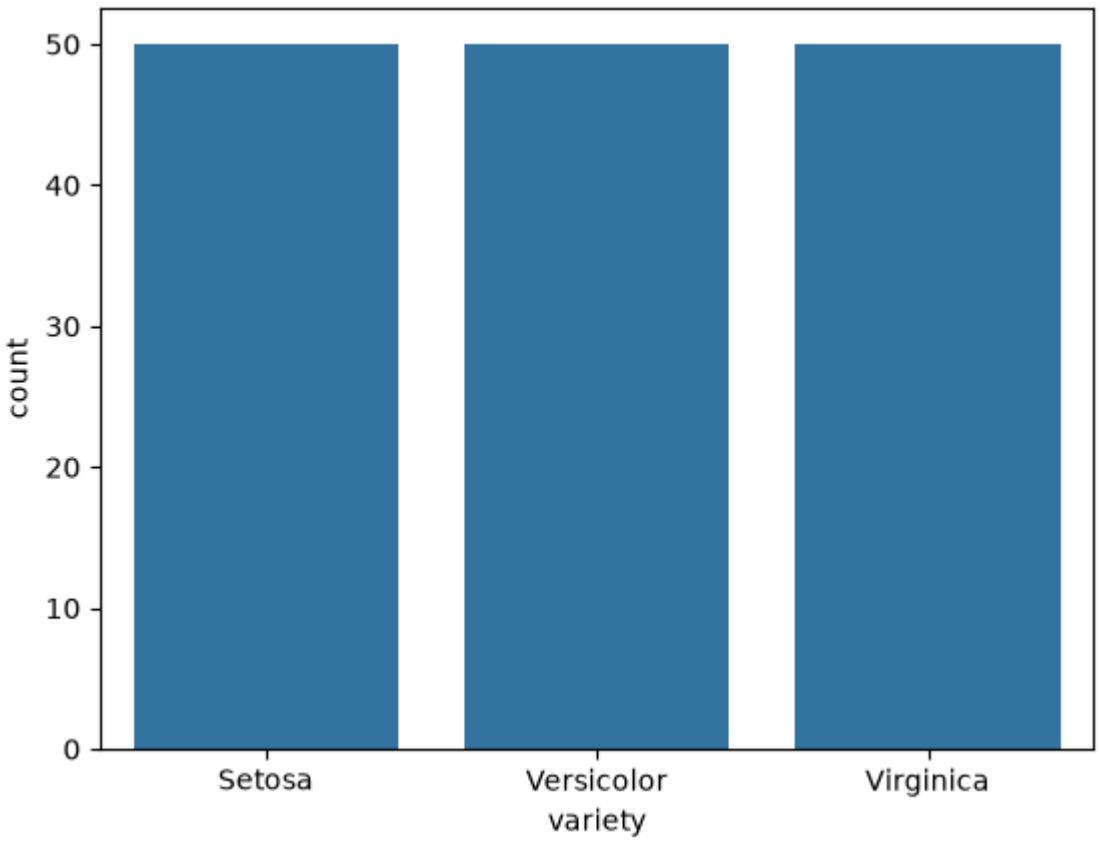
Out[5]:

	sepal.length	sepal.width	petal.length	petal.width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

In [6]:
```
data.value_counts('variety')
```

Out[6]:
```
variety
Setosa        50
Versicolor    50
Virginica     50
Name: count, dtype: int64
```

In [7]:
```
sns.countplot(x='variety',data=data,)
plt.show()
```



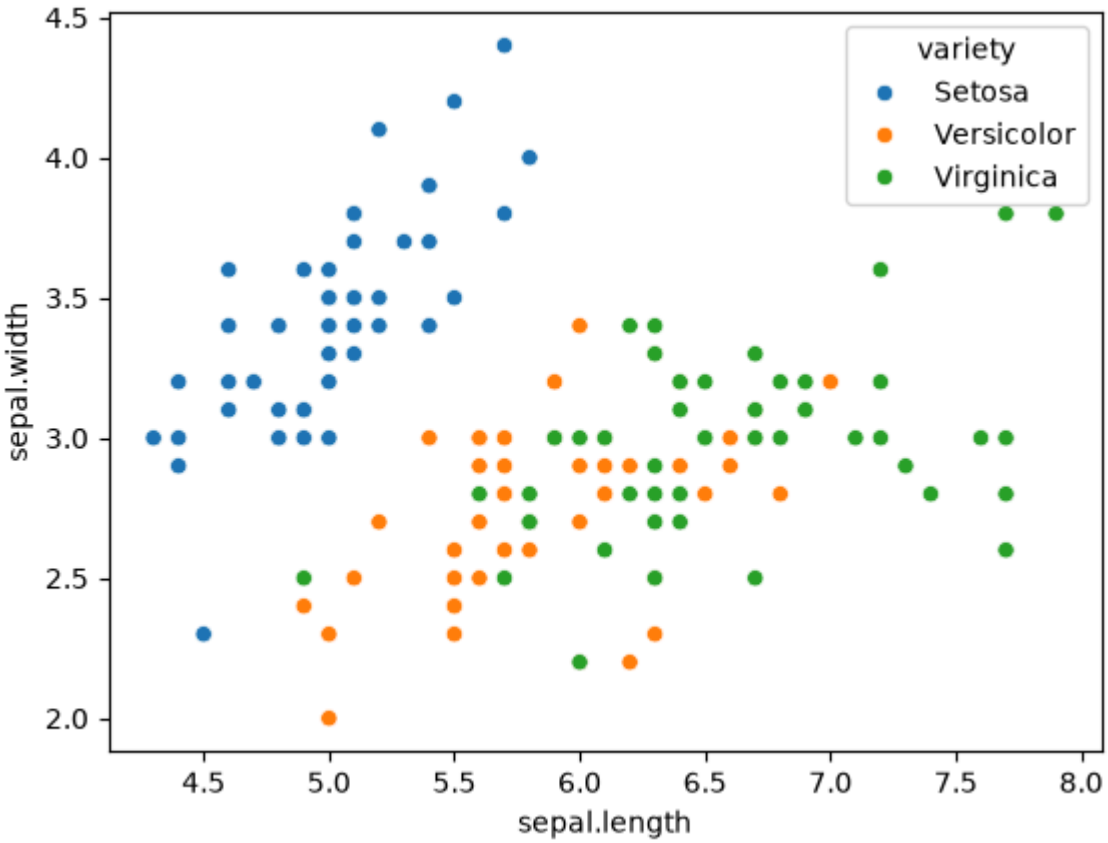
In [8]:
```
dummies=pd.get_dummies(data.variety)
FinalDataset=pd.concat([pd.get_dummies(data.variety),data.iloc[:,[0,1,2,3]]],axi
FinalDataset.head()
```

Out[8]:

	Setosa	Versicolor	Virginica	sepal.length	sepal.width	petal.length	petal.width
0	True	False	False	5.1	3.5	1.4	0.2
1	True	False	False	4.9	3.0	1.4	0.2
2	True	False	False	4.7	3.2	1.3	0.2
3	True	False	False	4.6	3.1	1.5	0.2
4	True	False	False	5.0	3.6	1.4	0.2

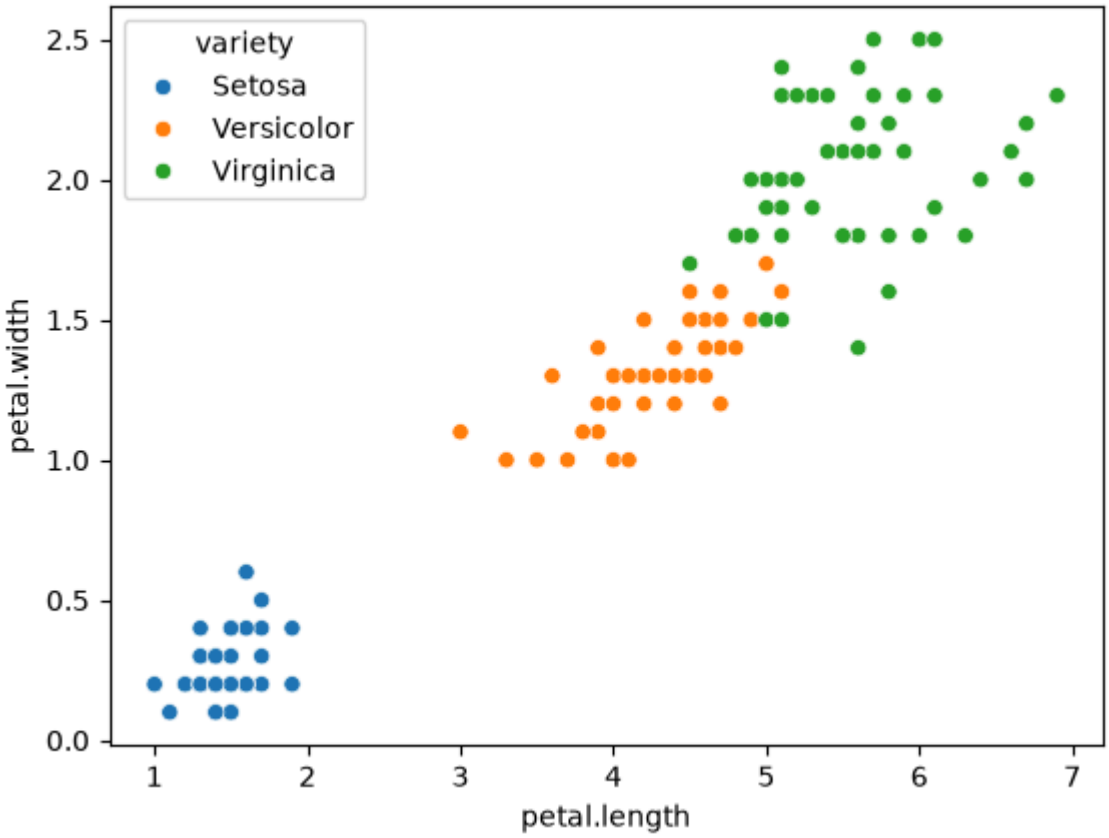
In [11]:
```
sns.scatterplot(x='sepal.length',y='sepal.width',hue='variety',data=data)
```

Out[11]:
```
<Axes: xlabel='sepal.length', ylabel='sepal.width'>
```

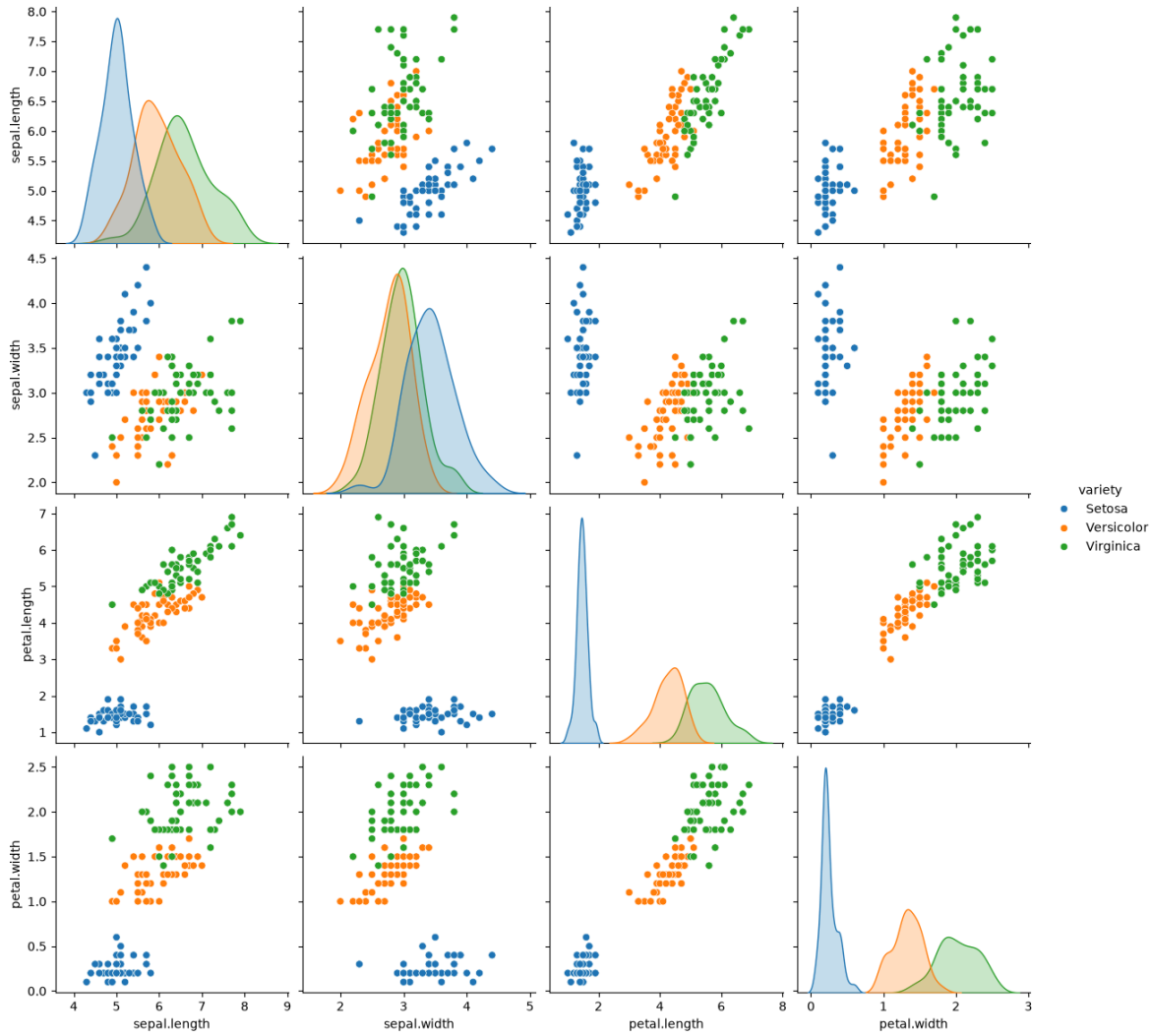


In [13]:
```
sns.scatterplot(x='petal.length',y='petal.width',hue='variety',data=data)
```

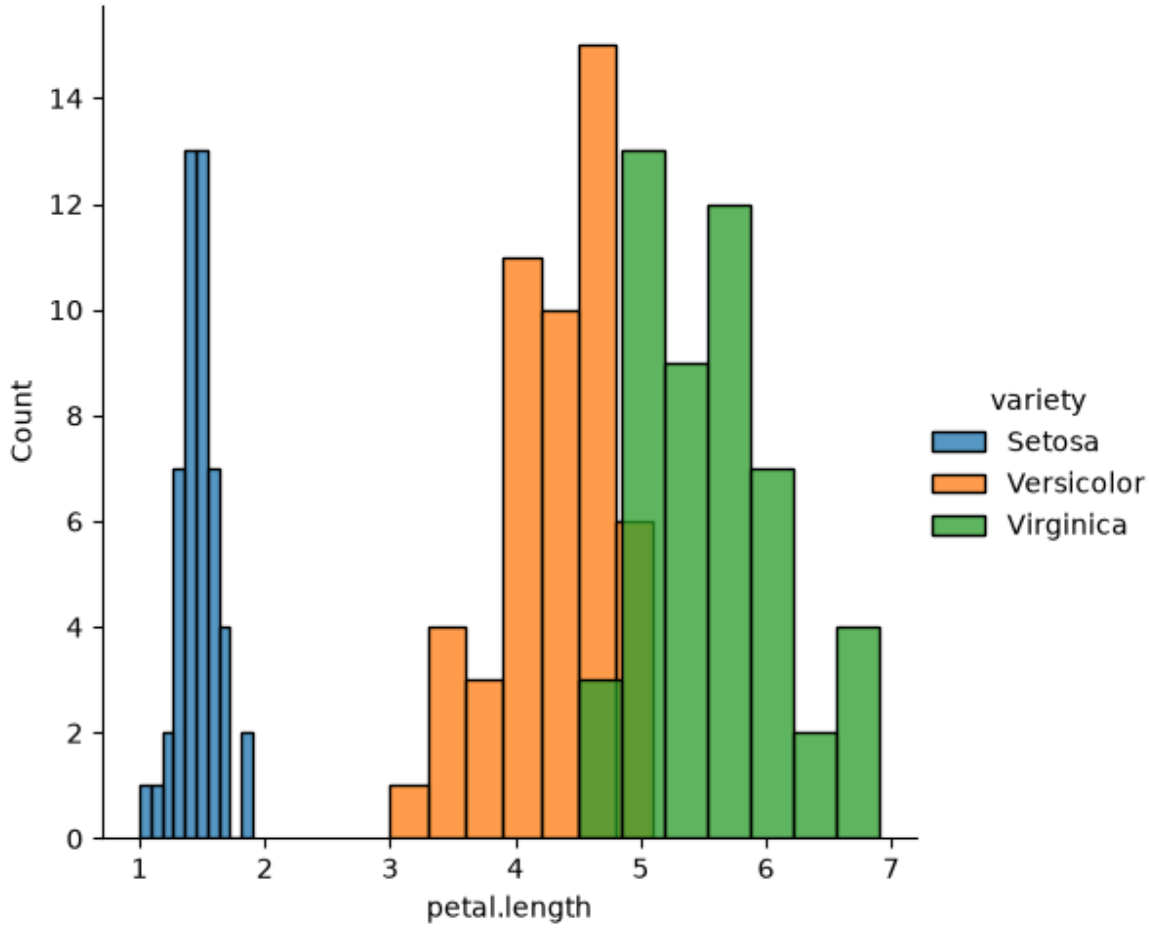
Out[13]:
```
<Axes: xlabel='petal.length', ylabel='petal.width'>
```



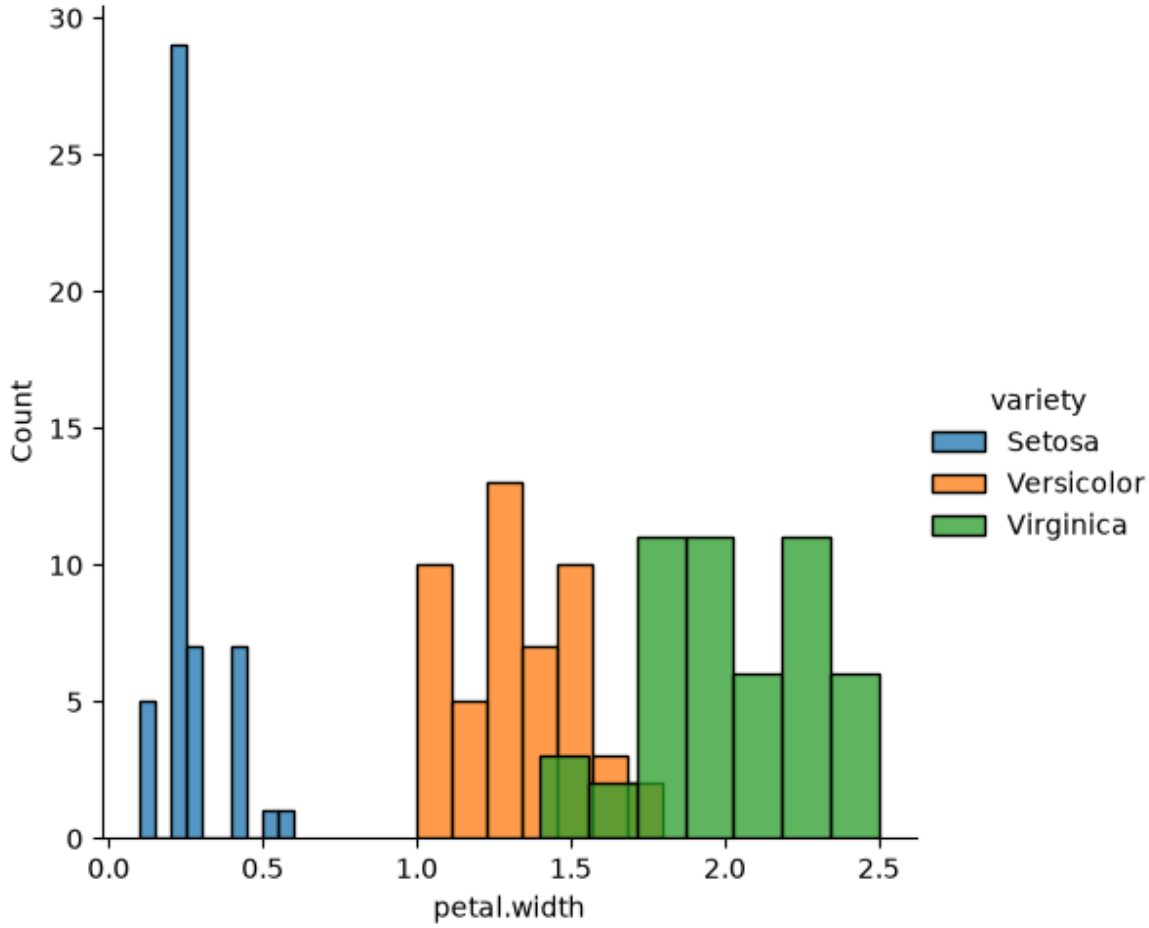
In [14]:
```
sns.pairplot(data,hue='variety',height=3);
```



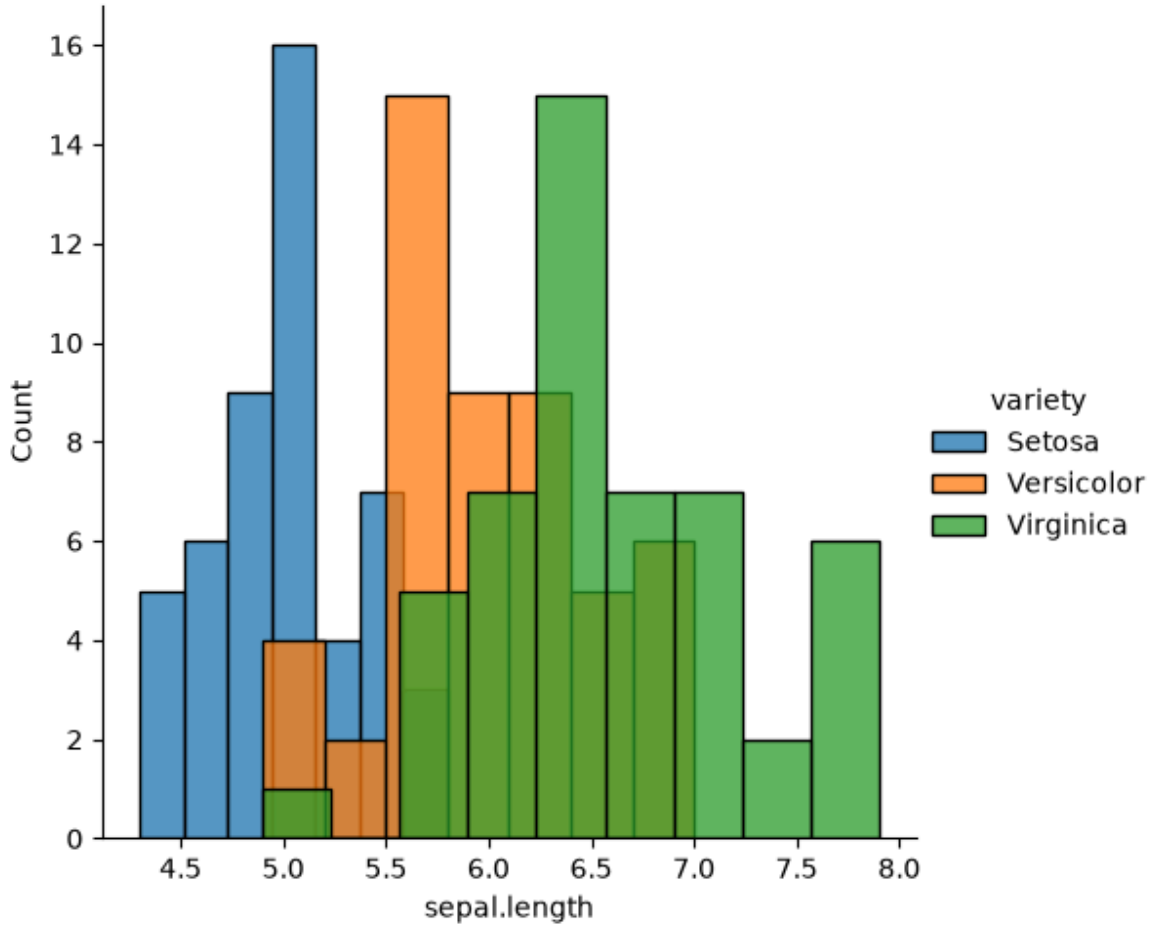
In [17]:
```
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'petal.length').add_
```



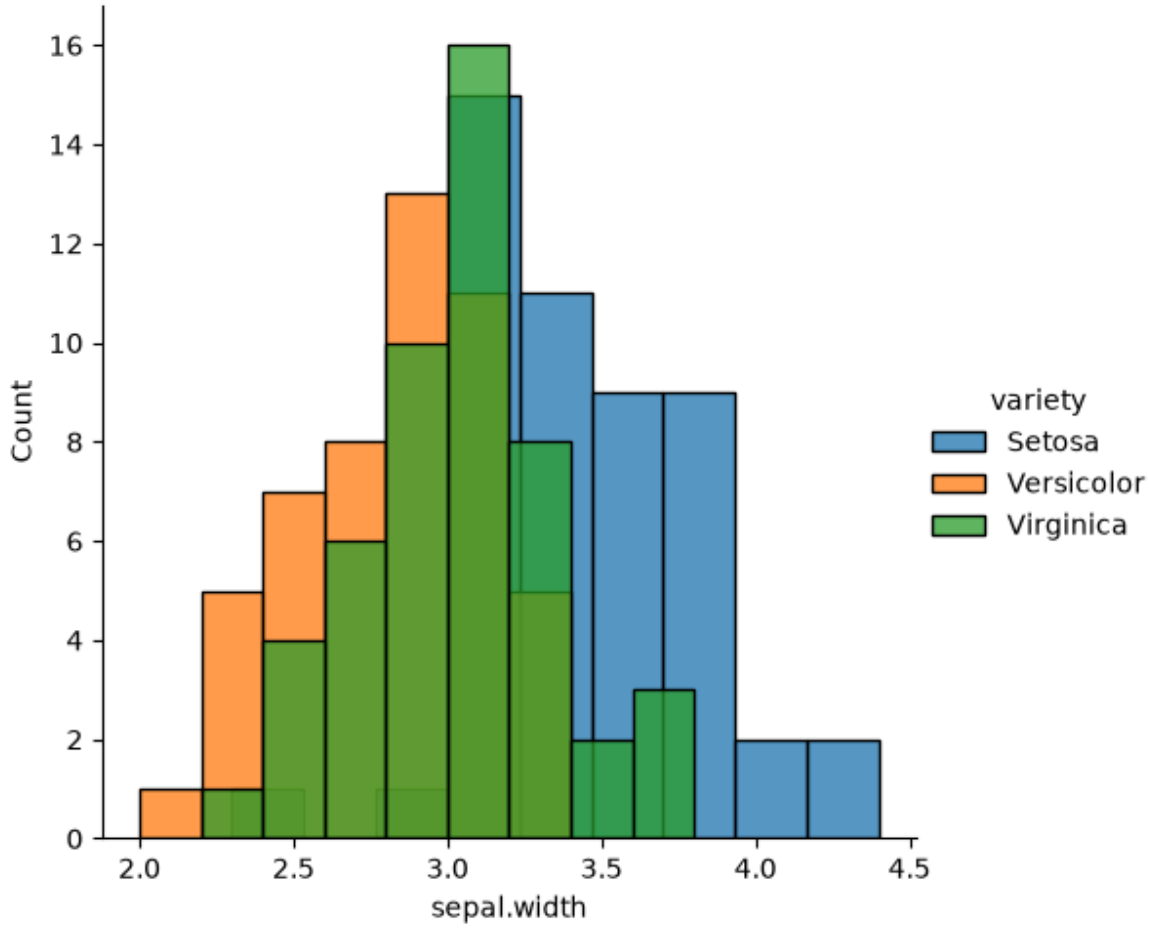
In [19]:
```
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'petal.width').add_l
plt.show();
```



In [20]:
```
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'sepal.length').add_
plt.show();
```



In [21]:
```
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'sepal.width').add_l
plt.show();
```


16
14
12
10
8
6
4
2
0
Count
2.0
2.5
3.0
3.5
4.0
4.5
sepal.width
variety
Setosa
Versicolor
Virginica

In [ ]:

In [1]:
```
import numpy as np
array=np.random.randint(1,100,9)
array
```

Out[1]:
```
array([39, 86, 40,  1, 99, 41,  8, 42, 97], dtype=int32)
```

In [2]:
```
np.sqrt(array)
```

Out[2]:
```
array([6.244998  , 9.2736185 , 6.32455532, 1.        , 9.94987437,
       6.40312424, 2.82842712, 6.4807407 , 9.8488578 ])
```

In [3]:
```
array.ndim
```

Out[3]:
```
1
```

In [4]:
```
new_array=array.reshape(3,3)
new_array
```

Out[4]:
```
array([[39, 86, 40],
       [ 1, 99, 41],
       [ 8, 42, 97]], dtype=int32)
```

In [5]:
```
new_array.ndim
```

Out[5]:
```
2
```

In [6]:
```
new_array.ravel()
```

Out[6]:
```
array([39, 86, 40,  1, 99, 41,  8, 42, 97], dtype=int32)
```

In [7]:
```
newm=new_array.reshape(3,3)
```

In [8]:
```
newm
```

Out[8]:
```
array([[39, 86, 40],
       [ 1, 99, 41],
       [ 8, 42, 97]], dtype=int32)
```

In [9]:
```
newm[2,1:3]
```

Out[9]:
```
array([42, 97], dtype=int32)
```

In [10]:
```
newm[1:2,1:3]
```

Out[10]:
```
array([[99, 41]], dtype=int32)
```

In [11]:
```
new_array[0:3,0:0]
```

Out[11]:
```
array([], shape=(3, 0), dtype=int32)
```

In [12]:
```
new_array[0:2,0:1]
```

Out[12]:
```
array([[39],
       [ 1]], dtype=int32)
```

In [13]:
```
new_array[0:3,0:1]
```

Out[13]:
```
array([[39],
       [ 1],
       [ 8]], dtype=int32)
```

In [14]:
```
new_array[1:3]
```

Out[14]:
```
array([[ 1, 99, 41],
       [ 8, 42, 97]], dtype=int32)
```

In [ ]:

In [1]:

```python
import numpy as np
import pandas as pd

# 1. Create a DataFrame from a list
data = [
    [1, 'Smith', 50000],
    [2, 'Jones', 60000]
]

df = pd.DataFrame(data)
df.columns = ['Empd', 'Name', 'Salary']

print("Employee DataFrame:")
print(df)

# 2. Display DataFrame information
print("\nDataFrame Information:")
df.info()

# 3. Create an employee dataset
employee_data = {
    'emp id': [1, 2, 3, 4, 5, 6, 7],
    'name': [
        'SREE VARSSINI K S',
        'SREEMATHI B',
        'SREYA G',
        'SREYASKARI MULLAPUDI',
        'SRI AKASH U G',
        'SRI HARSHAVARDHANAN R',
        'SRI HARSHAVARDHANAN R'
    ],
    'salary': [5000, 6000, 7000, 5000, 8000, 3000, 6000]
}

df = pd.DataFrame(employee_data)

# 4. Display the first and last five rows
print("\nFirst Five Rows:")
print(df.head())

print("\nLast Five Rows:")
print(df.tail())

# 5. Display DataFrame information
print("\nEmployee DataFrame Information:")
df.info()

# 6. Select the salary column
print("\nSalary Column:")
print(df['salary'])

# 7. Calculate statistical measures
print("\nMean Salary:", df['salary'].mean())
print("Median Salary:", df['salary'].median())
```

```python
print("Mode Salary:", df['salary'].mode().tolist())
print("Salary Variance:", df['salary'].var())
print("Salary Standard Deviation:", df['salary'].std())

# 8. Display numerical summary
print("\nStatistical Summary:")
print(df.describe())

# 9. Display summary of all columns
print("\nComplete Summary:")
print(df.describe(include='all'))

# 10. Display column names
print("\nColumn Names:")
print(df.columns)

# 11. Convert DataFrame into a NumPy array
emp_array = df.values
print("\nEmployee Array:")
print(emp_array)

# 12. Create a DataFrame from the NumPy array
employee_DF = pd.DataFrame(emp_array, columns=df.columns)

print("\nFinal Employee DataFrame:")
print(employee_DF)
```

```
Employee DataFrame:
   Empd   Name  Salary
0     1  Smith   50000
1     2  Jones   60000

DataFrame Information:
<class 'pandas.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      str
 2   Salary  2 non-null      int64
dtypes: int64(2), str(1)
memory usage: 190.0 bytes

First Five Rows:
   emp id                  name  salary
0       1     SREE VARSSINI K S    5000
1       2           SREEMATHI B    6000
2       3               SREYA G    7000
3       4  SREYASKARI MULLAPUDI    5000
4       5         SRI AKASH U G    8000

Last Five Rows:
   emp id                   name  salary
2       3                SREYA G    7000
3       4   SREYASKARI MULLAPUDI    5000
4       5          SRI AKASH U G    8000
5       6  SRI HARSHAVARDHANAN R    3000
6       7  SRI HARSHAVARDHANAN R    6000

Employee DataFrame Information:
<class 'pandas.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   emp id  7 non-null      int64
 1   name    7 non-null      str
 2   salary  7 non-null      int64
dtypes: int64(2), str(1)
memory usage: 410.0 bytes

Salary Column:
0    5000
1    6000
2    7000
3    5000
4    8000
5    3000
6    6000
Name: salary, dtype: int64

Mean Salary: 5714.285714285715
```

```
Median Salary: 6000.0
Mode Salary: [5000, 6000]
Salary Variance: 2571428.5714285714
Salary Standard Deviation: 1603.5674514745463

Statistical Summary:
         emp id       salary
count  7.000000     7.000000
mean   4.000000  5714.285714
std    2.160247  1603.567451
min    1.000000  3000.000000
25%    2.500000  5000.000000
50%    4.000000  6000.000000
75%    5.500000  6500.000000
max    7.000000  8000.000000

Complete Summary:
          emp id                    name       salary
count   7.000000                       7     7.000000
unique       NaN                       6          NaN
top          NaN  SRI HARSHAVARDHANAN R          NaN
freq         NaN                       2          NaN
mean    4.000000                     NaN  5714.285714
std     2.160247                     NaN  1603.567451
min     1.000000                     NaN  3000.000000
25%     2.500000                     NaN  5000.000000
50%     4.000000                     NaN  6000.000000
75%     5.500000                     NaN  6500.000000
max     7.000000                     NaN  8000.000000

Column Names:
Index(['emp id', 'name', 'salary'], dtype='str')

Employee Array:
[[1 'SREE VARSSINI K S' 5000]
 [2 'SREEMATHI B' 6000]
 [3 'SREYA G' 7000]
 [4 'SREYASKARI MULLAPUDI' 5000]
 [5 'SRI AKASH U G' 8000]
 [6 'SRI HARSHAVARDHANAN R' 3000]
 [7 'SRI HARSHAVARDHANAN R' 6000]]

Final Employee DataFrame:
  emp id                   name salary
0      1      SREE VARSSINI K S   5000
1      2            SREEMATHI B   6000
2      3                SREYA G   7000
3      4   SREYASKARI MULLAPUDI   5000
4      5          SRI AKASH U G   8000
5      6  SRI HARSHAVARDHANAN R   3000
6      7  SRI HARSHAVARDHANAN R   6000
```

In [ ]: